

UNITED STATES PATENT AND TRADEMARK OFFICE

BEFORE THE PATENT TRIAL AND APPEAL BOARD

ST. JUDE MEDICAL, CARDIOLOGY DIVISION, INC.
Petitioner

v.

VOLCANO CORPORATION
Patent Owner

Patent No. 7,134,994
Issue Date: November 14, 2006

MULTIPURPOSE HOST SYSTEM FOR INVASIVE CARDIOVASCULAR
DIAGNOSTIC MEASUREMENT ACQUISITION AND DISPLAY

Inter Partes Review No. Unassigned

DECLARATION OF W. ANTHONY MASON

I.	Background and Qualifications.....	1
A.	Professional Background	1
B.	Compensation	3
C.	Summary of Opinions	3
II.	Scientific Background.....	4
A.	The '994 Patent.....	4
B.	Kernel Mode	10
C.	Device Drivers	12
D.	Peripheral interface cards.....	13
E.	Component-Based Arrangement.....	15
III.	A Person of Ordinary Skill in the Art	18
A.	The "Relevant Timeframe"	18
IV.	Claim Construction	20
A.	"Multipurpose host system"	21
B.	"Component based arrangement"	22
C.	"Measurement processing components"	23
D.	"Rendering"	23
E.	"Kernel mode drivers"	23
F.	"Processed sensor data"	26
G.	"Peripheral interface card"	30
H.	"Multi-mode graphical user interface host"	32
I.	"Diagnostic measurement user interfaces"	36
J.	"External input signal bus interface"	36
K.	"Generic graphical interface specification"	38
V.	Legal Considerations	39
VI.	Technology Background.....	41
	1. The Incorporation of Kernel Mode Drivers for Purposes of Extracting Was Obvious	44
VII.	Invalidity Over the Prucka Operator's Manual And Other References.....	47
A.	Background Of The Prucka CardioLab.	47

B.	Discussion Of Obviousness Over The Prucka Manual And Other References.....	49
1.	Claim 1.....	62
2.	Claim 10.....	71
3.	Claim 18.....	72
4.	Claims 11-12 and 20 (see also subheadings below).	73
5.	Claim 11.....	74
6.	Claim 12.....	76
7.	Claim 20.....	78
VIII.	Invalidity Over the '669 Patent And Other References.....	79
A.	Background Of The '669 Patent, SmartFlow And Their Owner, Florence Medical	79
B.	Discussion Of Obviousness Of Claims 1, 10-11 and 18 Over The '669 Patent And Other References.....	82
1.	Claim 1.....	83
2.	Claim 10.....	99
3.	Claim 11.....	100
4.	Claim 18.....	101
C.	Discussion Of Obviousness Of Claims 2, 4-6, 8-9, 12-13, and 15-17 Over The '669 Patent, Krauss 1999, <i>Inside Windows</i> And The SmartFlow Manual.	102
1.	Claim 2.....	104
2.	Claims 4 and 15	105
3.	Claim 5.....	107
4.	Claim 6.....	108
5.	Claim 8.....	108
6.	Claim 9.....	109
7.	Claim 12.....	110
8.	Claim 13.....	111
9.	Claim 16.....	112
10.	Claim 17.....	114
A.	Invalidity Of Dependent Claims 2-9 and 12-17 and 19-20.....	115
1.	Claim 8.....	118
2.	Claim 9.....	119

3.	Claims 2 and 4-6	120
4.	Claim 19.....	124
5.	Claim 3.....	125
6.	Claim 7.....	127
7.	Claim 12.....	128
8.	Claim 13.....	131
9.	Claim 20.....	132
10.	Claim 15.....	132
11.	Claims 16 and 17	133
12.	Claim 14.....	135
IX.	Objective Evidence that the Claims are Obvious	137
A.	The Prucka CardioLab/Mac-Lab system as sold was an instance of simultaneous invention.	138
B.	SmartFlow System	221
C.	EP-WorkMate system	268
X.	Conclusion	305
A.	Possible Modifications or Supplementations to Analyses or Opinions	305

I. Background and Qualifications

1. I have been retained by counsel for St. Jude Medical, Cardiology Division, Inc., St. Jude Medical Systems AB, and St. Jude Medical S.C., Inc. (collectively, "St. Jude") to provide testimony relevant to U.S. Patent No. 7,134,994 (the "'994 patent"), device drivers in computer systems, computer software, the state of the art at the time the '994 patent was filed, the level of ordinary skill in the art, the meaning of certain terms in the '994 patent, prior art to the '994 patent, and the invalidity of the '994 patent.

A. Professional Background

2. I received my undergraduate degree in 1987 from the University of Chicago in mathematics with a computer science concentration. From 1987 through 1989, I was a research staff member at Stanford University, where my research primarily revolved around operating systems, including developing kernel-mode debuggers and device drivers and network protocol drivers (including device drivers and network protocol drivers that operate in the kernel mode) for various operating systems. From 1989 through 1992 I worked for Transarc Corporation, where I was part of a team that designed and developed kernel mode drivers and user mode applications responsible for organizing and managing information. From 1992 through 1994 I worked at FORE Systems, Inc., where I was responsible for creating and managing an engineering team responsible for

developing kernel mode device drivers for supporting network communications devices.

3. Since 1994 I have been Consulting Partner for OSR Open Systems Resources, Inc. In that role I have been the lead designer and developer on file system and file system filter driver development projects for Microsoft's Windows operating systems. I have also been a lead developer and instructor of courseware specific to Windows relating to source code, kernel mode debugging and file system driver development.

4. I am the co-author of Windows NT Device Driver Development (with Peter G. Viscarola) (1998) and lex & yacc (with Doug Brown and John Levine) (1st ed. 1990) & (2d ed. 1992)(Ex. 1044). I am also the author or co-author of over fifty articles relating to computer systems architecture and am the named inventor of four patents in the field. I am currently a member of the Association for Computing Machinery and the USENIX Association.

5. My areas of expertise include systems internal software design, debugging and device drivers used in a number of different operating systems, including Windows and UNIX. In connection with this inter partes review proceeding and a prior litigation between St. Jude Medical and Volcano Corporation (the "prior litigation"), I have reviewed a large amount of literature relating to the integration of invasive cardiovascular sensors into computer

systems, and am familiar with the kinds of sensors available and the motivation for using them in the 1999-2000 timeframe. My *curriculum vitae* is provided as Exhibit 1043.

6. In forming my opinion, I have relied on the '994 patent claims and disclosure, the prosecution history of the '994 patent, the exhibits to the Petition for inter partes review of the '994 patent, literature in the field of cardiovascular sensors and my own experience and expertise in the knowledge of the person of ordinary skill in the relevant art in the timeframe from about 2000 to May 19, 2002 (hereinafter the "relevant timeframe").

B. Compensation

7. I am being compensated at the rate of \$500 per hour for services rendered as an expert witness in this case. No part of my compensation is dependent on the outcome of this proceeding.

C. Summary of Opinions

8. Based upon my review of the relevant materials, it is my opinion that:

9. In relation to the '994 patent, a person with ordinary skill in the art is someone who has a Bachelor's of Science degree in computer science or electrical engineering with an emphasis on computer systems, or equivalent experience, in addition to two or three years applicable programming experience.

10. Claims 1-20 of the '994 patent would have been obvious to a person of ordinary skill in the art of computer systems in the relevant timeframe.¹

11. The subject matter of at least claim 1 was independently used in commercially available systems prior to the filing date (May 20, 2002) of the application leading to the '994 patent.

II. Scientific Background

A. The '994 Patent

12. The '994 patent specification relates primarily to the display of information on a computer screen. The Detailed Description of the Invention begins "[a] multipurpose host system for invasive cardiovascular diagnostic measurement acquisition and display provides an advantage of the prior known systems in regard to its ability to present multiple user display interfaces." '994 patent, col. 4, ll. 53-56. After column 9, line 15, the patent specification primarily describes the arrangement of Graphical User Interface (GUI) components on a display screen.

13. The Detailed Description of the Invention up to column 9, line 15 is a description of the host system. This description relies on elements that were well-

¹ I do not agree that there was any invention present in the '994 patent. Therefore, references to "inventor" and "invention" in this declaration mean "alleged inventor" and "alleged invention".

known prior to the filing date of the application leading to the '215 patent (May 20, 2002). For example, personal computers (col. 4, l. 65), PCI busses and PCI cards (col. 5, ll. 28-32), graphical display interfaces (col. 6, l. 8) and peripherals (col. 6, ll. 42-57) were all well-known in the relevant timeframe. Likewise, software functionality, including kernel mode drivers (col. 7, ll. 26-30) and processing components (COM objects, col. 8, ll. 18-34) were also well-known within the relevant timeframe. With the lone exception of the approximate relation for FFR (which was known, see paragraph 293, below), the '994 patent does not provide any processing algorithms for its measurement processing components.

14. The '994 patent is also not concerned with the internal functioning of cardiovascular sensors, and provides no disclosure concerning how to make or improve a sensor. Rather, the '994 patent presents sensors as modular blackboxes that can be plugged in to an appropriate computer system. The '994 patent states:

"In an embodiment of the invention, the devices that provide the input to the host system 110 are presently used in existing, special purpose processing boxes. This set is exemplary, as those skilled in the art will readily appreciate in view of this disclosure that alternative systems advantageously receive and process such diagnostic inputs as pH, ultrasound and light-based cross-sectional images of a vessel, biochemical markers, light spectrometry for tissue characterization, etc."

'994 patent, col. 5, ll. 10-18 (underlining added).

"It is noted that a wide variety of sensor types are known and the host system 100 is not limited to any particular type of sensor input. To the contrary, the present host system 100 is intended to provide a broad, extensible, multipurpose platform upon which a wide variety of application-specific modules are capable of processing and displaying sensor data rendered by a variety of sensor types and combinations of the sensor types including those described, by way of example herein."

'994 patent, col. 7, ll. 8-16 (underlining added).

15. The '994 patent describes the arrangement of these pieces at a general level, but does not describe how to put the pieces together or how to make the pieces. Rather, the '994 patent is relying on the reader already having the relevant skills. The '994 patent uses the pieces that make up the alleged invention in expected ways, and the pieces operate according to their expected and well-known functions.

16. From column 9, line 16, the '994 patent describes how to visually arrange elements on a display screen. However, the methods for arrangement of information on a display screen in a graphical user interface were well-known by the relevant timeframe. Standard tools available for Microsoft Windows-based PCs allowed for essentially arbitrary presentation of GUI components, including menus, push buttons, radio buttons, checkboxes, frames, graphical display regions

(e.g. for drawing graphs). Development of such a GUI involved little effort once a particular presentation format was chosen.

17. The '994 patent presents the key idea of the invention in the following way:

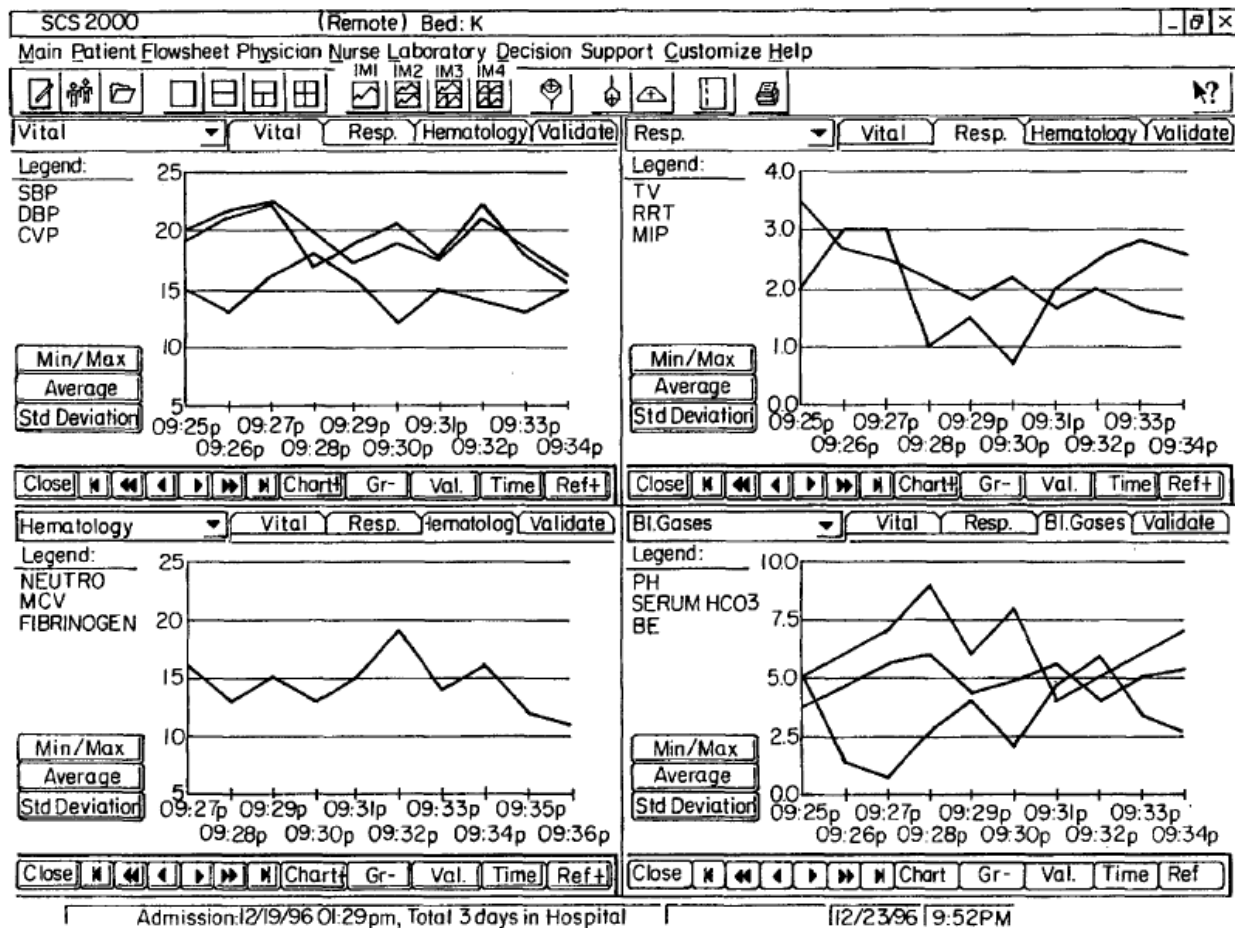
"Notwithstanding the interest to keep equipment and operations simple, there exists a variety of different sensors that are potentially inserted within a human vasculature to diagnose arterial disease (e.g., blockages) and/or monitor vital signs during a medical procedure. The approach taken in the field of interventional cardiac imaging has been to provide multiple, special-purpose monitor consoles. Each monitor type is linked to a particular type of sensor device....The present invention provides addresses a need to provide a flexible, multipurpose host system for processing and displaying signals rendered by invasive cardiovascular sensors to reduce the amount of equipment and complexity of procedures for diagnosing and determining the efficacy of treatment of cardiovascular stenoses."

'994 patent, col. 2, l. 66 – col. 3, l. 30.

18. As of the relevant timeframe, however, there were numerous medical sensor systems integrating data output from multiple sensors into a single, flexible graphical user input display interface.

19. For example, the Schoenberg patent (U.S. Pat. No. 6,322,502) cited by the Examiner during prosecution of the application leading to the '215 patent

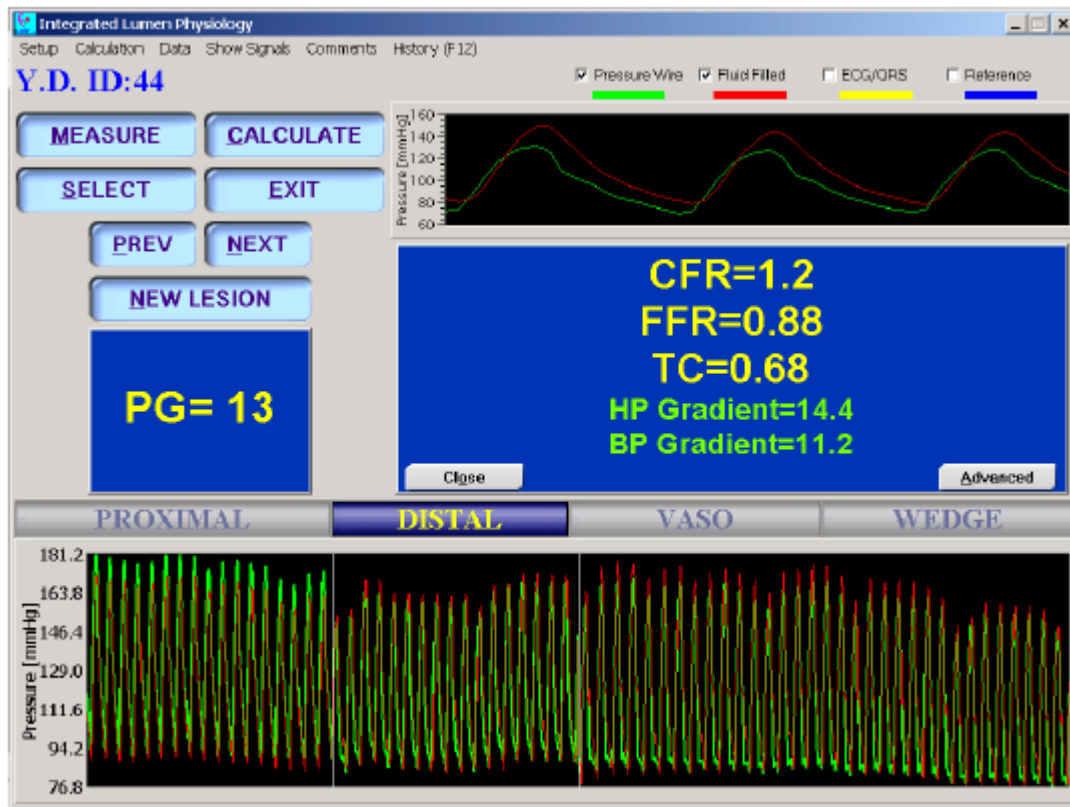
disclosed a graphical user interface presenting the output of numerous sensors, as shown in Fig. 2B:



20. As another example, the Prucka User's Manual (see the section beginning with paragraph 123, below), disclosed a graphical user interface presenting the output of numerous sensors, as shown in Figure 9-17 of the Manual:



21. As a third example, the SmartFlow Operator's Manual (see the section beginning with paragraph 239, below), also disclosed a graphical user interface presenting the output of numerous sensors, as shown in Fig. 17 of the Manual:



22. The claims of the '994 patent do include certain elements that do not directly concern the on-screen display. For example, claim 1 recites a "kernel mode driver", a "peripheral interface card" and a "component-based arrangement". I will provide some technical background for these claim limitations in the next few sections.

B. Kernel Mode

23. Computer processors routinely divide the computer software code being executed into various categories. Generally, these categories correspond to "privilege levels," which refers to the accessibility given to software programs to access specific resources and exploit different levels of functionality of the

computer processor's critical resources. In other words, "privilege levels" relate to the relative level of "trust" given to the specific software code. The most trusted software, such as operating systems and kernel mode device drivers, which I explain below, have privileged access to the most critical resources of the computer.

24. Most processors in the past 20 years have had at least two such levels of privilege. It is the practice in the art to refer to the most privileged level as the "kernel mode" and the least privileged level as the "user mode." Computer software that executes in the kernel mode has privileged access to the operating system's "kernel," or core functions. Computer software code that executes in the kernel mode is generally associated with the computer operating system or specific modules that extend the core capabilities of the operating systems. This distinction is important because kernel mode code operates in the most privileged level of the computer, where the operating system resides, and is therefore capable of causing a malfunction of the computer system (*e.g.*, a "crash" of the operating system itself, which in turn causes all activity on the system to terminate). Peter G. Viscarola & W. Anthony Mason, *WINDOWS NT DEVICE DRIVER DEVELOPMENT*, 51 ("Regardless of the number of modes available in the processor hardware Windows NT only uses two: the "least-privileged" mode, referred to as User mode; and the "most-privileged" mode, referred to as Kernel mode.")(Ex. 1044); *see also*

Microsoft, *MS Windows NT Kernel-mode User and GDI White Paper* (2011) (available at <http://technet.microsoft.com/en-us/library/cc750820.aspx>) (last accessed April 21, 2013) (describing a change in Windows NT 4.0 in which graphics drivers were modified from being *user mode* drivers to being *kernel mode* drivers) (Ex. 1045).

25. This concept of the distinction between kernel mode and user mode was not restricted to Windows and is common to other computer operating systems, such as Linux. *See generally*, Daniel P. Bovet, Professor, University of Rome "Tor Vergata", Speech at the UnixForum Chicago (Mar. 8, 2001) (PowerPoint presentation available at www.uniforum.chi.il.us/slides/linuxkernel/LinuxKernel.ppt) (last accessed April 21, 2013) ("CPUs run either in Kernel Mode or in User Mode," "when in User Mode, some parts of RAM can't be addressed, some instructions can't be executed, and I/O ports can't be accessed" and "when in Kernel Mode, no restriction is put on the program.") (Ex. 1046).

C. Device Drivers

26. Device Drivers are distinct computer software components that interface between a particular piece of computer hardware and the computer's operating system. Modern operating systems (such as Windows®, MacOS®, and Linux) are structured to only "load" the software components that are actually

necessary. By "load," I mean that the operating system copies the software from storage into memory to allow execution of the software by the computer's processor. Having the operating systems work in this fashion allows dynamic extension of the functionality of a computer system because new hardware can be added to a computer by installing the accompanying device driver that interfaces the new hardware with the operating system.

27. Accordingly, device drivers have been defined since at least 1997 as follows: "A driver is a program that interacts with a particular device or special (frequently optional) kind of software. The driver contains the special knowledge of the device or special software interface that programs using the driver do not." Search Storage, "What is a driver?" (June 1997), <http://searchstorage.techtarget.com/definition/driver> (*last accessed April 21, 2013*) (Ex. 1047).

D. Peripheral interface cards

28. Normally, computer systems provide a mechanism by which their functionality can be enhanced by installation of additional hardware resources in the form of circuit boards that are user-installable in slots appropriately configured in the computer. These devices interact with the computer hardware via a standard interface referred to as a "bus," such as the Peripheral Component Interconnect ("PCI") bus commonly found in computers both in the 2000 timeframe and today.

29. In 2002, many desktop-size personal computers had multiple slots for user-installable peripheral interface cards. Computers from standard computer manufacturers were frequently assembled and sold with as few as one and as many as several PCI card slots in which users could install PCI cards to extend the functionality of the computer system.

30. The use of peripheral interface cards to extend the functionality of a computer has been well known in the art since at least the 1990s. For example, the PCI bus and card format was first released in 1992. PCI SIG, PCI Family History (2006) (*available at* http://www.pcisig.com/specifications/PCI_Family_History.pdf) (*last accessed* April 21, 2013) (Ex. 1048).

31. Another common peripheral interface card bus standard was the ISA ("Industry Standard Architecture"). This standard card bus was a popular peripheral interface card bus available on many personal computers in the 2000-2002 timeframe. Weiss, *et al.*, Ex. 1020, p. 80.

32. Because a peripheral interface card is hardware that extends the functionality of a computer, a device driver is required to interface it with the operating system.

E. Component-Based Arrangement

33. When computers were first developed, software code was generally written in procedural languages, starting with assembly language and then evolving into more abstract “high level” languages such as FORTRAN or C. An individual application was typically a single entity, typically written in this linear procedurally structured fashion.

34. By the mid to late 1990s, "object oriented" software code became the dominant programming methodology. An "object" is a combination of data and operations done to that data, which are generally called "methods." By starting with this structure, software designers could more easily separate different parts of the software design to different programmers, so that multiple programmers could work on the implementation of different methods. Accordingly, software applications that are written in object oriented languages generally have a variety of individual software programming elements. Each of these elements can be programmed separately to interact with the other elements that make up a software application. For example, a software application such as typical computer game will normally have hundreds of distinct elements. By combining these elements – either explicitly (at link time) or dynamically (at run time) these software elements are available on the user’s computer.

35. To this end, a typical graphical user interface contains many discreet software elements that make up the visual display. Scroll bars, radio buttons,² drop-down boxes and menu items are examples of separate software elements – distinct programming objects– that interface with the main user application. Changes to either the main user application or these visual functions can occur independently, and additional radio buttons can be added to the interface, for example, by using that same, distinct software element in another part of the program.

36. The term "component based software" is commonly used to describe a software application that is extensible and modular. *See generally*, George T. Heineman & William T. Councill, COMPONENT-BASED SOFTWARE ENGINEERING – PUTTING THE PIECES TOGETHER, 6-14 (2001)(Ex. 1049).

² "Radio button" is a term used in the art to describe a graphical user interface display item that allows a user to select a choice from a predetermined set of options, as below:



37. C++ is an object oriented programming language that was originally released in 1985. To this end, C++ is generally used by software programmers to write component based software applications. Accordingly, a common programming technique using C++ is to use separate software elements (components) for distinct activities, such as processing data versus displaying data in a graphical user interface. The separate processing components in C++ are generally referred to as "classes."

38. Microsoft has created a framework called Microsoft Foundation Class ("MFC"), which is known in the art to be used for building object oriented, component-based applications. In other words, the MFC framework is a tool that facilitates writing portions of object oriented software code, and is frequently used to write component-based software applications. When programming using MFC, it is typical to have distinct components for processing and displaying data. For example, in C++, a "view" class (a display component) is typically derived from a "base" class (such as a processing component). The base class contains the data, and the view class displays the data that it receives from the base class. Accordingly, a device with multiple sources of data would typically have multiple base classes and multiple, corresponding display classes.

39. Microsoft also has a model for implementing component-based software called the "Component Object Model" or "COM". COM is a way to

develop software that provides for the development of small executable components. These components can be linked together to form applications or services for applications. COM is not a programming language, but rather a way of organizing executable code. COM is used with a variety of programming languages, including object-oriented programming languages, like C++. COM provided a way to develop applications such that they are modular and easily extensible through the addition or modification of components. See Dale Rogerson, *Inside COM*, Microsoft Press, 1997, p. 3 (Ex. 1018).

40. By the late 1990s, COM was being used with most applications for Microsoft Windows operating system. Dale Rogerson, *Inside COM*, Microsoft Press, 1997, Ex. 1018, p. 2). The '994 patent discloses the use of COM at, for example, col. 8, ll. 18-35.

III. A Person of Ordinary Skill in the Art

A. The "Relevant Timeframe"

41. I understand that, in patent law, the concept of a person of ordinary skill in the art at the time of the alleged invention was made is relevant, among other things, to the interpretation of claim language and prior art. I have been informed by counsel to assume that the applicable time frame is around May 20, 2002, as that is the date when the application leading to the '994 patent was filed. In the present declaration, I have also considered the state of the art somewhat

before May 20, 2002, approximately in the 2000- to May 19, 2002 timeframe (the "relevant timeframe").

42. I understand that I should consider the following factors in constructing the hypothetical person of ordinary skill in the art: (a) the educational level of the inventor; (b) the types of problems encountered in the art; (c) the prior art solutions to those problems; (d) the rapidity with which innovations were made; (e) the sophistication of the technology; and (f) the educational level of workers in the field. Counsel has also informed me that a person of ordinary skill is a hypothetical person who is presumed to be aware of all pertinent prior art. Finally, I have been informed that not all factors may be present in every case, and that one or more of the factors may predominate.

43. Taking into consideration the factors listed above, in my opinion, a person of ordinary skill in the art of the '994 patent is someone who has a Bachelor's of Science degree in computer science or electrical engineering with an emphasis on computer systems, or comparable experience, in addition to two or three years applicable programming experience. I understand that this level of skill in the art is consistent with the level of skill courts have found in other software patent disputes. *See, e.g., i4i Ltd. P'ship v. Microsoft Corp.*, Case No. 6:07CV113, Memorandum Opinion and Order at 20 (E.D.Tex. Apr. 10, 2008) (dckt 111); *AllVoice Computing PLC v. Nuance Comm'cns, Inc.*, 504 F.3d 1236, 1240 (Fed.

Cir. 2007) (citing cases). When I refer in this report to a person having ordinary skill in the art, I mean a person with this level of skill in the relevant timeframe, and in particular just before May 20, 2002.

IV. Claim Construction

44. I understand that claim construction is the legal process of determining definitions of terms that appear in the claims of a patent. I also understand that in an inter partes review proceeding, a claim term must be given its broadest reasonable construction in light of the specification of the patent in which it appears.

45. Claim 1 of the '994 patent recites:

A multipurpose host system for invasive cardiovascular diagnostic measurement acquisition and display incorporating a component based arrangement, the system comprising:

an external input signal bus interface for receiving data arising from cardiovascular diagnostic measurement sensors;

a plurality of measurement processing components, that operate at a user mode level in the multipurpose host system, for facilitating receiving data of particular sensor types and rendering diagnostic measurement parameter values according to the received data;

a multi-mode graphical user interface host comprising diagnostic measurement user interfaces including display components

corresponding to data output rendered by specified ones of the plurality of measurement processing components;

and one or more kernel mode drivers that extract processed sensor data from a peripheral interface card providing a hardware interface for one or more invasive diagnostic measurement devices.

'994 patent, col. 18 l. 57 – col. 19 l. 9.

46. I understand that, in construing the meaning of the claims, the first source is the claims themselves, followed by the remainder of the specification. I understand that I may take into account the view of a person of ordinary skill in the art in the relevant timeframe.

A. "Multipurpose host system"

47. A person of ordinary skill in the art, having read the patent in its entirety, would understand the term "multipurpose host system" as used in claim 1 of the '994 patent to mean "a personal or other multifunction computer." '994 patent, col. 18, l. 57.

48. The multipurpose host system is described in the patent specification as "by way of example, a personal computer" '994 patent, col. 4 l. 65.

Furthermore, the multipurpose host system is associated with item 100 in Figure 1, which depicts the architecture associated with a personal computer. For example, the figure shows the multipurpose host system as containing a PCI board, a CD-RW (which I understand to mean a compact disk – read/write drive), a network

interface, a graphical interface (which I understand to mean a computer monitor), audio, and a printer. '994 patent, fig. 1. Each of these features were common peripherals available in personal computers in the relevant timeframe.

49. This construction was agreed to in the prior litigation. Joint Claim Construction Brief, Ex. 1021, p. 15.

B. "Component based arrangement"

50. A person of ordinary skill in the art, having read the patent in its entirety, would understand the term "component based arrangement" as used in claim 1 of the '994 patent to mean "a software arrangement that is extensible, modular." The '994 patent equates modularity with "component-based", and then explains that modularity includes the idea of extensibility:

"The user mode level of the host 100 embodies a modular/**component based architecture**. The modular architecture provides a high degree of flexibility in developing and incorporating new sensor types, and corresponding graphical user interfaces, into the multimode host user interface. The user mode processes 206 include an extensible COM based host application 222 that is responsible for presentation of a multiple interface mode graphical user interface (preferably with touch screen functionality)."

'994 patent, col. 7, l. 64 – col. 8, l. 5.

51. This construction was agreed to in the prior litigation. Joint Claim Construction Brief, Ex. 1021, p. 15.

C. "Measurement processing components"

52. A person of ordinary skill in the art, having read the patent in its entirety, would understand the term "measurement processing components" as used in claim 1 of the '994 patent to mean "software components that process sensor data". The fact that the components process sensor data is visible from the claim language itself, as well as, for example, the '994 patent at col. 3, ll. 42-44.

53. This construction was agreed to in the prior litigation. Joint Claim Construction Brief, Ex. 1021, p. 15.

D. "Rendering"

54. A person of ordinary skill in the art, having read the patent in its entirety, would understand the term "rendering" as used in claim 1 of the '994 patent to mean "processing data into a different format including but not limited to drawing". This construction is supported by the specification at col. 5, ll. 25-27, col. 3, ll. 25-28 and 55-57 and col. 6, ll. 17-22.

55. This construction was agreed to in the prior litigation. Joint Claim Construction Brief, Ex. 1021, p. 15.

E. "Kernel mode drivers"

56. A person of ordinary skill in the art, having read the patent in its entirety, would understand the term "kernel mode drivers" as used in claim 1 of the '994 patent to mean "software components that execute on the most privileged level in a computer, which permit a computer system to communicate with a

device". Although the '994 patent does not define this term, this would have been the understanding of a person of ordinary skill in the art during the relevant timeframe, for the reasons given above in paragraphs 23 to 25. The '994 patent does not appear to use the term "kernel mode driver" in an unusual way.

57. A driver is "[a] software component that permits a computer system to communicate with a device." MICROSOFT COMPUTER DICTIONARY 155 ("device driver") (5th ed. 2002)(Ex. 1050).

58. The most protected level (the level with access to all of the computer system's memory) in a computer operating system is referred to as the kernel level. PETER G. VISCAROLA & W. ANTHONY MASON, WINDOWS NT DEVICE DRIVER DEVELOPMENT, (1999) (Ex. 1044).

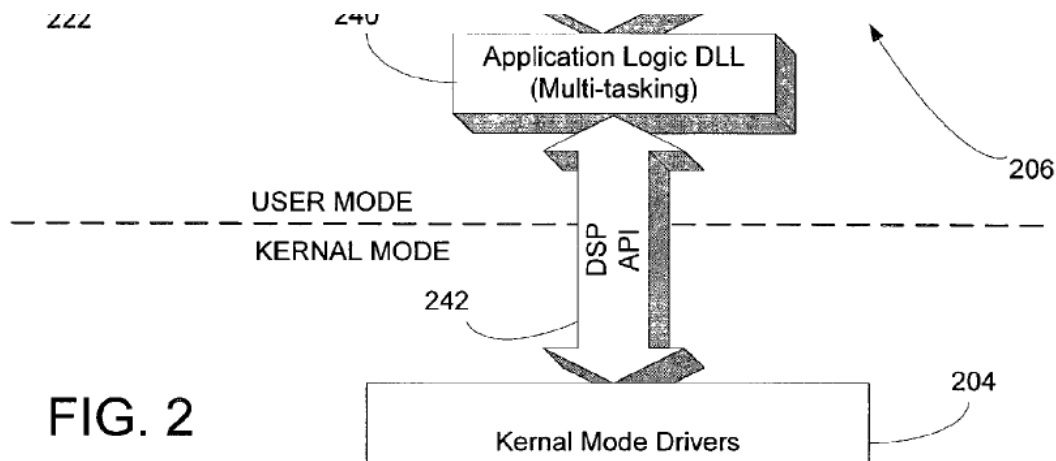
59. For example, the 1998 book *Inside Windows* explains:

"Kernel Mode vs. User Mode. To protect user applications from accessing and/or modifying operating system data, Windows NT uses two *processor access modes* (even if the processor on which Windows NT is running supports more than two): *user mode* and *kernel mode*. User application code runs in user mode, whereas operating system code (such as system services and device drivers) runs in kernel mode. **Kernel mode refers to a mode of execution in a processor that grants access to all system memory and all CPU instructions.** By providing the operating system software with a **higher privilege level** than application software has, the processor provides a necessary

foundation for operating system designer to ensure that a misbehaving application can't disrupt the stability of the system as a whole."

Inside Windows, Ex. 1004, p. 8 (bold and underlining added).

60. The '994 patent emphasizes the use of a multi-privilege-level computer system and distinguishes *kernel mode* drivers from *user mode* components. Compare '994 patent, col. 7, l. 26 (Kernel mode drivers 204 executing on the host 100 . . .") with *id.*, col. 7, ll. 6-7 (" . . . user mode processes executing on the host system 100."). Additionally, Figure 2, which discloses the only embodiment described in the specification, likewise distinguishes between the different levels of privilege in a computer system:



Id., fig. 2 (excerpt).

61. Additionally, in describing the functionality of the kernel mode drivers, the '994 patent specification discusses functions that are unique to a driver executing in the kernel mode. For example, the specification states that the drivers

"respond to interrupts generated by the PCI card," "read-write data from/to PCI configuration registers," and "set up interrupt handling." '994 patent, col. 7, ll. 49-58. Each of these functions is unique to drivers that execute in the kernel mode.

62. This construction was found to be correct by the court in the prior litigation. Amended Claim Construction Order, Ex. 1022, p. 9.

F. "Processed sensor data"

63. The meaning that a person with ordinary skill in the art would have ascribed to the term "processed sensor data" upon reading the '994 patent would have been: "sensor data that has been systematically modified, including by means of analog to digital conversion."

64. The '994 patent specification describes a system in which data from invasive cardiovascular sensors enters a computer system for acquisition and display. *See generally* '994 patent, col. 1 ll. 1-4. The signal enters the computer on a peripheral interface card, and kernel mode drivers extract the data from the peripheral interface card. *Id.*, col. 19 ll. 6-9. The specification describes some embodiments. One embodiment uses a PCI card as the peripheral interface card. *Id.*, col. 5 ll. 25-30; col. 7 ll. 2-3. Another embodiment also adds a digital signal processing ("DSP") chip on the PCI card, to further process the data before the data is extracted in the computer system. *Id.*, col. 7 ll. 17-23; col. 5 ll. 30-39. Another embodiment uses analog signal processing to process the data. *Id.*, col. 6 ll. 25-30.

65. The term "processed" in both data acquisition and in ordinary English means systematically modified or acted upon to produce a desired result. *See, e.g.*, THE RANDOM HOUSE DICTIONARY OF THE ENGLISH LANGUAGE 1542 (2d ed. Unabridged 1987) (Ex. 1052); MCGRAW-HILL DICTIONARY OF SCIENTIFIC AND TECHNICAL TERMS 1676 (4th ed. 1989) (Ex. 1051).

66. The '994 patent describes "processing" consistent with these definitions, since it includes a variety of signal manipulation processes in its definitions and descriptions, including both analog and digital signal processing. For example, the '994 patent refers to "existing, special-purpose *processing* boxes." *Id.*, col. 5 ll. 10-18 (emphasis added). The list of such "processing boxes" includes "systems [that] receive and process such diagnostic inputs as pH, ultrasound and light-based cross-sectional images of a vessel, biochemical markers, light spectrometry for tissue characterization, etc." *Id.* One with ordinary skill in the art of medical device design involving computer systems in May 2002 would know that many such "processing boxes" use analog signal processing. *See, e.g.*, Declaration of Sheri Prucka (the "Prucka Decl.", Ex. 1053) ¶ 20 (Ex. 1053).

67. The '994 patent specification further explains that "processing" includes analog signal processing because it describes means of processing sensor data before it enters the computer system using analog signal processing:

In a particular embodiment of the invention, the host system 100 relies on transducers and external diagnostic instrumentation to (1) process the raw sensor information rendered by transducers/sensors within a patient and (2) provide the information to the host **100** in particular digital or analog formats.

'994 patent, col. 6 ll. 25-30 (emphasis added).

68. Accordingly, the term "processed" as used in the '994 patent is not limited to digital signal processing; rather, the term "processed" as used in the '994 patent includes analog signal processing.

69. The '994 patent also uses the term "processes" to include converting the signal between analog and digital, in addition to FFT, level shifting, normalizing, and scaling:

"The PCI card **112** includes, by way of example, a digital signal processor (DSP) that samples data provided by the communicatively coupled input sensors and processes the sampled data to render digital data in a format expected by higher level components of the host system **100**. Exemplary processes performed by the DSP include: A/D and D/A conversions, FFTs, level shifting, normalizing, and scaling. After processing the data, it is stored in a dual port RAM accessed, via the PCI bus of the host **100**, by higher level application processes executing on the host system **100**."

'994 patent, col. 5 ll. 30-39 (emphasis added).

70. "A/D and D/A conversions" refers to the process of converting the signal from analog to digital. It should be noted that the function of the exemplary DSP chip in the '994 patent is referred to in two separate paragraphs. See '994 patent, col. 5 ll. 25-39 & col. 7 ll. 17-24. In the first paragraph, the system appears to describe the function of the DSP chip if the signal sent into the PCI card is an analog signal. To that end, it describes analog to digital conversion as one of the exemplary "*processes performed by the DSP chip.*" In the second paragraph, the system appears to describe the function of the DSP chip if the signal sent into the PCI card has already been converted to digital. See *id.*, col. 7 ll. 2-4 (referring to digital signal inputs on the PCI card). In that paragraph, the alleged inventors have not disclosed, for example, how the signal is converted to digital, and thereafter lists digital processes.

71. Accordingly, a person with ordinary skill in the art would understand that "processed sensor data" includes sensor data that has been converted between analog and digital formats.

72. The '994 patent also refers to a separate embodiment, in which a *digital* signal processor is included in the computer on the peripheral interface card, and the digital signal processor processes the sensor data. *Id.*, col. 5 ll. 25-39. Accordingly, a person with ordinary skill in the art would also understand that the

term "processed sensor data" includes sensor data that has been processed as a digital signal.

73. A person with ordinary skill in the art would therefore understand that the claim term "processed sensor data" refers to "sensor data that has been systematically modified, including by means of analog to digital conversion," because the word "process" in the patent refers to analog processing, digital processing, or processing by means of conversion between the two.

74. This construction was found to be correct by the court in the prior litigation. Amended Claim Construction Order, Ex. 1022, pp. 9-10.

G. "Peripheral interface card"

75. A person with ordinary skill in the art would understand "peripheral interface card" as used in claim 1 of the '994 patent to mean "a printed circuit board or adapter that can be plugged into a computer to provide added functionality or new capability, which provides specialized services that are not built into the computer, and that interfaces with peripherals."

76. In the computer industry, a "card" is known by those with ordinary skill in the art as:

"a printed circuit board or adapter that can be plugged into a computer to provide added functionality or new capability. These cards provide specialized services, such as mouse support and modem capabilities, that are not built into the computer."

Microsoft Computer Dictionary, Ex. 1050, p. 84 (5th ed. 2002).

77. For example, as disclosed in a patent that is prior art to the '994 patent:

"Continued reductions in the size of electronic devices, such as computers, have required similar reductions in the size of associated hardware components. The size reductions have prompted the establishment of dimensional standards, or "form factors," governing the width, length, and thickness of various hardware components. Many peripheral interface modules, for example, presently are manufactured as card-like packages that conform to "form factors" established by the Personal Computer Memory Card International Association (PCMCIA). The PCMCIA form factors have varying thicknesses, but share common surface areas that approximate the shape and surface area of a credit card. The peripheral interface cards can be plugged into an electronic device, such as a computer, in a modular fashion to provide a variety of functions."

U.S. Patent 5,463,261, col. 1 ll. 12-25 (Ex. 1054).

78. While computer cards, by definition, add functionality, not all computer cards interface with peripherals. As used in the '994 patent, the term "peripheral interface card" therefore means a card (as defined above) that interfaces with peripherals.

79. This construction is consistent with what is disclosed in the '994 patent specification. The only reference to cards in the specification is to

peripheral component interconnect ("PCI") cards, a specific type of peripheral interface card. '994 patent, col. 5 ll. 28-30. A person with ordinary skill in the art would understand "peripheral interface card" to refer to all such printed circuit boards that can be plugged into a computer to provide added functionality, including cards that fit the ISA (Industry Standard Architecture), MCA, PCI, and PCMCIA card slots found in personal computers in the 1980s, 1990s and early 2000s.

80. This construction was found to be correct by the court in the prior litigation. Amended Claim Construction Order, Ex. 1022, p. 10.

H. "Multi-mode graphical user interface host"

81. A person with ordinary skill in the art would understand the term "multi-mode graphical user interface" as used in claim 1 of the '994 patent term "multi-mode graphical user interface host" should be further construed to mean "software that supports a graphical user interface (including windows or portions of windows) for presenting at least two types of diagnostic information (including within the same window)."

82. I understand that the court in the prior litigation construed this term to mean "software that supports a human-computer interface with at least two distinct displays for presenting diagnostic information of at least two types." Amended Claim Construction Order, Ex. 1022, p. 8. I think this definition should be

modified somewhat under the broadest reasonable interpretation of the claims consistent with the specification.

83. A graphical user interface is understood by those familiar with computers to be an interface between a user and a computer system that makes use of input devices other than the keyboard and presentation techniques other than alphanumeric characters. This definition is consistent with the definition provided in the Dictionary of Computing, 215 (4th ed. 1996) (Ex. 1026), which I agree with, and which I understand has been relied upon by courts for the construction of computer-related terms in other cases. *See, e.g., nCube Corp. v. Seachange Intern., Inc.*, 436 F.3d 1317, 1327 (Fed. Cir. 2006) (construing "address"); *Linear Tech. Corp. v. Impala Linear Corp.*, 379 F.3d 1311, 1320 (Fed. Cir. 2004) (construing "circuit").

84. In this case, the claim term requires that said graphical user interface be capable of having multiple "modes." The specification in the '994 patent itself provides information regarding what constitutes a "mode." For example, the '994 patent specification explains that the mode of operation in which the user enters relevant system information (such as customer/institution name, time/data, printer, and local area network connection information) is a "mode" called the "System mode." '994 patent, col. 10 ll. 24-28 & fig. 6. The '994 patent specification goes on to discuss "diagnostic modes of operation," as those modes of operation

associated with pressure, flow, and combination. *Id.*, col. 11 ll. 20-27. The '994 patent discusses diagnostic and other modes of operation elsewhere as well. *See, e.g., id.*, col. 5 ll. 53-58) (" . . . operation of each *mode* is preferably independent of other modes, and each *diagnostic display mode* is driven by a designated set of parameter generation modules . . .") (emphasis added).

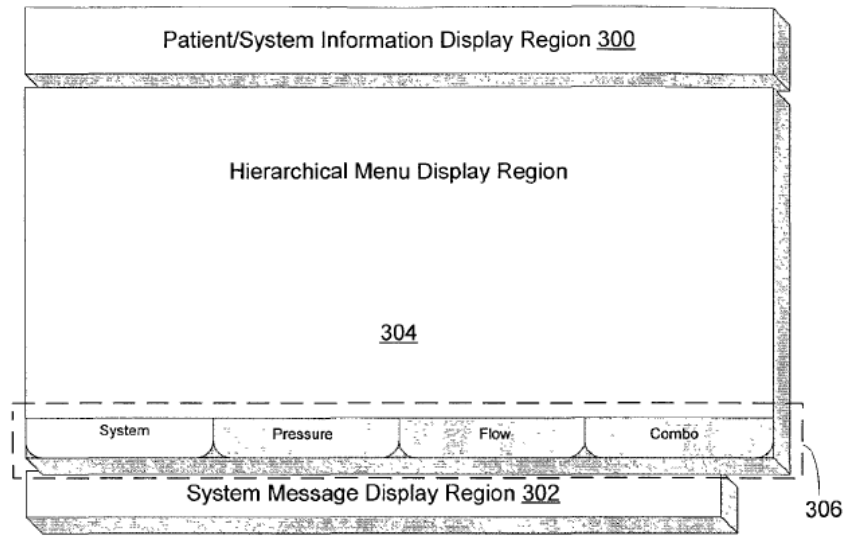
85. The '994 patent specification also discusses the multi-mode graphical user interface *host* as follows:

"The modular architecture provides a high degree of flexibility in developing and incorporating new sensor types, and corresponding graphical user interfaces, into the multimode host user interface."

Id., col. 7 l. 65 – col. 8 l. 1 (emphasis added).

86. This teaches that the multi-mode graphical user interface host is capable of supporting multiple display modes, though not all such display modes need be functional. Instead, the host needs to merely support the ability to add new display modes.

87. For the purpose of this inter partes review proceeding, some explanation of the term "display" as used in the '994 patent should be provided. The '994 patent uses the word "display" generally, to mean a portion of a graphical user interface. For example, Fig. 3 of the '994 patent shows how several "displays" can be arranged within a single window:



88. This is reinforced by the '994 patent's statement that it "supports simultaneous display of multiple distinct types of input signals on a single graphical user interface." '994 patent, col. 5, ll. 46-47. The '994 patent states with respect to Fig. 12, for example, that:

"a **multi-partitioned waveform display** depicts a full wave form graph 1202 as well as **two smaller waveform display** output segment graphs 1204 and 1206 corresponding to the base waveform and peak wave form (under hyperemic conditions)."

'994 patent, col. 13, ll. 60-65 (emphasis added).

89. This indicates that the word "display" is being used to both to describe the entire screen, and to describe portions of the screen directed to types of information.

I. "Diagnostic measurement user interfaces"

90. A person of ordinary skill in the art, having read the patent in its entirety, would understand the term "diagnostic measurement user interfaces" to mean "a portion of a graphical user interface (including windows) for displaying diagnostic information".

91. The '994 patent uses the phrase "diagnostic measurement user interfaces" at col. 3, ll. 52-57, as follows:

"The host system also includes a multi-mode graphical user interface host. The interface host comprises a set of diagnostic measurement user interfaces. The output interfaces are integrated with the processing components and carry out displaying, on a graphical user interface a set of output values corresponding to parameter values rendered by the processing components."

92. This would have indicated to a person of ordinary skill that the diagnostic measurement user interfaces are the portions of the graphical user interface that display diagnostic information.

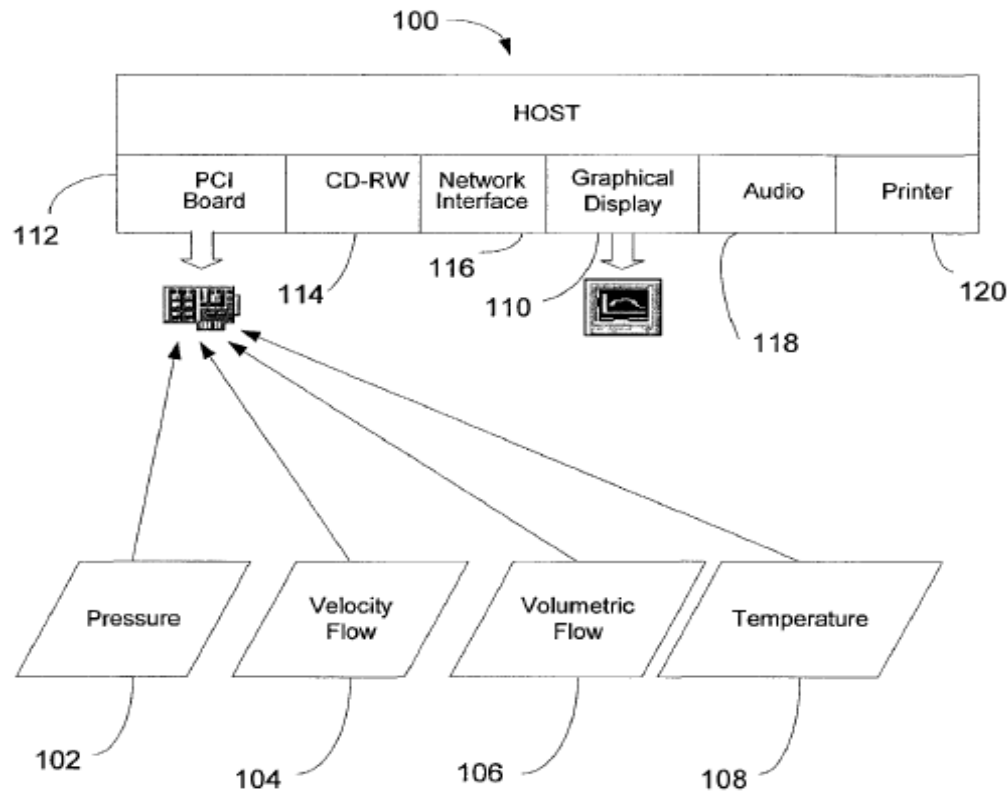
J. "External input signal bus interface"

93. A person of ordinary skill in the art, having read the patent in its entirety, would understand the term "external input signal bus interface" to mean a "connection between a peripheral interface card and a host."

94. The phrase "external input signal interface" (without the word "bus") is used at col. 4, l. 1, in part of a brief description of Fig. 1. The brief description

states that the external input signal interface is "for receiving diagnostic parameter values of multiple types". This function seems to correspond to numeral 112 in

Fig. 1:



95. The term "bus interface" is also used several times in the specification to refer to a PCI bus interface. '994 patent, col. 7, ll. 24, 31, and 34-35. For example, the '994 specification states:

"The kernel mode drivers 204 access PCI registers and ports on the PCI bus interface 202 to extract processed sensor data and to issue control commands to the PCI card 112."

'994 patent, col. 7, ll. 33-36 (emphasis added).

96. From the foregoing, it would have been apparent to a person of ordinary skill that the external input signal bus interface is used for communication between the PC and sensor signals arising externally from the PC, via the peripheral interface card. The fact that the phrase "external input signal bus interface" is followed in the claim by "for receiving data arising from cardiovascular diagnostic measurement sensors" implies that the "input signals" are signals from the sensors. The "bus" is the physical medium carrying the signals, according to the ordinary meaning of that term. MacMillan IT Dictionary, Ex. 1017 p. 69. The term "interface" is also used ordinary meaning in this field as "[t]he common boundary between independent systems or modules, where communication takes place." MacMillan IT Dictionary, Ex. 1017, p. 275. Therefore, the "external input signal bus interface" in the '994 patent should mean the "connection between a peripheral interface card and a host".

K. "Generic graphical interface specification"

97. A person of ordinary skill in the art, having read the patent in its entirety, would understand the term "generic graphical interface specification" in claim 11 to mean "a layout for one or more GUI elements".

98. The '994 patent does not define the term "generic graphical interface specification". The patent does state that "various display interface modes, while

different, preferably share a common look and feel based upon a generic graphical user interface specification." '994 patent, col. 9, ll. 20-23.

99. Figure 3 is described as showing an exemplary generic graphical user interface specification. '994 patent, col. 9, ll. 23-24. I do not think that there is any difference between the "generic graphical interface specification" of the claims and the "generic graphical user interface specification" of Fig. 3. Figure 3 shows a window having various displays provided generically in a layout. Figure 3 thus shows how elements are arranged on the screen shown to the user, without showing some other details of the elements. Therefore, the phrase "generic graphical interface specification" means "a layout for one or more GUI elements".

V. Legal Considerations

100. I have been informed that obviousness requires an evaluation from the perspective of a person of ordinary skill in the art at the time of the alleged invention. The obviousness inquiry includes an analysis of the scope and content of the prior art, the differences between the prior, the level of ordinary skill in the pertinent art, and any objective evidence of obviousness / non-obviousness. I understand that under the Supreme Court's ruling in the case *KSR v. Teleflex* that obviousness is a flexible analysis, and that "[t]he combination of familiar elements according to known methods is likely to be obvious when it does no more than yield predictable results."

101. I am of the understanding that objective evidence of obviousness / non-obviousness can be persuasive on the question of whether a patent claim is obvious over prior art. I understand that such objective evidence can include, for example:

- market forces that would have pushed a person of skill in the art toward or away from the claimed subject matter;
- the commercial success of the claimed subject matter, if it is a consequence of (has a "nexus" to) a novel claim element;
- evidence that third parties copied a product embodying the patent claims;
- evidence that a product embodying the patent claims solved a long-felt need that others had failed to solve;
- evidence that others independently developed the claimed subject matter, which might indicate that the invention became obvious, occurring to multiple parties around the same time;
- evidence that the prior art was skeptical that the claimed subject matter would work;
- evidence that the named inventors were praised for their work when it became known.

VI. Technology Background

102. In 1965, Gordon Moore, co-founder of Intel, famously observed that the number of transistors per square inch on an integrated circuit had doubled every year since their invention. Gordon Moore, "Cramming more components onto integrated circuits," *Electronics*, Vol. 38, No. 8 (Apr. 19, 1965) (Ex. 1079). While the rate of doubling has dropped to every 18 months, it continues today and is referred to as "Moore's Law." As a result of the increase in power, computers built today are exponentially more powerful than computers built prior to 2000.

103. Although personal computers ("PCs") first started to appear in the early 1970s, it was the release of the IBM PC in August 1981 that created a standardized product architecture, including an open architecture that allowed third parties to extend the capabilities of the basic machine. *The Birth of the IBM PC* (available at http://www-03.ibm.com/ibm/history/exhibits/pc25/pc25_birth.html) (last accessed April 21, 2013) (Ex. 1055). Indeed, while the modern personal computer supports numerous new features, it still supports virtually all of the features of the original IBM PC from 1981. The open and extensible architecture encouraged development of specialized peripheral devices that could provide complex data collection and analysis functions.

104. As the personal computer platform's basic support grew from the original DOS operating system to Windows and Linux platform support, the

industry naturally increased exploitation of the extensible functionality of the platform. For example, systems designers increased the use of the personal computer's native support for applications, input devices (such as mice and keyboards), as well as output devices (such as video displays). Systems designers also increasingly coupled these systems with specialized hardware capable of performing specialized processing operations beyond the processing power of the central processing unit itself.

105. In this time frame, a number of vendors, such as PLX Technologies, Applied Micro (AMCC), and National Instruments, provided standard chipsets, ASIC modules, and peripheral interface cards for developing specialized systems that would allow interaction with a vast array of external devices. These additional processing components sometimes came in the form of outboard devices, meaning that they were devices that were outside of the computer chassis. Sensors would be plugged into the external device, which handled some of the processing, after which the processed, digital signal was sent into the computer through a peripheral interface card mounted in a card slot inside the computer. In some cases, systems designers mounted general purpose central processing unit chips, such as the Intel i960, or other digital signal processors, onto the peripheral interface cards directly. The sensor signal would enter the computer by being plugged directly into the

peripheral interface card, and then get processed by the processor on that peripheral interface card prior to entering the computer's CPU.

106. In each of these devices, it was necessary to transfer the processed sensor data from the peripheral interface card to the computer's operating system for further processing, recording and display.

107. By the early 1990s, many companies designing real-time monitoring systems, including those in the medical industry, started using personal computers as the foundation for medical monitoring devices, both to improve time-to-market as well as to drive down the cost of the product. By the late 1990s, personal computers were commonly used for medical monitoring products. Personal computers of that era provided several advantages over the prior art, which included a graphical user interface, standardized architecture and peripherals, lower cost, and sufficient processing power to combine the functionality of multiple, single-function monitoring devices. '994 patent, col. 2 l. 60 – col. 3 l. 7. Additionally, personal computers allowed the user to not just collect data, but to analyze it, save it, catalog it, and share it with others on a network as well.

108. By the late 1990s, many companies designing real-time monitoring systems, including those in the medical industry, started using Windows NT as the operating system for those systems. *See, e.g.*, Section IX, below. As disclosed in the prior art I describe below, common practice at the time was to use kernel mode

drivers to extract the processed sensor data from the peripheral interface cards in these real-time systems.

1. The Incorporation of Kernel Mode Drivers for Purposes of Extracting Was Obvious

109. The application underlying the '994 patent was filed with the PTO May 20, 2002. The application covered a "multipurpose host system" for analyzing and displaying information from cardiovascular diagnostic measurement sensors, and contained twenty-five (25) claims. Claims 11-25 were later removed from further consideration. *In re Alpert*, Application No. 10/151,423, Response to Restriction Requirement (Dec. 28, 2004). On March 24, 2005, all of the remaining claims (1-10) were rejected as anticipated or obvious.

110. On September 23, 2005, the patent applicants amended the claims and added eleven new claims, including claims 33 and 34, which claimed the multipurpose host system of claim 1, but with "kernel mode drivers that extract processed sensor data from a peripheral interface card . . ." As of the September 23, 2005 amendment, claim 1 read as follows:

A multipurpose host system for invasive cardiovascular diagnostic measurement acquisition and display incorporating a component based arrangement, the system comprising:

an external input signal bus interface for receiving data arising from cardiovascular diagnostic measurement sensors; a plurality of measurement processing components, that operate at a user mode

level in the multipurpose host system, for facilitating receiving data of particular sensor types and rendering diagnostic measurement parameter values according to the received data; and

a multi-mode graphical user interface host comprising diagnostic measurement user interfaces including display components corresponding to data output rendered by specified ones of the plurality of measurement processing components;

In re Alpert, Application No. 10/151,423, Reply to Office Action (Sept. 22, 2005) (Ex. 1014).

111. In other words, as of the September 22, 2005 amendment, claim 1 recited all limitations that it does today, with the exception of the final limitation:

112. and one or more kernel mode drivers that extract processed sensor data from a peripheral interface card providing a hardware interface for one or more invasive diagnostic measurement devices.

113. That language was found in newly-added dependent claim 33, one of the eleven new claims added on that date. *Id.* The kernel mode drivers in newly-added claim 33 were also mentioned in further-dependent claim 34. *Id.*

114. On February 14, 2006, the Examiner rejected amended claim 1 and each of the dependent claims as either anticipated or obvious over various combinations of U.S. Patents 6,322,502, 6,358,202, 5,088,036 and 6,565,514. *In re Alpert*, Application No. 10/151,423, Office Action (Feb. 14, 2006) (Ex. 1015).

The Examiner noted, however, that "Claims 33 and 34 define over the art in that none of the art has the kernel mode drivers, as claimed." *Id.* The Examiner stated that claims 33 and 34 "would be allowable if rewritten in independent form including all of the limitations of the base claim and any intervening claims." *Id.*

115. On May 4, 2006, the application was amended again, this time to cancel claim 33 and to add the kernel mode driver limitation from claim 33 to claim 1. *In re Alpert*, Application No. 10/151,423, Reply to Office Action (May 4, 2006) (Ex. 1016). With the kernel mode driver limitation added to claim 1, the Examiner allowed the remaining claims. *In re Alpert*, Application No. 10/151,423, Notice of Allowability (July 24, 2006) (Ex. 1080).

116. Accordingly, the PTO has already determined that each limitation of claim 1 of the '994 patent is anticipated by the prior art except for the final limitation, relating to kernel mode drivers. In this section of my report I explain why, notwithstanding the Examiner's unfamiliarity with kernel mode drivers, the use of kernel mode drivers on a computer-based medical monitor in May 2002 was obvious.

117. At the time of the filing of the '994 patent, essentially *all* device drivers used for extracting information from peripheral interface cards in computers using a protected mode operating system (such as Linux and Windows) operated in the kernel mode. This was due to the inherent nature of peripheral

interface cards: peripheral interface cards generate an interrupt, and any device that generates an interrupt requires handling by privileged code. That privileged code must either have been a part of the operating system or have been a separate device driver. As of 2002, the use of user mode device drivers (the only alternative way of accessing peripheral interface cards) was uncommon. Accordingly, it would have been customary and obvious for one with ordinary skill in the art in 2002 to have used kernel mode device drivers to extract information from a peripheral interface card. *See, e.g.,* Prucka Decl., Ex. 1053, ¶ 21 (stating that Ms. Prucka's common practice for accessing peripheral interface devices when designing a medical monitoring system was to use device drivers provided by the manufacturer of the peripheral interface device); Byrd Decl., Ex. 1067, ¶ 14 (same).

VII. Invalidity Over the Prucka Operator's Manual And Other References

118. This section of the declaration explains my opinion that claims 1-20 are invalidated by the Prucka Operator's Manual in view of various other prior art references.

A. Background Of The Prucka CardioLab.

119. In the 1990s, Marquette Medical Systems, Inc. ("Marquette Medical") manufactured a product called Mac-Lab®, a hemodynamic recording system. Prucka Decl., Ex. 1053, ¶ 3. In 1998, GE Medical Systems, Inc. ("GE Medical

Systems") purchased Marquette Medical. *Id.* The former Marquette subsidiary became known as GE Marquette Medical Systems, Inc. ("GE Marquette"). *Id.*

120. Also in the 1990s, Prucka Engineering, Inc. ("Prucka Engineering"), under the leadership of founders Matt and Sheri Prucka, created CardioLab, an advanced, computerized diagnostic recording device, which it first released in 1991. *Id.* ¶ 2. CardioLab was used in electrophysiology labs and catheterization labs to diagnose and treat arrhythmias and hemodynamic problems using cardiac catheters. *Id.* In 1999, Matt and Sheri Prucka sold the assets of Prucka Engineering to GE Medical Systems and took employment with GE Marquette. *Id.* ¶ 3. Upon the acquisition, Matt and Sheri Prucka entered into a two year contract with GE Marquette to continue development of a new Windows NT® version of the CardioLab. *Id.*

121. In 2000, GE Marquette, under the leadership of the Pruckas, released a single-platform version of both CardioLab and Mac-Lab, which ran on Microsoft's Windows NT operating system. The new system was based on the CardioLab's underlying architecture, and incorporated elements of the Mac-Lab system. The CardioLab version of the system was then marketed and sold under the moniker, Prucka CardioLab, and the Mac-Lab version of the system was then marketed and sold under the moniker, Prucka Mac-Lab®, however both systems were based on the same general purpose central processing unit. The Windows NT

version of the joint Prucka CardioLab/Mac-Lab system was granted clearance from the United States Food and Drug Administration (the "FDA") on June 27, 2000.

Prucka Decl., Ex. 1053, ¶ 4.

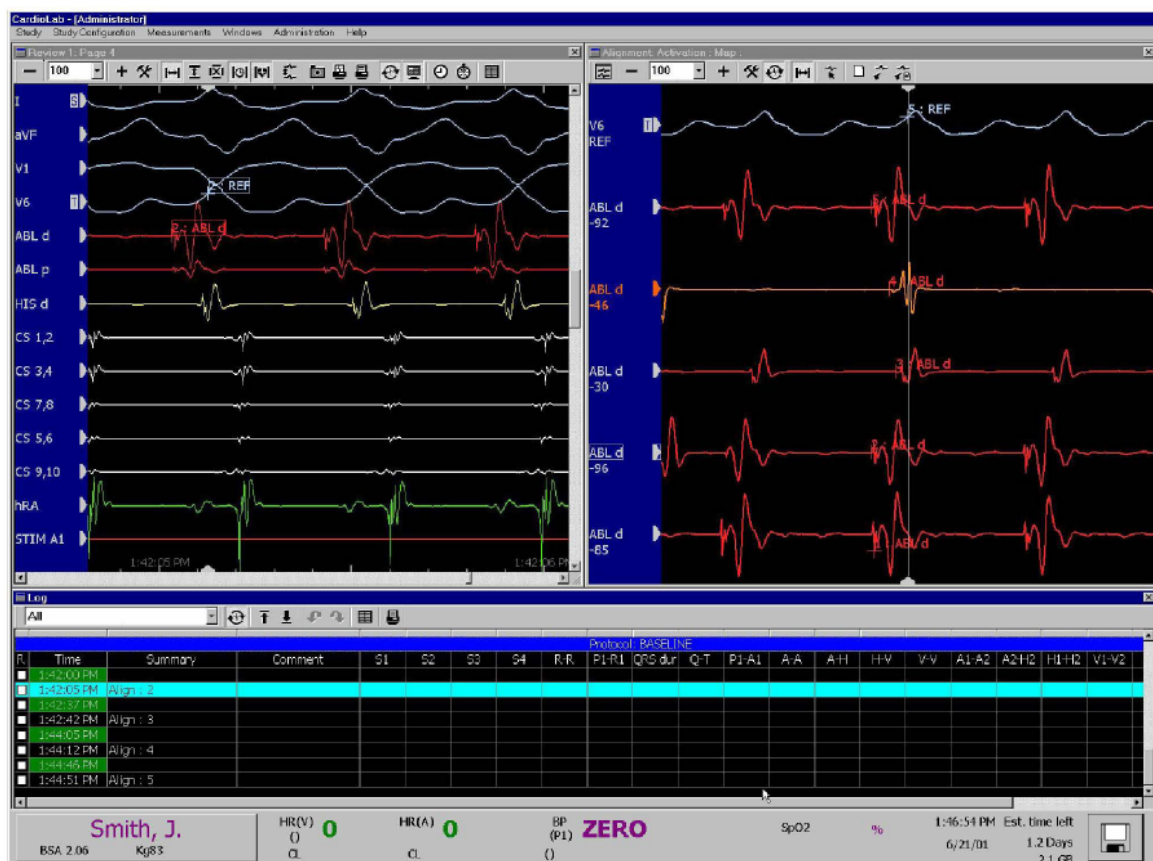
122. The Windows NT versions of the Prucka CardioLab and the Prucka Mac-Lab were each available in three different versions: models 2000, 4000, and 7000. Prucka Decl., Ex. 1053, ¶ 5 & Exs. 3-6 thereof (corresponding to exhibits 1087-1090). The software architecture between the various levels of the systems was the same; the differences indicated the feature set preferred by the customers. Prucka Decl., Ex. 1053, ¶ 5. The Prucka CardioLab/Mac-Lab system used external amplifiers depending on which application (CardioLab or Mac-Lab) was used. The external amplifier for the CardioLab application was called a CardioLab II or CardioLab II Plus amplifier, and the amplifier for the Mac-Lab application was called a TRAM. Id. ¶ 7.

B. Discussion Of Obviousness Over The Prucka Manual And Other References.

123. The operation of the Prucka CardioLab system is described by the "Prucka CardioLab 2000/4000/7000 Operator's Manual, Software Version 5.1" (Ex. 1003). I will refer to this as the "Prucka Manual". I understand that it qualifies as a prior art printed publication.

124. The Prucka Manual describes a system ("CardioLab") for invasive cardiovascular diagnostic measurements. This system is very similar in overall

function to the system described in the '994 patent. In particular, the Prucka Manual describes a system that can use multiple, invasive cardiovascular sensors. The information carried by invasive pressure sensors is conveyed to a host PC by a fiber optic cable. The host PC processes the information from those sensors and renders them in a graphical user interface-based display. The Prucka Manual discloses numerous types of information that can be obtained, numerous mathematical analyses that can be performed (see chapters 3 and 4, for example), and describes a complex Windows-based graphical user interface, having numerous displays corresponding to different types of information. An example of this display can be seen in Figure 9-7 of the Prucka Manual, shown here:



125. The Prucka Manual tells the reader that CardioLab is implemented on a Windows NT® system. Prucka Manual, pp. 2-3, 2-11. In the relevant timeframe, Windows NT® was a very popular operating system for personal computers, and in particular for personal computers used in industrial, medical or commercial contexts. Windows NT® was, at the time of the alleged invention, one of the most frequently used versions of the Windows® operating systems. These operating systems held the dominant market share for personal computers of the type described in the '994 patent.

126. The Prucka Manual is an operator's manual, and as such, does not disclose the source code of the CardioLab software. However, several things about the system would have been immediately apparent to a person of ordinary skill in the art. First, it would have been apparent that the data from the external sensors (carried by the fiber optic cable) needed to be transmitted to the Windows NT® host PC. It would also have been apparent that this could best be done through a peripheral interface card expansion slot that interfaces the fiber optic line to the PC. Next, it would have been immediately apparent from the fact that Windows NT® is being used that a kernel mode driver would be needed to extract sensor data from the peripheral interface card. Third, it would have been obvious to use a component based arrangement (such as object-oriented programming and/or Microsoft's Component Object Model ("COM")) to implement the CardioLab

software, because such arrangements were the dominant programming methods of the day.

127. The techniques described in the previous paragraph were all standard on and before May 20, 2002. For example, U.S. Pat. No. 5,819,115 ("Hoeser")(Ex. 1005), issued on October 6, 1998, teaches that it is standard to interface a fiber optic data cable to a PC using a network interface card. Hoeser states:

"There are many different types of networks and network systems for sharing files and resources or for otherwise enabling communication between two or more computers. Networks may be categorized based on various features and functions, such as message capacity, range over which the nodes are distributed, node or computer types, node relationships, topology or logical and/or physical layout, architecture based on cable type and data packet format, access possibilities, etc. For example, the range of a network refers to the distance over which the nodes are distributed, such as local-area networks (LANs) within an office or floor of a building, wide-area networks (WANs) spanning across a college campus, or a city or a state, global-area networks (GANs) spanning across national boundaries, etc. The architecture of a network generally refers to the cabling or media and media access used as well as the packet structure of the data transmitted across the media. **Various architectures are common, including** Ethernet using coaxial, twisted pair or **fiber-optic cables** for operation at 10 megabits per second (Mbps) (e.g. 10Base-T, 10Base-F) or fast Ethernet operating at 100 Mbps (e.g. 100Base-T, 100Base-FX). ARCnet (Attached Resource Computer

Network) is a **relatively inexpensive network architecture using coaxial, twisted pair or fiber-optic cables** for operation at 2.5 Mbps. IBM Token Ring topologies use special IBM cable or **fiber-optic cable** for operation at 4 or 16 Mbps. Of course, many other types of networks are known and available."

Hoese, col. 1, ll. 16-43 (Ex. 1005) (emphasis added).

128. Hoese states that expansion cards are available to interface networks of particular media types (such as fiber optics) to a PC using a driver:

"A computer system or node typically includes an expansion or input/output (I/O) bus with one or more slots for receiving expansion cards that expand or otherwise add functionality to the computer system. One example is a network adapter card or NIC for enabling the computer to attach to and communicate with a compatible network. **The NIC is implemented to support the particular physical media and network protocol desired.** For example, a NIC could include an RJ-45 plug for coupling to a twisted-pair cable or a BNC T-connector for coupling to a coaxial cable. The NIC is configured appropriately for interfacing the computer to an Ethernet network, such as 10Base-T or 100Base-TX, or to a token ring network, or to any other type of protocol supported by a network. A computer system equipped with an appropriate NIC for interfacing with a network **requires compatible drivers for proper operation.**"

Hoese, col. 1, l. 54 – col. 2, l. 3 (Ex. 1005) (emphasis added).

129. Hoese states that common types of peripheral interface busses standards include the Peripheral Component Interconnect (PCI) and the Industry Standard Architecture ("ISA"):

"The computer system 102 includes a host processor 200 coupled to a host bus 202, such as the **peripheral component interconnect (PCI) bus** or the like, which is also coupled to 60 a host or main memory 204....The computer system 102 further includes an input/output (I/O) or expansion bus 206, such as the **industry standard architecture (ISA)** or AT bus or the Extended ISA (EISA) **bus** or the like. The **expansion bus 206 is coupled to the host bus 202 through an interface** bridge or bus controller 208. The expansion bus 206 typically **includes one or more I/O slots (not shown) for receiving plug-in expansion cards for expanding or otherwise adding functionality to the computer system 102.** In the embodiment shown, a network interface card (NIC) 210 is plugged into a slot of the expansion bus 206 for interfacing the computer system 102 to the segment 106 and thus to the network 104. In particular, the **NIC 210 operates to enable the computer system 102 to communicate with the one or more network devices of the network 104.**"

Hoese, col. 4, l. 57 – col. 5, l. 10 (Ex. 1005) (emphasis added).

130. I note that well before May 20, 2002, there were several fiber-to-PC expansion cards commercially available. For example, a fiber-to-PC PCI card sold by Emulex is reported as early as 1995 in Exhibits 1056 and 1057. As reported in

Exhibit 1057, the Emulex PCI card was suitable for "point-to-point" configuration, as was used between the Prucka CardioLab CLAB device and host PC.

131. I also note that the Prucka CardioLab was a real commercial product, and used a fiber-to-PC expansion card as well, as discussed in the section on simultaneous invention, beginning at paragraph 335 below.

132. Hoese and the fiber-to-PC cards would have been relevant to the issues faced by the alleged inventors of the '994 patent. In particular, the '994 patent seeks to integrate cardiovascular diagnostic sensors into a standard PC to increase the modularity of the overall system. '994 patent, col. 6, ll. 17-35. The '994 patent turned to the standard mechanisms for integrating such sensors: expansion card slots and device drivers to manage communication with those slots. '994 patent, claim 1. Hoese teaches methods of interfacing a computer to networks, such as the one used in the '994 patent (*e.g.*, '994 patent, Ex. 1001, col. 8, ll. 61-62). Hoese describes the state of the industry generally in its background section, including the use of standard architectures for network connections. Hoese further describes drivers for network interfaces. The alleged inventors of the '994 patent, in turn, were concerned with moving from specialized systems to a multipurpose host. In using a multipurpose host, the alleged inventors were required to consider how to interface a standard PC to external sensors not commonly used with PCs. This would naturally have led to consideration of the use of Network interfaces

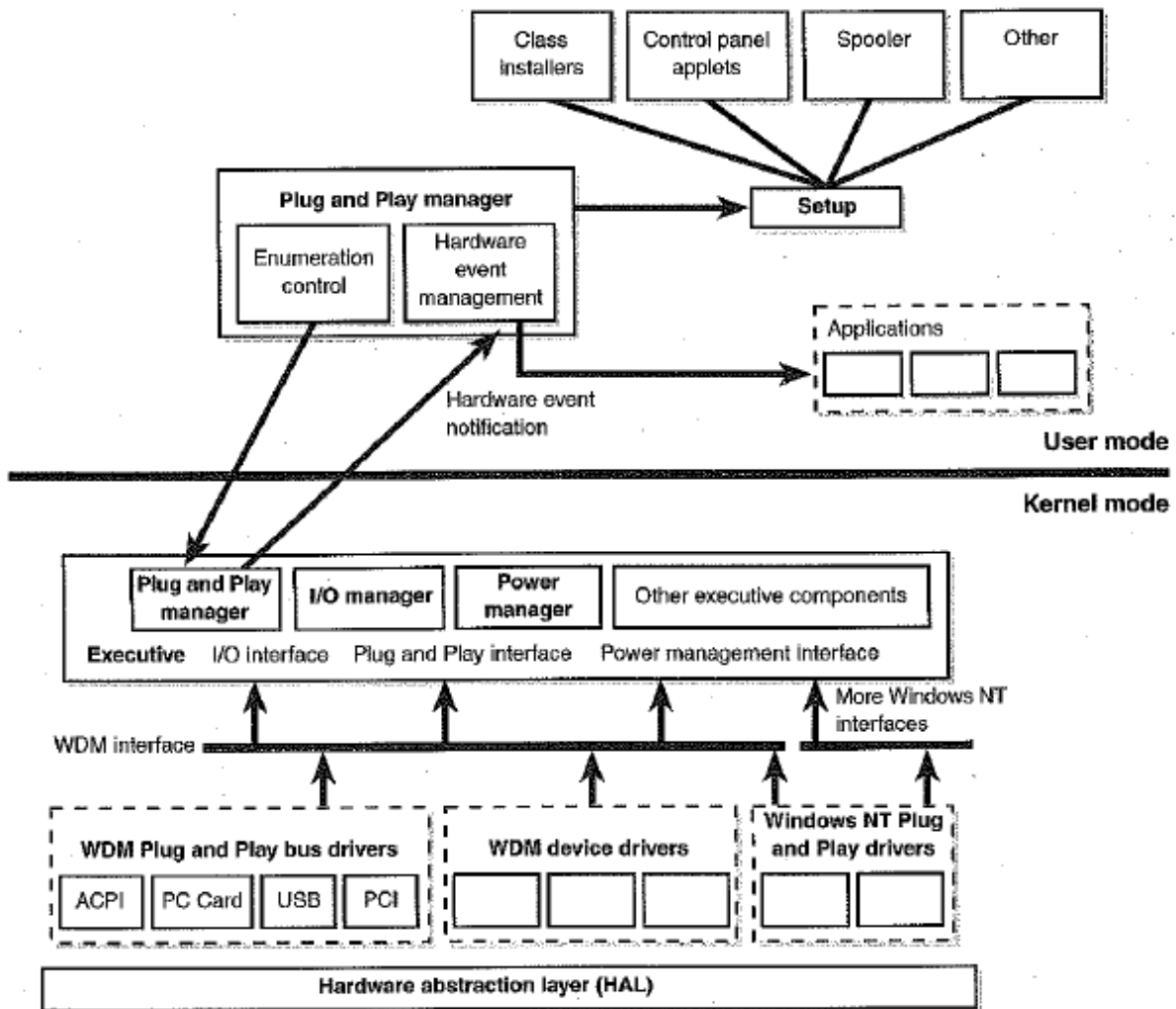
discussed by Hoeser. The fact that claim 1 recites "drivers", an "external input signal bus interface", a "peripheral interface card" and the extraction of sensor data supports the notion that the alleged inventors were concerned with the problem of interfacing external sensors to a multipurpose host, and that Hoeser was pertinent to this problem. Hoeser specifically states that it is applicable to any type of adapter card (col. 3, ll. 4-5), and also that its techniques are appropriate for interfacing fiber optic cable to a PC as expressly taught by Prucka (col. 4, ll. 12-21).

133. A Windows NT host, as described in Prucka, would have used kernel mode drivers to extract data from a peripheral interface card, for the reasons given above in paragraphs 23 to 27. At the time of the alleged invention, the use of kernel mode drivers to communicate with and extract data from peripheral interface cards was standard practice. For example, the 1998 book *Inside Windows* (Ex. 1004) states:

"Kernel Mode vs. User Mode. To protect user applications from accessing and/or modifying operating system data, Windows NT uses two *processor access modes* (even if the processor on which Windows NT is running supports more than two): *user mode* and *kernel mode*. User application code runs in user mode, whereas operating system code (such as system services and device drivers) runs in kernel mode."

Ex. 1004, p. 8.

134. The following diagram from *Inside Windows* shows an example of standard configuration for a Windows operating system:



135. In the diagram, the division between user mode and kernel mode is shown conceptually by the horizontal line in the middle. Applications, such as the programmable measurement processing done in the '994 patent and the Prucka Manual, run above the horizontal line, in "user mode". This is so that these components, which are most often programmed by applications programmers not

associated with source known for operating systems expertise, do not write code that can corrupt the operating system or other applications running on the operating system.

136. The operating system runs below the line in "kernel mode". Device drivers are shown in kernel mode across the bottom above the Hardware Abstraction Layer (i.e. interfacing with hardware components).

137. *Inside Windows* was a standard reference work for Windows programming and operating systems in general in the timeframe immediately preceding 2000. A person of ordinary skill would have found its teachings reasonably pertinent to the problems faced by the '994 patent. In particular, claim 1 of the '994 patent recites a "multipurpose host", and it was well-known that Windows® was a general-purpose operating system at the relevant time. A person of ordinary skill in the art would have reasonably sought ways to connect the data flow an external sensor to a standard PC. Standard PCs typically ran a version of Microsoft Windows. In business contexts, that version of Microsoft Windows was usually Windows NT, because Windows NT provided a number of security, stability and communications features geared toward professional environments. Therefore, a person of ordinary skill looking to interface a sensor with a PC would have looked toward standard information concerning Windows NT, and *Inside Windows* is a standard reference work, well known in the field.

138. It would further have been obvious to implement the software portions of the Prucka system as a "component-based arrangement". To a person of ordinary skill, a "component-based arrangement" would have been the only practical choice, because of the overall preeminence of object-oriented programming techniques in the timeframe immediately before the alleged invention. While it may have been technically possible to avoid using a component based arrangement, it would not have been realistic. Such an effort would have taken significantly longer, had significantly higher cost, significantly more software defects ("bugs") and would have risked commercial failure by taking too long to market. In the relevant timeframe, for example, there were really no development tools available that did not use component-based arrangements. One would have had to eschew all standard software development tools in order to avoid a component-based arrangement. The resulting software would also have been significantly more difficult to maintain, because any developer working on updating a small portion of the system would have had to understand the unique architecture of the entire system.

139. In object-oriented programming at the relevant time, it was considered good practice to break up conceptual programming tasks into individual objects. This increased reusability.

140. Furthermore, for applications running on a Microsoft Windows operating system, it would have been quite obvious to use the Microsoft Component Object Model or "COM". The COM system was a methodology for writing applications that allowed the applications to be easily extensible within a Microsoft Windows operating system.

141. The reference *Inside COM* (Ex. 1018) is a standard reference work describing Microsoft COM programming. As noted in *Inside COM*, COM was being used in most Microsoft applications immediately prior to the invention. Ex. 1018, p. 2. *Inside COM* further notes that COM allows applications to be more modular and extensible. Ex. 1018, p. 3.

142. It would have been obvious to combine the Prucka Manual with the teachings in Hoese, *Inside Windows* and *Inside Windows*. The Prucka Manual states that Windows NT should be used. This naturally leads to an application of the techniques in *Inside Windows* and *Inside COM*, which were standard for Windows at the time immediately prior to the alleged invention. The Prucka Manual further states that a fiber optic network cable should connect the instrumentation to the PC. In order to do that, one would look to ways for interface a fiber optic cable to a PC. At the relevant time, the only realistic way to accomplish this was by using a peripheral interface card such as those taught in

(Hoeser and Exhibits 1056 and 1057) plugged into an expansion slot, such as the ISA or PCI slots, on the PC.

143. I further understand (based on the case *KSR v. Teleflex*) that "[t]he combination of familiar elements according to known methods is likely to be obvious when it does no more than yield predictable results." In this case, the invasive cardiovascular system taught in the Prucka Manual, the employment of user mode applications and kernel mode drivers, the use of a "component-based arrangement" and external input signal bus interfaces and peripheral interface cards were all standard components in the time of the alleged invention and for years prior thereto. The combination of these components to implement the system described in the Prucka Manual would have all components performing their normal and expected functions. There would have been no unpredictable results. The '994 patent reports no such results, and treats all components as easily implementable as soon as they have been mentioned. Also, the fact that the Prucka system was implemented, and as implemented met the language of claim 1 (*see* paragraphs 337 to 430, below), is further evidence that the results were predictable.

144. I have reviewed the petition for inter partes review, including the claim charts which correspond the elements of claims 1, 10 and 18 to the Prucka Manual, *Inside Windows*, *Inside COM* and Hoeser. I am of the opinion that the

petition is accurate, and that the claims would have been obvious to a person of ordinary skill in the art.

1. Claim 1

a) "A multipurpose host system"

145. Regarding claim 1, a personal computer or "PC" is a multipurpose host system, because it can be used to run multiple different applications. *See also* paragraphs 124 to 125, above.

b) "for invasive cardiovascular diagnostic measurement acquisition and display"

146. The Prucka Manual shows that it is for invasive cardiovascular diagnostic measurement acquisition and display, as is clear throughout the manual. In particular, the Manual teaches the use of invasive pressure sensors and the display of data output from the sensors and processed by the system. *See also* paragraphs 124 to 125, above.

c) "incorporating a component based arrangement"

147. It would have been obvious to implement the software-based measurement and signal processing functionality described in the Prucka Manual as "components". First, macros are implemented as editable, extensible separate files, and are thus components. Prucka Manual, Ex. 1003, pp. 4-9 – 4-10. As noted above, macros can implement other measurement functionality.

Second, it would have been obvious to use the COM methodology to create specific components for each type of measurement and signal processing based on *Inside COM*. *Inside COM* teaches that components are more reusable when they deal with a specific and narrow function. In the Prucka Manual "measurement functions" are referred to as "functions". Prucka Manual, Ex. 1003, pp. 3-34 - 3-35 and 3-70 – 3-71) Per *Inside COM*, this leads to the conclusion that each function should be in a separate component in order to increase usability. *Inside COM* teaches to break identifiable functions into smaller tasks, and implement each as a component:

"A small interface [to a component] represents a single behavior, while a bigger interface represents many behaviors. The more behaviors represented by an interface, the more specific to a situation the interface becomes. The more specific an interface becomes, the less likely it is that it can be reused by another component. If an interface is not reused, client code that uses the interface can't be reused. For example, which is more reusable: a single interface that represents the behaviors of a helicopter, including flying, floating, hovering, lifting; rolling, vibrating, shaking, and falling, or multiple interfaces representing each of these behaviors individually? An interface that represents flying is much more reusable than an interface that represents a helicopter. Hardly anything but a helicopter behaves like a helicopter. However, lots of things fly."

Inside COM, Ex. 1018, pp. 27-28.

148. The term "interface" is equivalent to "component" in COM. *Inside COM*, Ex. 1018, p. 16 ("Interfaces are everything in COM. To the client, a component is a set of interfaces.").

149. *See also* paragraphs 155 to 157 below, discussing Prucka's macros as components.

d) "an external input signal bus interface for receiving data arising from cardiovascular diagnostic measurement sensors;"

150. The Prucka Manual describes a fiber optic network cable connecting the output of at least invasive pressure sensors to a PC via the CLAB II unit. Prucka Man., Ex. 1003, pp. 2-8 and A-5. The connection of the fiber optic cable to the PC would have required a fiber-to-PC interface, which would have, at the relevant time, been most obvious to implement as a peripheral interface card (see paragraphs 126 to 132, above). Such a card would necessarily interact with an external input signal bus interface, such as the common ISA or PCI busses, in order to receive information sent from the instruments over the fiber optic cable to the PC.

e) "a plurality of measurement processing components,"

151. The Prucka Manual describes software that performs measurement processing functions. There are a large variety of such measurement processing functions, including at least "measurement functions", "signal functions" and

"macros". The Prucka Manual describes "measurement functions" that perform automated analysis of pressure signals. Prucka Manual, Ex. 1003, p. A-5. For example, the Prucka Manual discloses that:

The Measurement function performs automated analysis of the pressure signals. To access one of these measurement items, you can either press the appropriate keys on the keyboard, or you can select the appropriate menu option.

Ex. 1003, pp. 3-70 – 3-71.

152. The various types of measurement processing that can be performed are described on pp. 3-70 *et seq.* of the Prucka Manual, and include conduction intervals, SNRT, ARP, RRP and manual measurements. Prucka Manual, Ex. 1003, pp. 3-70, *et seq.*

153. Second, the Prucka Manual further discloses "signal functions". These signal functions allow various signals, including intracardiac signals, ECG signals and pressure signals to be processed by software prior to display. Prucka Manual, Ex. 1003, pp. 3-20 – 3-29. For example, for pressure signals, the Prucka Manual describes a software-generated window that allows a user to set display parameters to be applied. Prucka Manual, Ex. 1003, p. 3-27. The Prucka Manual describes that setting the display parameters includes the ability to:

19. Select the label, scale, color, and mean settings from the pull-down menus in the *Label*, *Scale mmHg/Cm*, *Color*, and *Mean* cells.

Prucka Manual, Ex. 1003, p. 3-28.

154. The Prucka Manual further describes the ability to alter the display of a measured signal, for example by adjusting the gain or clip of a measurement.

Prucka Manual, Ex. 1003, pp. 3-34 - 3-35.

155. Third, the Prucka Manual describes the use of macros to automate other tasks (including the measurement and signal functions above). Prucka Manual, Ex. 1003, pp. 3-34 - 3-35. The Prucka Manual states:

Macros - A series of events that are expected to occur during a case. These can be programmed within the system to be performed with a single keystroke.

Prucka Manual, Ex. 1003, p. A-4.

156. The Prucka Manual further describes that macros have executable code. For example, the Prucka Manual states that macros can have signal functions (highlighted yellow) and measurement functions (blue):

Executable - An action phrase that is placed within in a line of a macro which automatically performs a specific task when entered. (e.g., Label Channel #1 LV, **Go To 50 Scale**, and **Measure**)

Prucka Manual, Ex. 1003, p. A-3.

157. Macros are "components" because they are separately contained as executables, and are extensible (editable). Prucka Manual, Ex. 1003, pp. 4-9 – 4-10.

f) "that operate at a user mode level in the multipurpose host system,"

158. As discussed in paragraphs 133 to 137, above, in a Windows NT system such as that discussed in the Prucka Manual, it would have been obvious (and indeed, necessary), to have measurement processing components of the type discussed in the Prucka Manual operating at a user-mode level.

g) "for facilitating receiving data of particular sensor types and rendering diagnostic measurement parameter values according to the received data;"

159. The measurement processing components, in order to process data, must necessarily have access to that data, and thus must necessarily "facilitate receiving data of particular sensor types". Therefore, it would have been obvious to have the measurement processing components facilitate reception of data of particular sensor types. The measurement functions described above produce (or "render") diagnostic measurement parameter values. For example, the measurement functions render "conduction interval values" (Prucka Man., Ex. 1003, p. 3-71), SNRT event parameters such as the pacing interval, the sinus node recovery time and the corrected sinus node recover time (Prucka Man., Ex. 1003, p. 3-73), antegrade refractory periods (Prucka Man., Ex. 1003, p. 3-73), and retrograde refractory periods (Prucka Man., Ex. 1003, p. 3-74). The signal functions performing rendering of diagnostic data from drawing on screen. The

macros provide for the execution of measurement and signal functions, and thus similarly render diagnostic measurement parameter values.

**h) "a multi-mode graphical user interface host
comprising diagnostic measurement user interfaces"**

160. As shown above in paragraphs 124 to 125 and in connection with Fig. 9-7, the Prucka Manual shows a Windows-based GUI for displaying diagnostic information of multiple types within the same window.

161. For example, the Prucka Manual describes the availability of:

- a "Real-Time Window" and "Review Window" (Prucka Manual, Ex. 1003, p. A-3),
- a "Navigator Window" (Prucka Manual, Ex. 1003, p. 1-12),
- a "Patient Information Window" (Prucka Manual, Ex. 1003, p. 1-13),
- a "Study Menu Window" ((Prucka Manual, Ex. 1003, p. 3-12),
- an "ADT Search Window" (Prucka Manual, Ex. 1003, p. 3-12),
- a "Login Window" (Prucka Manual, Ex. 1003, p. 3-13),
- a "Filter Catalog Window" (Prucka Manual, Ex. 1003, p. 3-74),
- various software and hardware signal function display setting windows (Prucka Manual, Ex. 1003, pp. 3-20 – 3-27),
- a "Vitals Configuration Window" (Prucka Manual, Ex. 1003, p. 3-30),

- a "Channel Properties Window" (Prucka Manual, Ex. 1003, p. 3-37),
and
- a "Calipers Window" (Prucka Manual, Ex. 1003, p. 3-38).

162. These windows are programmed as child windows and dialog boxes in the CardioLab Windows® NT application. This allows multiple windows to be arranged within the main CardioLab window, much like different spreadsheets can be arranged within the parent window of Microsoft Excel.

i) "including display components corresponding to data output rendered by specified ones of the plurality of measurement processing components; and"

163. The Prucka Manual describes that display components correspond to (i.e. show) data output rendered by specified ones of the measurement processing functions, which are in turn directed to the output of specified sensors. See paragraphs 151 to 162, above.

164. Various available windows are described in the Prucka Manual for displaying diagnostic information. In particular, here were "real-time", and "review" and "log" windows available to review the processed output of sensor data. Regarding real-time and review windows, the Prucka Manual states:

"Real-Time window - The window that displays signal data as it is occurring. Review window - The window that is used to review and manipulate all saved signal data."

(Prucka Man., Ex. 1003, p. A-3).

165. A "log window" is also described as recording (and thereby displaying) data and diagnostic events. Prucka Man., Ex. 1003, pp. 3-72 – 3-74.

j) "including display components corresponding to data output rendered by specified ones of the plurality of measurement processing components; and"

166. As shown in Figure 9-7 of the Prucka Manual, the CardioLab GUI had multiple areas for displaying the output of particular sensors and processed data from sensors. For example, the Prucka Manual notes that the displayed signals are output data originally acquired from various channels, where the selection of the channels is up to the user. Prucka Man., Ex. 1003, p. A-3.

167. These data are processed by measurement processing components. These components include signal functions, which render the data for display. Prucka Man., Ex. 1003, pp. 3-34 - 3-35. Furthermore, the results of measurement functions for conduction intervals and calipers are displayed on these signals when they are performed. Prucka Manual, Ex. 1003, pp. 3-71 and 3-75. The results of SNRT, ARP and RRP functions are displayed in a log window, which is also a child window, and therefore a display component. Prucka Manual, Ex. 1003, pp. 3-72 – 3-74.

- k) **"one or more kernel mode drivers that extract processed sensor data from a peripheral interface card providing a hardware interface for one or more invasive diagnostic measurement devices."**

168. As explained above in paragraphs 133 to 136, it would have been obvious, and indeed necessary, to use kernel mode drivers to extract processed sensor data from the peripheral interface card used to connect the fiber optic cable taught in the Prucka Manual to the Prucka Windows NT-based PC. The necessity of drivers is explained in Hoese, Ex. 1005, col. 2, ll. 1-3 and *Inside Windows*, Ex. 1004, pp. 477-479.

169. I noted that Prucka teaches the processing of sensor data in the CLAB II device prior to the communication of the processed data to the PC. This processing included electronic filtering and gain settings. Prucka Manual, Ex. 1003, pp. 3-32, 3-23, 3-25, 3-26, 3-29 and 3-46. Furthermore, the Prucka Manual describes that the data is collected in analog form, and converted to digital form by the CLAB II unit. Prucka Manual, Ex. 1003, p. 3-26.

2. Claim 10

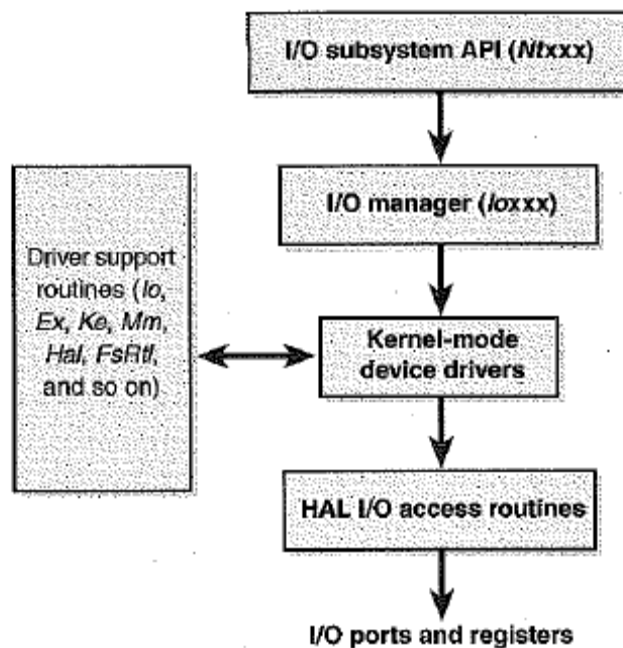
170. Claim 10 reads: "*10. The multipurpose host system of claim 1 wherein the plurality of measurement processing components are instantiated from component objects.*"

171. Specifically regarding claim 10, COM components are created by a process of "instantiation". This involves calling the "createInstance" method, which generates an "instance" of a component. This is the process of instantiation.

3. Claim 18

172. Claim 18 reads: "18. The multipurpose host system of claim 1 further including a sensor component application interface through which the measurement processing components issue requests to the kernel mode drivers."

173. Regarding claim 18, communication in a Windows NT system from an application in user mode to a driver in kernel mode usually happened via an "application programming interface" or API. The architecture is shown in Fig. 7-1 of *Inside Windows*:



174. Here, the I/O subsystem API is the API that would typically have been used between an application component in user mode and a driver in kernel mode. This included functions like sending control information to the device as well as reading processed sensor data (“read” and “write” operations).

4. Claims 11-12 and 20 (see also subheadings below).

175. Claims 11-12 and 20 require a "generic graphical interface specification". I believe that this term means "a layout for a GUI element".

176. In any Windows-based GUI application, essentially all parts of a window are based on an underlying object. For example, the window frame itself has a corresponding object, pull down menus have corresponding objects, radio buttons have corresponding objects, text entry fields have corresponding objects, and graphics areas have corresponding objects. These objects are typically instantiated with certain properties that give them a size and location, or an express layout scheme. Thus, an object-based GUI will always have a "generic graphical interface specification". This is discussed in Morris, "Object-Oriented Programming for Windows 95 and NT" (Ex. 1042). Morris is a standard book for programming Windows NT, the same operating system used by the Prucka Cardiolab and described in *Inside Windows*, and for which the COM principles of *Inside COM* can be used. In particular, Morris relates to the programming of a graphical user interface, a problem with which the alleged inventors were

particularly concerned (see paragraphs 16-17). It would have been obvious to apply the teachings of Morris in the context of a component-based arrangement on a Windows NT system. Such a combination would have been of known components and would have led to a completely predictable result.

5. Claim 11

177. Claim 11 reads: "*11. The multipurpose host system of claim 1 wherein the multi-mode graphical user interface host supports the set of diagnostic measurement user interfaces through a generic graphical interface specification.*"

178. Regarding claim 11, Morris describes that when designing the layout of elements in a Windows-based GUI window, the programmer first creates a main window, giving it a set of properties that define its size, position, color, etc. Morris, Ex. 1042, p. 160. The programmer then adds different "controls", such as child windows, menus, checkboxes, image panels, etc. Morris, Ex. 1042, chapter 6. Each of these controls is its own software object. Each such control object consists of a set of separate objects, and each has properties that specifies the on-screen size, location and color of the corresponding display component. Morris, Ex. 1042, pp. 160, 180-184. The properties are determined in the first instance by the programmer. Morris describes using development software to define a layout for a GUI by placing GUI elements on a form having a grid. Morris states:

"When developing a form there is a background grid of points for aligning controls. If you change the controls' size or position by dragging, the control corners 'snap' to the nearest grid points."

Morris, Ex. 1042, p. 182.

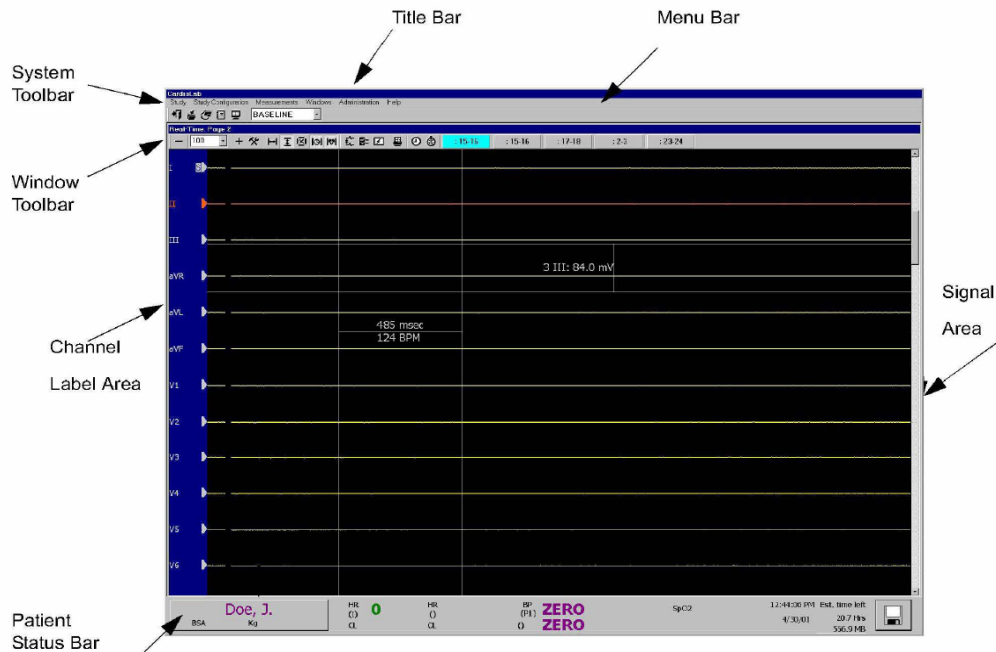
179. The form grid (or the manual setting of properties) determines the size and position of the display elements. Morris explains:

"The size and position of a control are determined by the same set of four properties as for a form: Width and Height for the size, Left and Top for position. Left and Top give the position of the top-left corner of the control relative to the top-left corner of the inside of the form (the client area). These properties can be changed either by dragging the sizing handles on the corners and sides of the control or by entering new settings directly."

Morris, Ex. 1042, p. 182.

180. The collection of the main window properties together with control properties determines the application GUI layout, and constitutes the "generic graphical interface specification". Such a specification would be present in any Windows application. A GUI layout from the Prucka Manual is shown in the diagram on page 1-14 of the Prucka Manual, shown here:

This is the main application window.



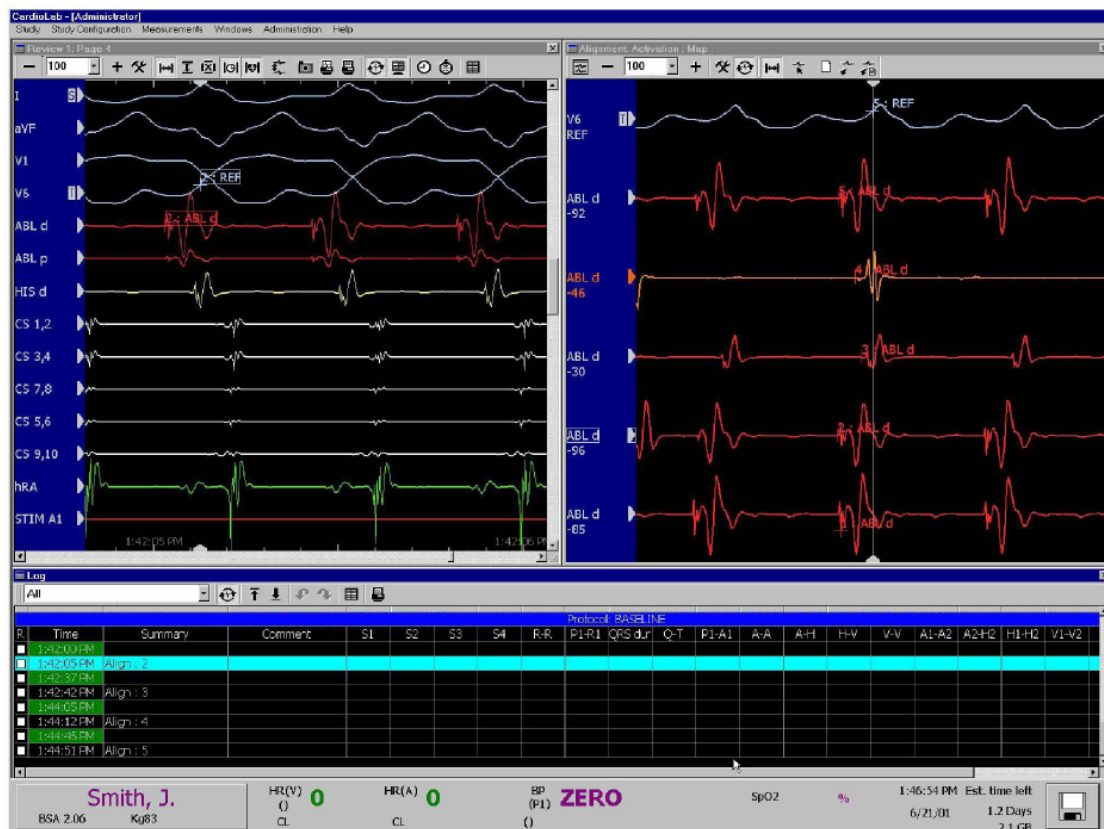
181. Some of the layout elements will be fixed, for example, the "Title Bar", "Menu Bar", "System Toolbar", "Window Toolbar", and "Patient Status Bar" shown above. Within the child window (labeled "Real-Time Page 2"), the "Channel Label Area" and "Signal Display Area" are programmed using objects as described in Morris, having properties such as relative size and location. This gives the application a particular look-and-feel.

6. Claim 12

182. Claim 12 reads: "12. *The multipurpose host system of claim 11 wherein the generic graphical interface specification includes: a patient information display region; and a mode-specific data display region including graphical display components corresponding to data output rendered by specified*

ones of the measurement processing components associated with a current display mode."

183. Regarding claim 12, the Prucka Manual shows a patient information display region at the bottom left (with name and weight: "Smith, J. kg83". Prucka Man., Ex. 1003, p. 9-7, see also p. 1-14, showing "patient status bar").



184. The Prucka Manual also states that there are windows corresponding to different display modes (also shown in the figure on page 9-7). For example, the "Realtime Window" allows signals to be seen in real-time. The "Review Window" allows signals to be seen in the past. The "Log Window" allows various

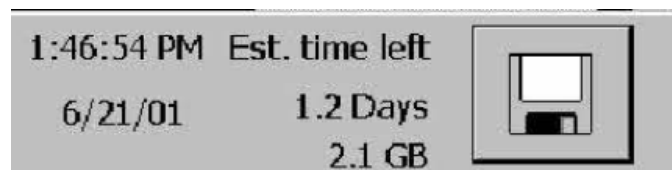
events to be seen. See paragraphs 161 to 165, above. Each of these are modes. These windows display output rendered by measurement processing components as discussed above under paragraphs 163 to 167.

185. The generic graphical interface specification includes these elements, as discussed in paragraphs 175 to 181, because the programmer must specify the size, location and appearance of the child windows, dialog boxes and control components in object properties when the windows are initially drawn.

7. Claim 20.

186. Claim 20 reads: "*20. The multipurpose host system of claim 12 wherein the generic graphical interface specification includes a system message display region.*"

187. Regarding claim 20, Figure 9-7 of the Prucka Manual shows that a system message display area is provided as part of the generic graphical interface specification, i.e., it is in the lower right hand corner of the screen. The system message display area is shown here;



188. This area has a time and date, which are system messages. The area also provides disk space remaining and a disk status icon which are system messages. Prucka Man., Ex. 1003, pp. 1-16 (top), 3-51, bottom. The system

message display area is part of the generic graphical interface specification, because it is in a non-variable, known location within the window (Fig. 9-7). This location is specified within one or more objects underlying the GUI. See paragraphs 175 to 181, above.

VIII. Invalidity Over the '669 Patent And Other References

189. This section of the declaration explains my opinion that claims 1-2, 4-6, 8-11, 12-13, 15-18 and 20 are invalidated by the '669 patent in view of various other prior art references.

A. Background Of The '669 Patent, SmartFlow And Their Owner, Florence Medical

190. Florence Medical was a private, development stage company located in Israel (Florence Medical, Ltd.) and Boston (Florence Medical, Inc., referred to herein collectively with Florence Medical, Ltd. as "Florence Medical").

Declaration of Frank Martin (the "Martin Decl.") ¶ 2 (Ex. 1058). It was founded by scientists and engineers with expertise in computational fluid dynamics. *Id.*

191. During its early years, Florence Medical received five U.S. Patents (the "Florence Medical Patents"). Declaration of Dr. Chen Barak (the "Barak Lit. Decl.") ¶ 5. As described in the Florence Medical Patents, Florence Medical designed a system for measuring and displaying intravascular blood pressure, flow and other data. During the development of their technology, Florence Medical conducted experiments using personal computers for data acquisition and display

and described the system and their experiments in their patents. The Florence Medical Patents described experiments in which Florence Medical used intravascular guidewires manufactured by both Radi Medical Systems and Cardiometrics. *See, e.g., id.* ¶ 5 & U.S. Patent 6,193,669 (Ex. 1024) at col. 17, ll. 19-24. Florence Medical's inventors plugged the guidewires into signal processing and conditioning devices such as the Radi Pressure Wire Interface Type PWI10. *Id.* col. 13, ll. 5-11. Doctors used pressure-sensing guidewires such as the Radi PressureWire® to make invasive diagnostic measurements by inserting the wire into a patient's femoral artery and carefully moving it through the patient's vessels into the heart.

192. The signals were then plugged into a personal computer running a Windows operating system by means of a peripheral interface card. *Id.* col. 13, ll. 14-29, col. 17, ll. 50-61. The peripheral interface card was an off-the-shelf card

manufactured by National Instruments Inc. model PCI-MIO-16E-4 PCI card,³ *Id.* col. 17, ll. 51-53. The inventors used the device driver from National Instruments that came with the PCI-MIO-16E-4 PCI card to extract data from this peripheral interface card.⁴ Barak Lit. Decl., Ex. 1134, ¶ 5. Data acquisition and analysis was done with LabVIEW and MatLab software programs. '669 Patent, col. 17, ll. 54-60 (Ex. 1024). Using these software packages, the inventors analyzed invasive, cardiovascular blood pressure and blood flow data. Barak Lit. Decl., Ex. 1134, ¶ 5. The '669 Patent was filed December 11, 1998.

³ The name of the card in the '669 Patent appears to contain a typographical error. It references a "PC-NIO-16E-4" card. *Id.* col. 17, ll. 52-53. U.S. Patent 6,471,656, which predates the '669 Patent, names several of the same inventors, was also assigned to Florence Medical, and describes the same computer system, uses the correct model number – PCI-MIO-16E-4 – in identifying the PCI card. *See id.*, col. 12, ll. 17-48.

The PCI-MIO-16E-4 was subsequently renamed the PCI-6040E. National Instruments, "NI PCI-6040E" <http://sine.ni.com/nips/cds/print/p/lang/en/nid/10795> (last accessed April 17, 2013) (Ex. 1078), at "Product Name."

⁴ *See* n. 8, *supra*.

193. By the fall of 2000, Florence Medical had developed a computer-based medical monitoring device called the SmartFlow system. Martin Decl., Ex. 1058, ¶ 3. The SmartFlow system was a personal computer-based system for the interventional cardiologist, which displayed various parameters, including blood pressure and blood flow, using software running on a Windows operating system. Accordingly, the SmartFlow system had inputs for intracoronary blood pressure data. *Id.* Using the data from the sensor inputs, the SmartFlow system used proprietary software to render calculations of coronary flow reserve ("CFR") and fractional flow reserve ("FFR"). *Id.* The system had a color display for displaying signals and processed data. *Id.* The display included a touch screen, including a soft keyboard, for entry of patient and control data. *Id.*

B. Discussion Of Obviousness Of Claims 1, 10-11 and 18 Over The '669 Patent And Other References.

194. It is my opinion that claims 1, 10-11 and 18 are unpatentable under 35 U.S.C. § 103(a) over U.S. Pat. No. 6,193,669 ("the '669 patent")(Ex. 1024), when supplemented or modified by the following:

- Krauss, *et al.*, "LabView™ for sensor data acquisition", Trends in Analyt. Chem., Vol. 18, No. 5, 1999 ("Kraus 1999")(Ex. 1025);
- and David A. Solomon, "*Inside Windows NT Second Edition*", Microsoft Press, 1998 ("*Inside Windows*")(Ex. 1004).

195. I understand that each of the references is prior art, and that Krauss 1999 and *Inside Windows* qualify as printed publications, having been published in 1999 and 1998, respectively.

196. I have reviewed the petition for inter partes review, including the claim charts which correspond the elements of claims 1, 10-11 and 18 to the '669 patent, *Inside Windows* and Kraus 1999. I am of the opinion that the petition is accurate, and that the claims would have been obvious to a person of ordinary skill in the art.

1. Claim 1

a) "A multipurpose host system"

197. Regarding claim 1, the '669 patent provides basically the same system as the '994 patent. Figure 1 of the '669 patent is shown here:

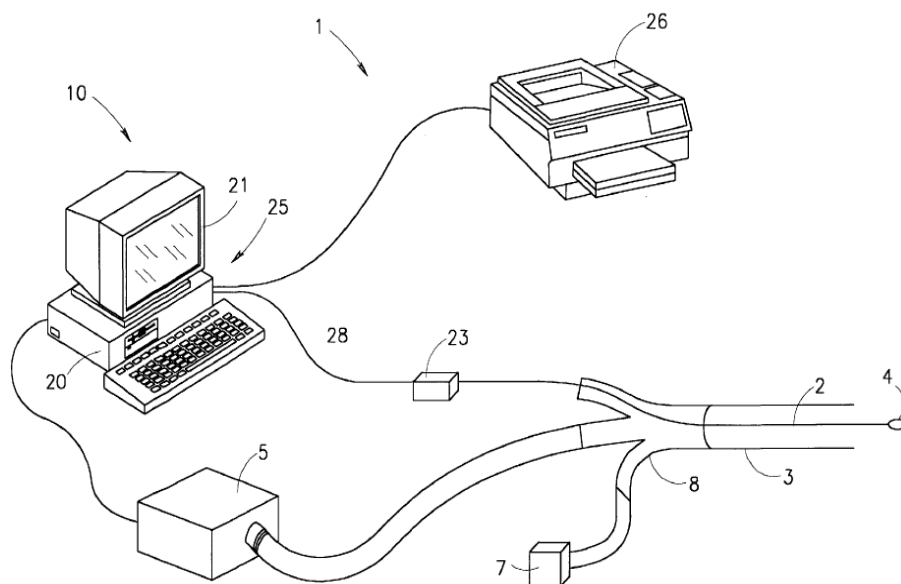


FIG.1

198. Like the '994 patent, the '669 patent uses a standard Pentium 586 PC (labeled "20"). This is a multipurpose host system.

b) "for invasive cardiovascular diagnostic measurement acquisition and display"

199. The PC processes signals received from invasive cardiovascular sensors (e.g. "4"). The '669 patent states as follows in the Abstract:

"This invention relates to a method and devices for detection, localization and characterization of occlusions, aneurysms, dissections stent position, dissections stent mal-position, wall characteristics and vascular bed. The invention is based on introducing an artificial pressure or flow excitation signal (a single signal or more) into the blood vessel (or in other tubular flowing fluid conduits), measurement and analysis of the pressure and or flow."

'669 patent, Ex. 1024, Abstract.

200. Figure 3 of the '669 patent is shown below:

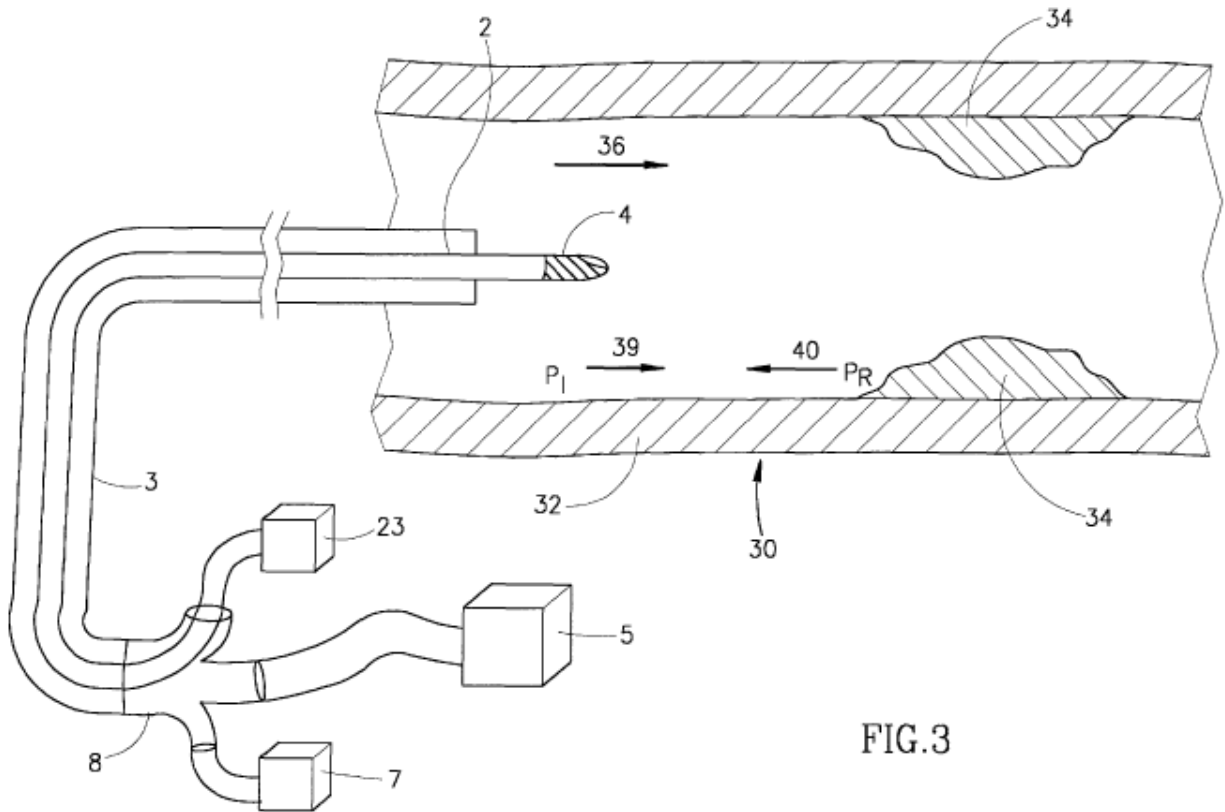


FIG.3

201. Figure 3 shows an invasive sensor 4 in artery 30 in order acquiring a cardiovascular diagnostic measurement. '669 patent, Ex. 1024, Abstract, col. 15, ll. 1-12.

c) "an external input signal bus interface for facilitating receiving data of particular sensor types"

202. The acquired signals are transmitted to a signal conditioning unit 23 and then A/D converted in a converter 28 (see Fig. 1), each of which would qualify as an external input signal bus interface. The processed signals are then passed to a peripheral interface card (not shown in the diagram), where the PC acquires the signals. '669 patent, Ex. 1024, col. 17, ll. 48-56 and col. 13, ll. 9-16. The fact that the PC acquires the signals indicates the existence of an external input signal bus

interface communicating with the peripheral interface card. Once acquired, the signals are processed by the PC 25, and then displayed. '669 patent, Ex. 1024, col. 13, ll. 16-29; Figs. 12, 14-17, 19, 21-28.

d) "one or more kernel mode drivers that extract processed sensor data from a peripheral interface card providing a hardware interface for one or more invasive diagnostic measurement devices."

203. Concerning the system setup, the '669 patent states the following:

"Data acquisition was performed using a PC (Pentium 586) with an E series multifunction I/O board 28 model PC-NIO16E- 4 of the type commercially available from National Instruments Inc., TX, U.S.A. The I/O board was controlled by a Labview graphical programming software, commercially available from National Instruments Inc., TX, U.S.A. 10 sec interval of pressure and flow data were sampled at 5000 Hz, displayed during the experiments on the monitor and stored on hard disk."

'669 patent, Ex. 1024, col. 17, ll. 50-61.

204. The National Instruments I/O board is a peripheral interface card for extracting processed sensor data by the PC. The board was controlled using kernel mode drivers. This would have been obvious to a person of ordinary skill in the art from the '669 patent that the only realistic way to control data acquisition from the National Instruments board was with a kernel mode driver. This would also have been obvious from *Inside Windows*, which states that drivers run in kernel mode.

It would have been obvious to apply this teaching of Inside Windows, because of the '669 patent's disclosure of a PC, and the dominant position of the Windows operating systems in the PC market. The reasoning in paragraph 137 applies here.

205. In fact, the National Instruments board did require kernel mode drivers, as evidenced by the datasheet for the board (Ex. 1059) (see ¶192, above, regarding the typographical error in the '669 patent's designation of the board). The datasheet indicates that the board uses Direct Memory Access (DMA) of memory in the host. This means that the board employed a kernel mode driver in the relevant timeframe, because there was no other way to perform DMA of the host memory on a Windows system. Even today, where Windows has limited support for user mode drivers, a peripheral interface card using Direct Memory Access requires a kernel mode driver on a Windows system.

206. The '669 patent describes that the board is controlled at a user interface level using Labview software, which was also available from National Instruments. The '669 patent discloses A/D conversion (processing) of sensor data prior to acquisition by the host PC. '669 patent, col. 13, ll. 11-17. Krauss 1999 also states that some other forms of processing can be moved from the host PC to the peripheral interface card, leading to advantages in certain situations. Kraus 1999, Ex. 1025, pp. 314-315. Thus, it would have been obvious to do so.

207. Labview software was widely known in the computer systems industry in the relevant timeframe. Krauss 1999 describes Labview software. Krauss 1999 is entitled "LabView™ for sensor data acquisition". Krauss 1999 describes the authors' evaluation of Labview as a programming tool, and their use in a trial sensor data acquisition experiment. The authors state that:

"LabView™ by National Instruments [1] is a universal programming system, with both a graphical user interface ('Panel', Fig. 1) and a graphical program code ('Diagram', Fig. 2). It was designed for programming data acquisition, data handling and data display."

Kraus 1999, Ex. 1025, p. 312.

- e) **"a multi-mode graphical user interface host comprising diagnostic measurement user interfaces including display components corresponding to data output rendered by specified ones of the plurality of measurement processing components;"**

208. Krauss 1999 further states that Labview "makes it easy to form an attractive and structured GUI." Kraus 1999, Ex. 1025, p. 312, middle of right column. Krauss 1999 states that the Labview system is a "very powerful" system for developing data acquisition, handling and display programming in teams:

"LabView is a promising alternative to conventional compilers for programming data acquisition, data handling and storage, and data display software. The advantage is the easy realization of simple and small tasks with emphasis on a visual interface. Its boundary conditions are not time-critical programs and the hardware is

supported by LabView. After an indispensable intense training period and after getting accustomed to the graphical programming, the user of LabView has a very powerful system for software development in teamwork."

Kraus 1999, Ex. 1025, p. 318.

f) "incorporating a component based arrangement" and "a plurality of measurement processing components, for receiving data arising from cardiovascular diagnostic measurement sensors;"

209. Labview as described by Krauss 1999 was a component-based arrangement. Each Labview program took the form of a "virtual instrument" or "VI" (*i.e.* a "component") that could in turn be subdivided according to function into "sub-virtual instruments" or "sub-VIs" (other "components"). Kraus 1999, Ex. 1025, pp. 312, bottom of left column and pp. 316-317.

210. This use of VIs and sub-VIs makes the program structure extensible and modular. Krauss 1999 states:

"According to the modular hardware concept the program code is divided into 32 sub-VIs. This modular program structure allows the project partners to adapt and to optimize the software part related to their sensors."

Kraus 1999, Ex. 1025, pp. 316-317.

"LabView thus allows a modular layout of a program with an easy to define interface which is the panel of the sub-VI. This possibility should be used to maximum extent...."

Kraus 1999, Ex. 1025, p. 316 top of left column.

211. Krauss 1999 teaches that it is advisable to divide VIs and sub-VIs into separate units, each of which can handle a particular sensor function:

"This modular program structure allows the project partners to adapt and to optimize the software part related to their sensors. Numerous functions such as log-ins for different users and programmers, program history displays and info windows for user-defined remarks give valuable support to the programming team. This support and the modularity were the main reasons for choosing LabView for the project."

Kraus 1999, Ex. 1025, pp. 316-317.

212. This teaching of Krauss 1999 would have suggested to a person of ordinary skill in the art in the relevant timeframe to break a sensor acquisition and control system into a series of smaller conceptual functions, and to implement a sub-VI for each function. For example, it would have been quite natural to create a sub-VI to acquire and process data from each specific sensor-type within a system, so that a programmer knowledgeable about the specific requirements of a particular sensor type could specialize in that sub-VI.

g) "for facilitating receiving data of particular sensor types and rendering diagnostic measurement parameter values according to the received data;"

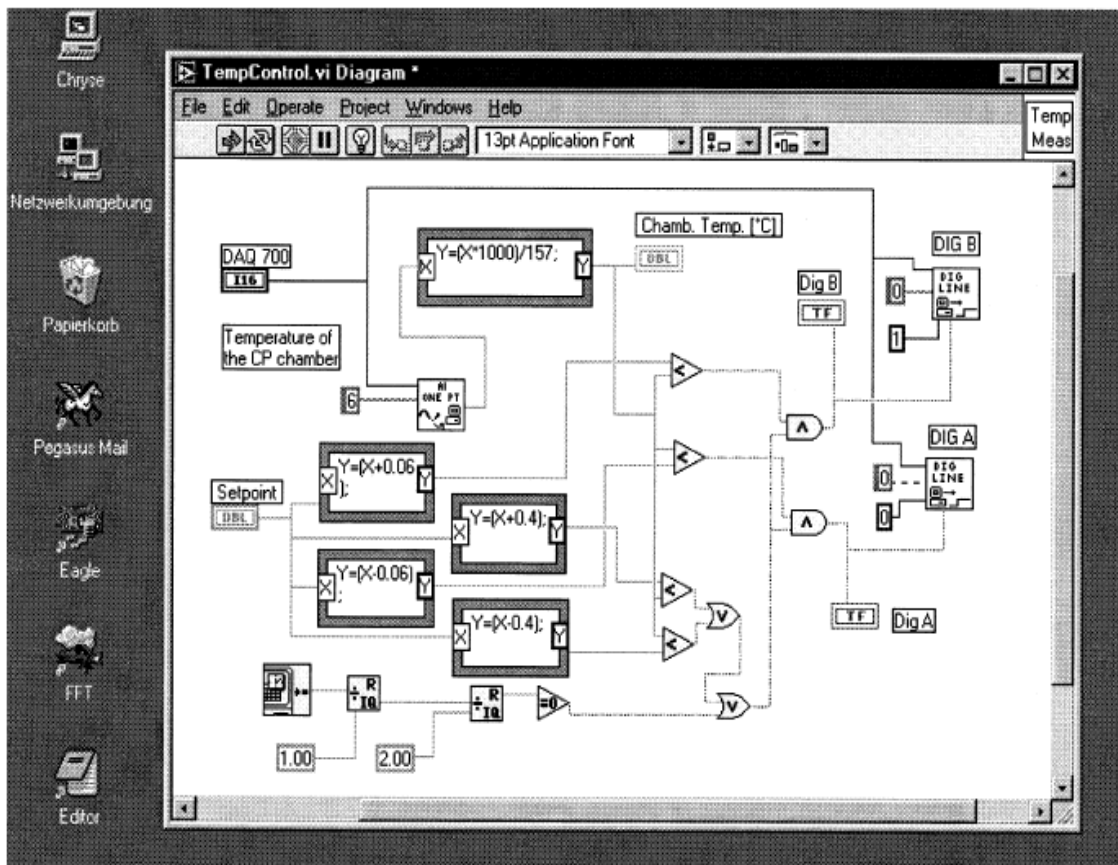
213. The '669 patent teaches rendering diagnostic measurement parameter values according to the received data. Specifically, the '669 patent describes taking diagnostic sensor data as input and producing diagnostic measurement parameter values. For example, in Procedure 1 (beginning at col. 18, l. 15), the '669 describes taking an input comprising "[t]he pressure measured by a downstream sensor" '669 patent, Ex. 1024, col. 9, l. 35 – col. 11, l. 41. This and other input data is processed to result in "a function including two peaks representing the forward and reflected pressure wave and having different amplitudes." '669 patent, Ex. 1024, col. 9, l. 35 – col. 11, l. 41. As described by the '669 patent, "[t]he ratio of the area under those peaks serves to estimate the severity of the stenosis or aneurysm", and is thus "diagnostic". '669 patent, Ex. 1024, col. 20, ll. 14-16. Similar sensor inputs leading to diagnostic outputs are also disclosed for Procedures 2, 3 and 4. '669 patent, Ex. 1024, col. 9, l. 35 – col. 11, l. 41, col. 9, l. 35 – col. 11, l. 41, col. 20, ll. 14-16, col. 18, l. 14 – col. 22, l. 6.

214. Krauss 1999, in turn, describes the extensive processing that Labview was capable of performing to sensor data. Krauss 1999 teaches that the processing performed by Labview included the use of mathematical operators ("Functions") for operating on input data:

"[T]he connectors of the panel components in the diagram can be connected to each other or to additional elements, such as file management functions, in- and outputs of data acquisition cards, mathematical operators ('Functions', Fig. 4), by a wiring tool. The large amount of different control and indicator elements makes it easy to form an attractive and structured GUI."

Kraus 1999, Ex. 1025, p. 312, top right.

215. Examples of such functions in use are shown in Fig. 6 of Krauss 1999:



216. In figure 6, various mathematical and logical operators are applied to sensor data and in control loops within a VI or sub-VI.

- h) "a multi-mode graphical user interface host comprising diagnostic measurement user interfaces including display components corresponding to data output rendered by specified ones of the plurality of measurement processing components"**

217. The '669 patent further teaches displaying the results of this processing. The '669 patent states:

"The computer 25 processes the pressure data which is sensed by the pressure sensors 4 and acquired by the A/D converter 28 or the data acquisition card (not shown) and generates textual, numerical and/or graphic data that is displayed on the display 21."

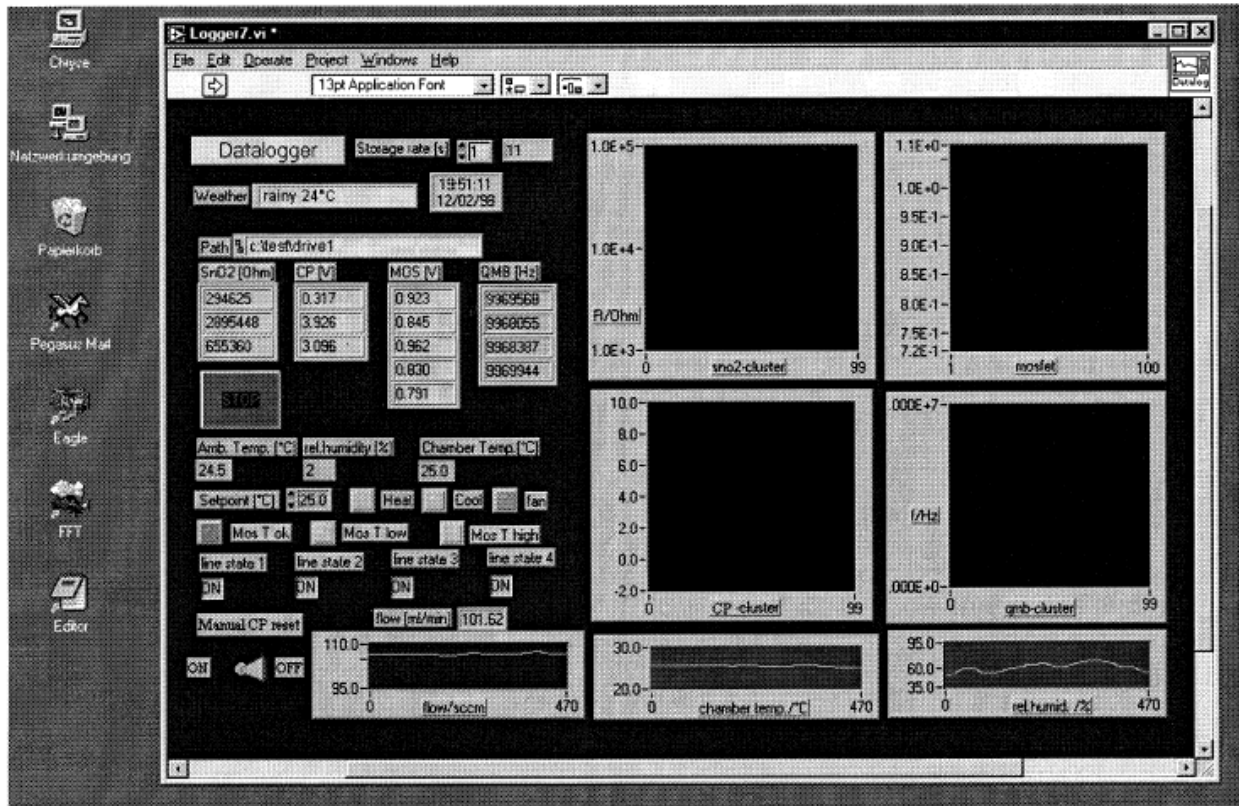
'669 patent, Ex. 1024, col. 13, ll. 25-29.

218. Specific displays for Procedures 1-3, for example, are given in Figs. 17, 23 and 27-28 of the '669 patent. '669 patent, Ex. 1024, col. 5, ll. 58-60, col. 6, ll. 10-12, and col. 6, ll. 23-29.

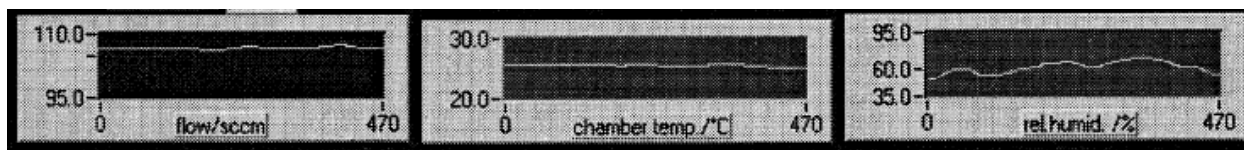
- i) "a plurality of measurement processing components, that operate at a user mode level in the multipurpose host system, for facilitating receiving data of particular sensor types and rendering diagnostic measurement parameter values according to the received data;"**

219. Likewise, Krauss 1999 teaches having measurement processing components (virtual instruments or sub-virtual instruments), which as user applications would run in user mode (see paragraphs 24-25 and 59) and "render" sensor data for display. Krauss 1999 states that its code is "designed for

programming data acquisition, data handling and data display." Kraus 1999, Ex. 1025, p. 312. For example, the graphing of sensor data is shown in Fig. 7 of Krauss 1999:



220. The specific example of Fig. 7 involved "software [that] was written to acquire, process, display and save data from different chemical sensors for monitoring indoor quality in a car." Sensor outputs and diagnostic calculations are displayed as separate components in one or more GUI windows. Kraus 1999, Ex. 1025, p. 316, section 3. As can be seen from Fig. 7 of Krauss 1999, a single window displays the output of several sensors, including air flow, temperature and humidity across the bottom of Fig. 7, as shown in the Figure excerpt here:



221. It would have been obvious to use Labview as described in Krauss 1999 to acquire and process data for the '669 patent system. The '669 patent expressly suggests Labview as the data acquisition tool. '669 patent, col. 17, ll. 50-61. Labview was also manufactured by National Instruments, the manufacturer of the peripheral interface card cited in the '669 patent. *See id.*

222. Krauss 1999 teaches that Labview was "designed for programming data acquisition, data handling and data display"—exactly the general tasks required in the '669 patent. Krauss 1999 further describes Labview as a "very powerful system for software development in teamwork," and states that it was "easy to form an attractive and structured GUI." This would have indicated to a person of ordinary skill that Labview was an appropriate tool to implement a system for acquiring, processing and displaying data from invasive cardiovascular sensors. Krauss 1999 further describes the advantages to using a component-based arrangement (e.g. the division into VIs and sub-VIs to handle discrete functionality), which makes Labview software extensible and modular.

223. Specifically, it would have been obvious to use the teachings of Krauss 1999 to implement the multimode display of the '669 patent. Krauss 1999

teaches an easy method for placing multiple diagnostic measurement user interfaces on the same display panel, in accordance with Fig. 3 (reproduced here with highlighting added by the Petitioner):

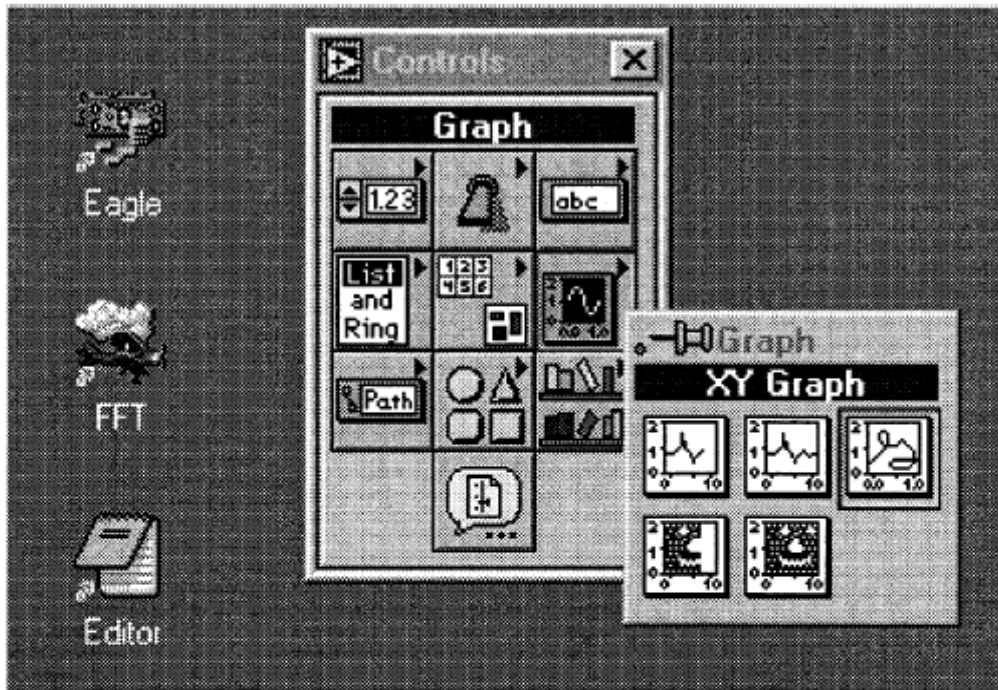


Fig. 3. The controls window, here with the pull-down menu 'graph' with predefined XY graphs. The elements in the menu (e.g. the marked and shaded XY graph) can be picked and placed in the panel, the corresponding function (with the connectors for data input) automatically appears in the diagram.

224. The '669 patent, in turn, teaches the output of multiple types of displays for processed diagnostic measurement data, and suggests that multiple displays are found on a single screen. The '669 patent also teaches displaying

multiple "graphs" (plural) resulting from "calculations" (plural) performed by the computer 25. The '669 patent states:

"The signal analyzer 20 includes a computer 25, and optionally a display 21 connected to the computer 25 for displaying text numbers and graphs representing the results of the calculations performed by the computer 25...."

'669 patent, Ex. 1024, col. 13, ll. 25-29.

225. The '669 patent further teaches that multiple measurements are displayed on a single display screen ("Measurements are displayed digitally on a system screen.") '669 patent, Ex. 1024, col. 14, ll. 18-19.

226. Thus, the '669 patent teaches the use of a multimode display, and further suggests the use of Labview. Krauss 1999 describes a Labview implementation that makes multiple displays of processed sensor output on the same panel easy to design, and in fact describes a specific implementation having multiple displays of processed sensor output on the same panel. The combination would therefore have been obvious.

227. The combination is further obvious under *KSR v. Teleflex*. Here, the '669 patent and Krauss 1999 describe known elements. These elements can be combined according to known programming methods—namely those described for Labview in Kraus 1999. Indeed, the '994 patent describes no method for implementing its graphical user interface. This effectively admits that such

implementation was within the reach of a person of ordinary skill. Furthermore, the results are predictable. Specifically, the software as constructed will operate as described in the '669 patent and Kraus 1999. Indeed, the '994 patent describes no unpredictable result that arises from the use of a "multi-mode graphical user interface host".

228. Krauss 1999 is reasonably pertinent to the problems faced by the alleged inventors, and would naturally have commended itself to the alleged inventors' attention. The alleged inventors were concerned with (among other things), creating a modular and extensible sensor-based diagnostic system. This is reinforced by the elements of claim 1 requiring a "component based arrangement" and the acquisition of data from external sensors. Krauss 1999 teaches modular and extensible software specifically for sensor data acquisition and is further specifically referenced by the '669 patent, which is prior art in the same medical field.

229. In describing the use of a standard PC, a National Instruments board and Labview software, the '669 patent renders obvious an invasive cardiovascular sensor system having a component-based arrangement. The components are the user-mode virtual instruments and sub-virtual instruments of the Labview software, where each sub-VI corresponds to a particular sensor function. The components acquire data from a peripheral interface card through kernel mode

drivers into user-mode virtual instruments and sub-virtual instruments, and apply processing to them. The processing would obviously have been of the type described in the '669 patent, for example the pressure sensor signal analysis for diagnostic purposes (669 patent, Ex. 1024, col. 9, l. 35 – col. 11, l. 41) or the more specific processing described in columns 18-22 of the '669 patent, although a person of ordinary skill in the art would not have felt limited by that disclosure.

230. The combination would also have been of known components, by known methods, yielding predictable results. Labview as described by Krauss 1999 and Windows NT as described by *Inside Windows* were known and well-regarded software products in the timeframe prior to the alleged invention. Each was designed for flexibility in designing and running a variety of applications. Labview was also specifically intended to integrate external sensors into a PC. The combination would have exhibited no unpredictable results, and neither the '669 patent nor the '994 patent describes any unpredictable results.

2. Claim 10

231. Claim 10 reads: "*10. The multipurpose host system of claim 1 wherein the plurality of measurement processing components are instantiated from component objects.*"

232. Regarding claim 10, Krauss 1999 describes implementing sensors as VIs or sub-VIs, which has the effect of increasing the modularity of the system.

Kraus 1999, Ex. 1025, pp. 316-317. Each VI or sub-VI is a component. As

Krauss 1999 explains:

"LabView allows the integration of program code into 'sub-VIs'. Instead of the code, just an icon with terminals for wires appears in the diagram of the virtual instrument. Double-clicking on the icon opens the sub-VI and the code can be edited. LabView thus allows a modular layout of a program with an easy to define interface which is the panel of the sub-VI. This possibility should be used to maximum extent...."

Kraus 1999, Ex. 1025, pp. 315-316.

233. As explained in Krauss 1999, instances of sub-VIs are created (*i.e.* "instantiated") from VIs or other sub-VIs ("component objects") by the user to form modular sub-VIs.

3. Claim 11

234. Claim 11 reads: "*11. The multipurpose host system of claim 1 wherein the multi-mode graphical user interface host supports the set of diagnostic measurement user interfaces through a generic graphical interface specification.*"

235. Regarding claim 11, Figure 3 of Krauss 1999 shows that users of Labview are able to click and drag display components to build a layout for the Labview GUI. This means that the layout exists in a data structure managed by Labview. Figure 3 and its caption are shown here:

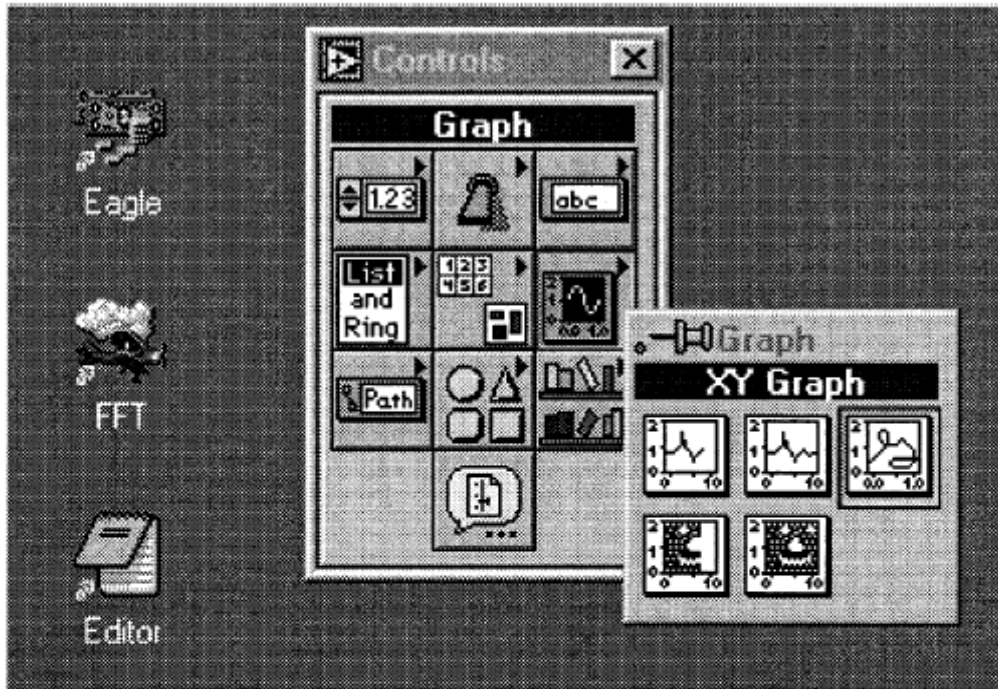


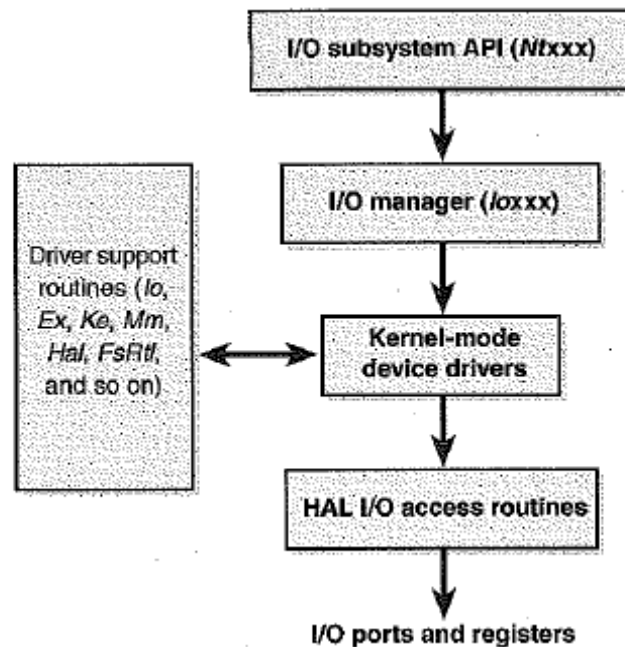
Fig. 3. The controls window, here with the pull-down menu 'graph' with predefined XY graphs. The elements in the menu (e.g. the marked and shaded XY graph) can be picked and placed in the panel, the corresponding function (with the connectors for data input) automatically appears in the diagram.

4. Claim 18

236. Claim 18 reads: "*18. The multipurpose host system of claim 1 further including a sensor component application interface through which the measurement processing components issue requests to the kernel mode drivers.*"

237. Regarding claim 18, communication in a Windows NT system from an application in user mode to a driver in kernel mode almost always happened via an "application programming interface" or API. One could imagine ways of

communicating to avoid an API, but these ways would have been unusual and artificial. The usual architecture is shown in Fig. 7-1 of *Inside Windows*:



238. Here, the I/O subsystem API is the API that would typically have been used between an application component in user mode and a driver in kernel mode.

C. Discussion Of Obviousness Of Claims 2, 4-6, 8-9, 12-13, and 15-17 Over The '669 Patent, Krauss 1999, *Inside Windows* And The SmartFlow Manual.

239. It is my opinion that claims 2, 4-6, 8-9, 12-13, and 15-17 are unpatentable under 35 U.S.C. § 103(a) over the '669 patent, when supplemented or modified by Krauss 1999, *Inside Windows* (as discussed above), and the SmartFlow™ Integrated Lumen Physiology Operator's Manual, Version 5.0

("SmartFlow Manual")(Ex. 1027). I understand from the declaration of Barack that the SmartFlow Manual is printed publication prior art.

240. The SmartFlow Manual describes the "SmartFlow" system from Florence Medical (see paragraphs 190 to 193, above). This is the same owner as the '669 patent, and relates to the same system.

241. Furthermore, both the SmartFlow Manual and the '669 patent are directed to the problem of diagnosing arterial lesions with invasive sensors. '669 patent, Ex. 1024, col. 3, l. 66 – col. 4, l. 6; SmartFlow Manual, Ex. 1027, p. 5, §1.1. The '669 patent and the SmartFlow Manual both use invasive sensors to transmit signals to a signal conditioning unit, which then transmits signals to a National Instruments board to be acquired by a PC, which processes and displays the processed information. '669 patent, Ex. 1024, col. 17, ll. 48-56 and col. 13, ll. 9-16; SmartFlow Manual, Ex. 1027, p. 10, §3.1.

242. The combination of the '669 patent and the SmartFlow Manual would also have been obvious under *KSR v. Teleflex*. Both systems represent known prior art elements that could be combined according to known methods, yielding no unpredictable result (and none is described in the '994 patent). As described below in paragraphs 431, *et seq.*, the SmartFlow system was a real commercial system in the relevant timeframe, thus indicating that there were no unpredictable results or difficulties in implementing its techniques.

1. Claim 2

243. Claim 2 reads: *"2. The multipurpose host system of claim 1 wherein the plurality of measurement processing components includes a blood pressure processing component and a blood velocity processing component."*

244. Regarding claim 2, the SmartFlow Manual discloses measurements of pressure and extensive calculation and rendering of pressure measurements.

SmartFlow Manual, Ex. 1027, p. 5, §1.1 and Figs. 15, 17 and 19. Regarding blood velocity, the SmartFlow Manual describes processing to compute Coronary Flow Reserve ("CFR"). SmartFlow Manual, Ex. 1027, p. 6. The SmartFlow Manual does not expressly disclose that a blood velocity measurement processing component would be necessary for this. However, the '669 patent discloses that:

"Among the more sophisticated diagnostic tools are...intravascular Doppler velocity sensor (IDVS)....IDVS provides velocity measurements, enabling discriminating various degrees of occlusion according to coronary flow reserve (CFR) criteria."

'669 patent, col. 2, ll. 50-64.

245. Thus, it would have been obvious to use a known Doppler velocity sensor to measure velocity values as taught by the '669 patent, and process them into CFR values as taught by the '669 patent and the SmartFlow Manual. It would also have been obvious to provide separate Labview measurement processing

components (sub-VIs) relating to the pressure and blood velocity processing for the reasons given under paragraphs 209 to 212.

2. Claims 4 and 15

246. Claim 4 reads: *"4. The multipurpose host system of claim 2 wherein the diagnostic measurement user interfaces include a pressure measurement display interface wherein a ratio of a first measured pressure and a second measured pressure is rendered."* Claim 15 reads: *"15. The multipurpose host system of claim 12 wherein a pressure display mode supports rendering a fractional flow reserve value."*

247. Regarding claims 4 and 15, the SmartFlow Manual teaches computing FFR, which is "the ratio of the mean post-stenotic pressure to the aortic pressure during vasodilatation." SmartFlow Manual, Ex. 1027, p. 7. The SmartFlow manual further teaches displaying on the user interfaces the FFR. SmartFlow Manual, Ex. 1027, p. 17. Figure 20 of the SmartFlow shows a display of CFR and FFR values. Figure 20 is reproduced here:

Plot CFR/FFR

The program provides a graphic representation for CFR and FFR values. These graphs present the CFR and FFR values per pulse and may give an indication of the quality of vasodilatation. This option may be called from the Measure and the Self Test screens by selecting the Data / Plot CFR FFR graph and Calculation / Plot CFR FFR options respectively.

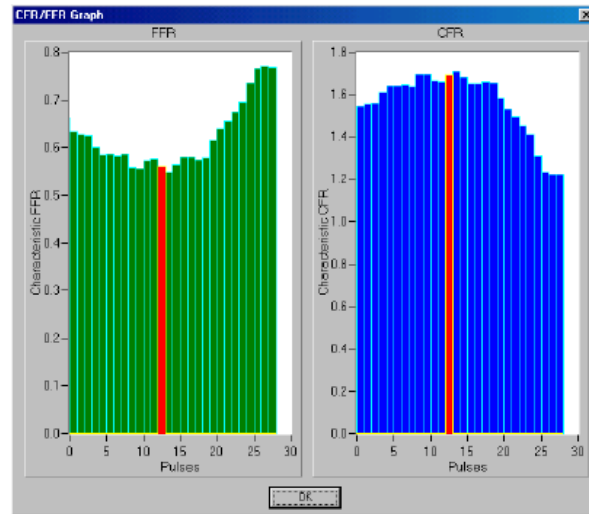


Fig 20: A graphic presentation of the CFR and FFR values per beat.

248. Figure 17 on page 27 also shows a display of FFR in a pressure display mode (note display of pressure readings and pressure gradients):

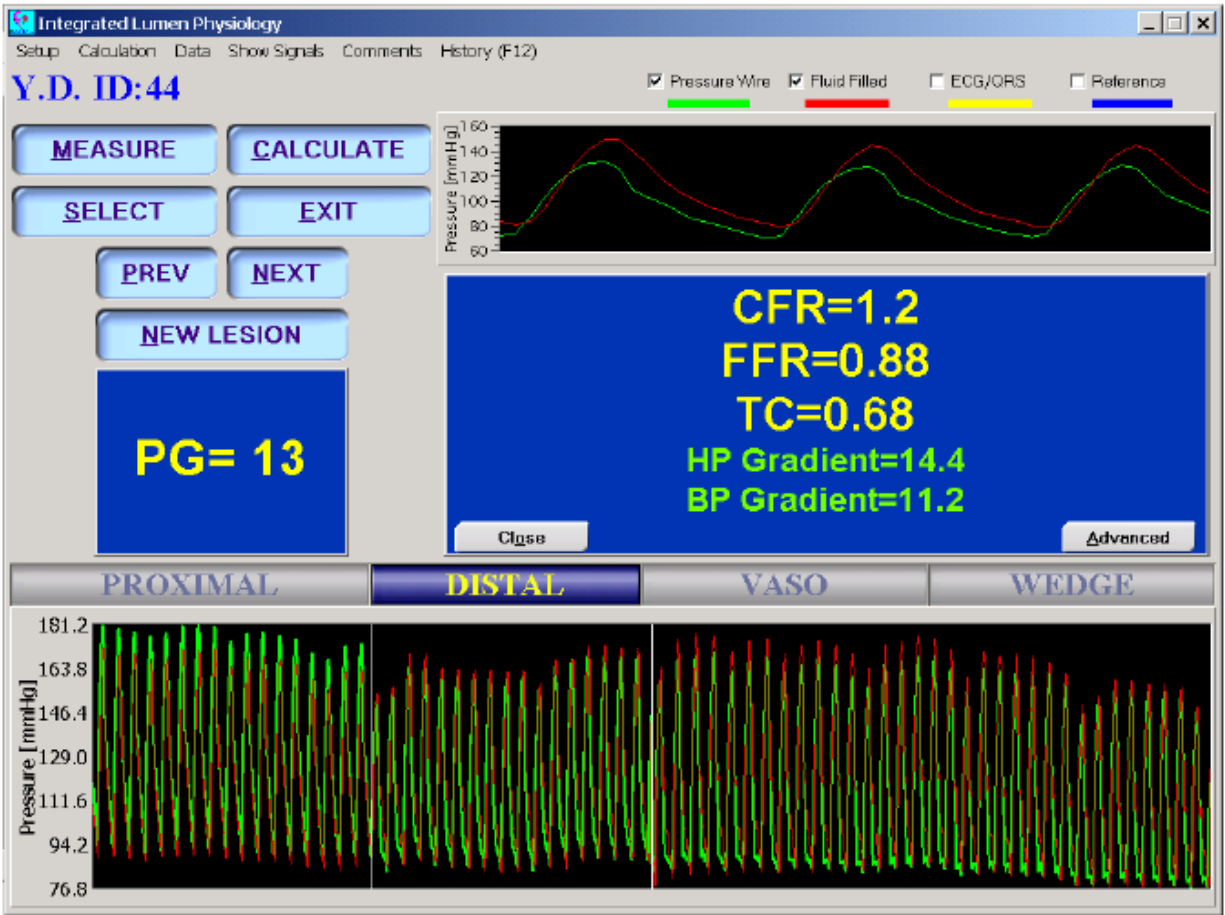


Fig 17: Discrete lesion results screen

3. Claim 5

249. Claim 5 reads: "5. The multipurpose host system of claim 2 wherein the diagnostic measurement user interfaces include a pressure measurement display interface wherein a pressure gradient between a first measured pressure and a second measured pressure is rendered."

250. Regarding claim 5, The SmartFlow Manual teaches displaying on the user interfaces several different pressure gradients:

"Push Calculate and the program result screen will pop-up, presenting the CFR and FFR, hyperemic pressure gradient (HPG) and base pressure gradient (BPG) computed for the total lesion. Additionally, the calculated total pressure drop at rest which would yield $FFR_{target}=0.9$ and $FFR_{target}=0.8$ are presented. The multiple lesion graphic result will be displayed as well, indicating the baseline pressure gradient of each individual lesion."

SmartFlow Manual, Ex. 1027, p. 32.

4. Claim 6

251. Claim 6 reads: *"The multipurpose host system of claim 2 wherein the diagnostic measurement user interfaces include a flow measurement display interface wherein a ratio of flow velocity under high flow conditions and low velocity conditions is rendered."* Regarding claim 6, the SmartFlow Manual discloses calculating CFR. CFR is the "ratio between the hyperemic flow to the flow at rest in an artery". SmartFlow Manual, Ex. 1027, p. 6. The SmartFlow manual further teaches displaying CFR. For example, Fig. 20 shown above displays CFR, as does Figure 17 on page 27 (both shown above in paragraphs 247 and 248). It would have been obvious to measure flow as taught by the '669 patent, for the reasons given in paragraphs 244 to 245.

5. Claim 8

252. Claim 8 reads: *"8. The multipurpose host system of claim 1 wherein the multi-mode graphical user interface host includes a first display interface mode*

wherein blood pressure measurements are displayed and a second display interface mode wherein blood flow measurements are displayed."

253. Regarding claim 8, The SmartFlow Manual discloses allowing a user to switch between pressure and flow display modes using a "show signals menu":

"Once entering the Measure Screen, data acquisition is automatically activated, and pressure tracings are displayed on the top display. One may use the show signals menu on the top left corner to have a better look at all relevant signals (pressures, ECG and **optionally flow**)."

SmartFlow Manual, Ex. 1027, p. 25 and Fig. 15 (emphasis added).

254. It would therefore have been obvious to include a first mode for pressure measurements and a second mode for flow measurements.

6. Claim 9

255. Claim 9 reads: "9. *The multipurpose host system of claim 8 wherein the multi-mode graphical user interface host further includes a third display interface mode graphically depicting a combination of display elements including one or more pressure measurement display elements from the first display interface mode and one or more flow measurement display elements from the second display interface.*"

256. Regarding claim 9, as discussed above with respect to claim 6 in paragraph 251, the SmartFlow Manual in connection with Figures 17 and 20 teaches the display of both FFR and CFR. In particular with regard to claim 17,

the signals can be changed between pressure and flow, as shown below in paragraph 265. It would therefore have been obvious to include a combination of display elements having both flow and pressure display elements.

7. Claim 12

257. Claim 12 reads: *"12. The multipurpose host system of claim 11 wherein the generic graphical interface specification includes: a patient information display region; and a mode-specific data display region including graphical display components corresponding to data output rendered by specified ones of the measurement processing components associated with a current display mode."*

258. Regarding claim 12, Figs. 15, 17 and 19 of the SmartFlow Manual show "Measure" windows that have measurement processing information (see, e.g., paragraphs 240 to 241 above). The windows also include patient information in the form of a name (initials only in the Manual) and patient ID near the upper left corner. The SmartFlow Manual states:

"MEASURE. The program displays the preselected (Patient Select option in Setup) patient name and ID number at the top of the screen."

SmartFlow Manual, Ex. 1027, p. 22.

259. As discussed above in paragraphs 253 to 256, these screens also show data from pressure, ECG and flow sensors, which would have been obvious to

render by specified ones of the measurement processing components for reasons provided in paragraphs 209 to 212. Because the rendered data is displayed in the modes shown, the measurement processing components are necessarily "associated with the display mode".

260. Because the patient information is found in all measurement display modes in a consistent location relative to the data display regions, it would have been obvious to provide a layout specifying where the patient information is to be displayed relative to the graphical display components in the data display region.

8. Claim 13

261. Claim 13 reads: *"13. The multipurpose host system of claim 12 wherein the generic graphical interface specification includes a first set of tabs for selecting one of a set of supported display modes, wherein each supported display mode includes a specified set of graphical display components rendered in the mode-specific data display region."*

262. Regarding claim 13, The SmartFlow Manual teaches displaying a variety of tabs in its various display modes, for example, "proximal", "distal", "vaso" and "wedge" in Figs. 15, 17 and 19. SmartFlow Manual, Ex. 1027, pp. 25, 27-28.

263. Because these tabs are found in all display modes, it would have been obvious to include them in the generic graphical interface specification (GUI

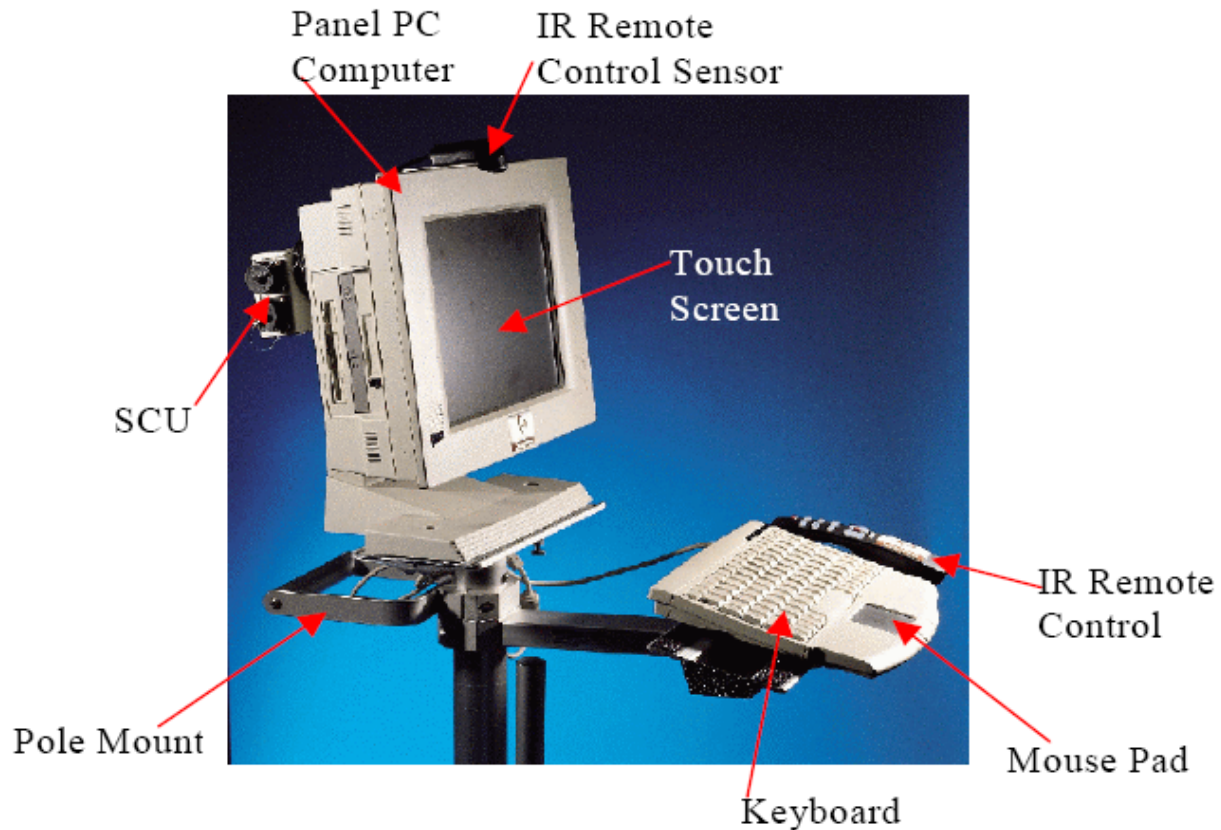
layout) discussed in paragraphs 208 and 217 to 218. As shown in Figs. 15, 17 and 19 of the SmartFlow Manual, each tab corresponded to a display mode that included a graph.

9. Claim 16

264. Claim 16 reads: "*16. The multipurpose host system of claim 11 wherein the generic graphical user interface specification includes a touch screen interface for selecting display components.*"

265. Regarding claim 16, the SmartFlow Manual teaches a touch screen interface:

"The SmartFlow™ is a PC based system comprising of a full color display with a touchscreen for patient data entry and control, and software calculations. The display and touchscreen are mounted in a shielded metal housing. The SmartFlow™ is presented in Figure 1."



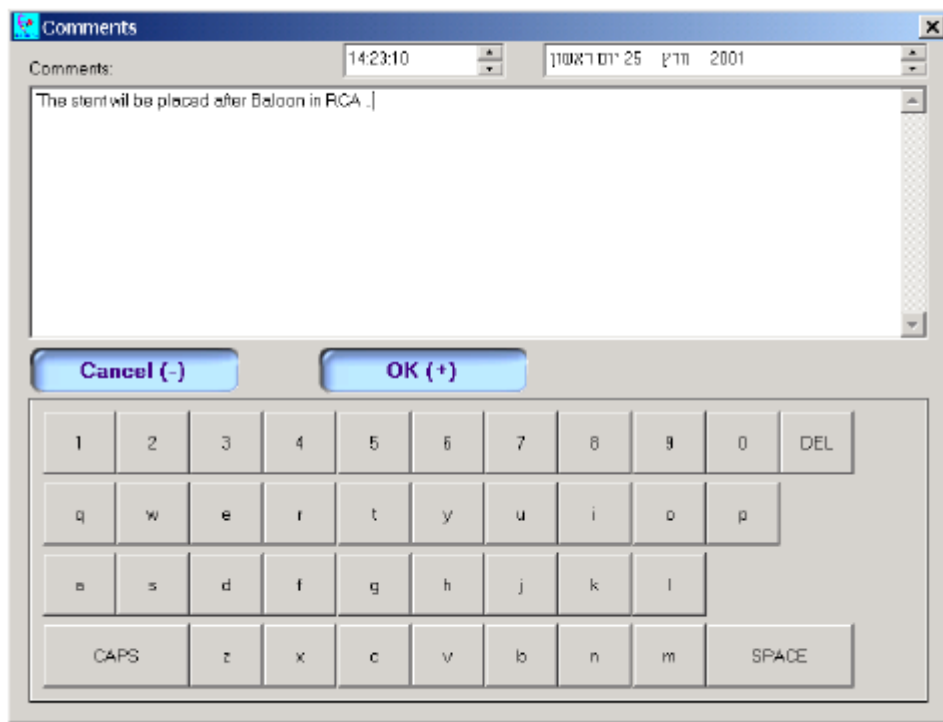
SmartFlow Manual, p. 10, Fig. 1.

266. It would have been obvious to include the elements of the touchscreen interface within the layout for the GUI elements. For example, the buttons on the keyboard are arranged in a QWERTY order. Within the SmartFlow system, it would have been obvious to include layout data indicating, for example, that the "Q" key needs to appear to the left of the "W" key, which is above the "S" key, and so forth. This would have been true for any element that could have been used by activated by the touchscreen for "patient data entry" and for "control".

10. Claim 17

267. Claim 17 reads: "17. *The multipurpose host system of claim 16 wherein a touch screen keyboard is supported by the generic graphical user interface specification.*"

268. As shown in Fig. 24 on page 31 of the SmartFlow Manual (shown below), the touchscreen interface includes a keyboard where necessary:



269. It would have been obvious to include the elements of the touchscreen interface within the layout for the GUI elements for the reasons given in paragraph 266.

A. Invalidity Of Dependent Claims 2-9 and 12-17 and 19-20.

270. It is further my opinion that certain dependent claims would have been obvious modifications of the teachings in the '669 patent in combination with *Inside Windows* and Krauss 1999 claim 1 (see paragraphs 194 to 230, above). I will refer to these teachings collective as "the SmartFlow patent teachings", because they represent the application of common computing techniques to the system described in the '669 patent, which in turn was based on the SmartFlow system. To be clear, the following analysis is not based on the SmartFlow Operator's Manual discussed beginning in paragraph 239, above.

271. The dependent claims that would have been obvious, and the prior art references that provide the necessary teachings are listed as follows:

- Claims 8-9 are rendered obvious by modifying the SmartFlow patent teachings with Kern, *et al.*, J. Am. Coll. Cardiol. 1997; 30(3):613-620 ("Kern 1997").
- Claims 2 and 4-6 are rendered obvious by modifying the SmartFlow patent teachings with Kern 1997 and U.S. Pat. No. 5,873,835 ("Hastings").
- Claim 19 is rendered obvious by modifying the SmartFlow patent teachings with Hastings and U.S. Pat. No. 6,754,608 ("Svanerudh").

- Claim 3 is rendered obvious by modifying the SmartFlow patent teachings with Kern 1997, Hastings and U.S. Pat. No. 6,083,170 ("Ben-Haim").
- Claim 7 is rendered obvious by modifying the SmartFlow patent teachings with Kern 1997, Hastings, Ben-Haim and Svanerudh.
- Claims 12-13 and 20 are rendered obvious by modifying the SmartFlow patent teachings with U.S. Pat. No. 6,322,502 ("Schoenberg").
- Claims 14 and 16-17 are rendered obvious by modifying the SmartFlow patent teachings with Schoenberg and U.S. Pat. No. 5,724,985 ("Snell").
- Claim 15 is rendered obvious by modifying the SmartFlow patent teachings with Schoenberg, Hastings and Kern 1997.

272. Kern 1997 is a journal article from the Journal of the American College of Cardiology. It describes the diagnostic value of the Coronary Flow Reserve (CFR) and Fractional Flow Reserve (FFR) measurements that were later described in the '994 patent (e.g. '994 patent. col. 1, l. 48 – col. 2, ll. 22).

273. Hastings is a U.S. patent that describes sensors for intracoronary pressure and flow velocity measurements, which are useful for calculating the CFR and FFR values described by Kern 1997.

274. Like Hastings, Ben-Haim is a U.S. patent that describes sensors for intracoronary measurements. Ben-Haim is focused on the problem of properly positioning sensors within a blood vessel.

275. Svanerudh is a U.S. patent that describes a multimode graphical user interface system having multiple external sensors for carrying out cardiovascular diagnostics.

276. Schoenberg is a U.S. patent describing a multimode graphical user interface system for patient information display. The system collects and integrates various kinds of patient and sensor data into a single Windows-based GUI system.

277. Like Schoenberg, Snell is a U.S. patent describing a multimode graphical user interface system for patient information display. The system displays various kinds of patient and sensor data into a single Windows-based GUI display for a portable computer.

278. Each of Kern 1997, Hastings, Ben-Haim and Svanerudh falls within the narrow field of cardiovascular diagnostics. Each of Schoenberg and Snell deals with the display of multiple types of medical information, including medical sensor information. The references are thus all in the same field of endeavor as the '994 patent. Similarly, these references would have been reasonably pertinent to the problems faced by the alleged inventors of the '994 patent. In particular, Kern

1997 teaches the desirability of certain known cardiovascular diagnostic techniques (including CFR and FFR), and Hastings, Ben-Haim and Svanerudh teach ways to achieve them. Furthermore, Svanerudh, Schoenberg and Snell are directed to the best ways to display multiple different types of information, such that a clinician can easily access the information, much like the '994 patent.

279. I have reviewed the petition for inter partes review, including the claim charts which correspond the elements of claims 2-9 and 12-17 and 19-20 to the Kern 1997, Hastings, Ben-Haim, Svanerudh, Schoenberg and Snell references. I am of the opinion that the petition is accurate, and that the claims would have been obvious to a person of ordinary skill in the art.

1. Claim 8

280. Claim 8 reads: "*8. The multipurpose host system of claim 1 wherein the multi-mode graphical user interface host includes a first display interface mode wherein blood pressure measurements are displayed and a second display interface mode wherein blood flow measurements are displayed.*"

281. Kern 1997 teaches displaying blood flow velocity related measurements in Fig. 1, and separately in Fig. 2, displaying pressure related measurements. Kern 1997, Ex. 1030, pp. 614-615, Figs. 1 and 2.

282. Kern 1997 teaches that it is valuable to measure both pressure and blood velocity values:

"Because each current physiologic method individually reflects only one aspect of the pressure–flow relation, borderline or ambiguous data, acquired by either pressure or flow velocity alone, can be theoretically confirmed using the complementary technique. A combined guide wire device with both pressure and flow could eliminate questions related to borderline values."

Kern 1997, Ex. 1030, p. 616.

283. It would have been obvious to display both kinds of measurements. Once the data is collected, a related measurement must be presented to the clinician in order to have value.

284. Claim 8 also would have been obvious under *KSR v. Teleflex*. As Kern 1997 clearly shows, it was desirable to display both pressure and flow measurements. See paragraph 282, above; Kern 1997, Ex. 1030, p. 616. Because a standard PC running a Microsoft Windows system can be programmed to perform essentially arbitrary calculations and display data in an arbitrary fashion, with no unexpected results, it would have been obvious to implement these in the system of the '994 patent if desired by the clinician.

2. Claim 9

285. Claim 9 reads: "9. *The multipurpose host system of claim 8 wherein the multi-mode graphical user interface host further includes a third display interface mode graphically depicting a combination of display elements including*

one or more pressure measurement display elements from the first display interface mode and one or more flow measurement display elements from the second display interface."

286. Kern 1997 teaches displaying blood flow velocity related measurements in Fig. 1, and separately in Fig. 2, displaying pressure related measurements. Kern 1997, Ex. 1030, pp. 614-615, Figs. 1 and 2.

287. It would have been obvious to display both kinds of measurements for the reasons given above under paragraphs 281 to 284. Furthermore, the combination of two forms of measurement data, both known in the prior art to be valuable, into a combined visual representation by known techniques has no unpredictable result, and is thus obvious under *KSR v. Teleflex*. The '994 patent does not provide any teachings concerning how to create the third user interface mode, nor does the '994 patent describe any unexpected results.

3. Claims 2 and 4-6

288. Claim 2 reads: "*2. The multipurpose host system of claim 1 wherein the plurality of measurement processing components includes a blood pressure processing component and a blood velocity processing component.*"

289. With regard to claims 2 and 4-6, it would have been obvious to combine the SmartFlow patent teachings with Kern 1997 for the reasons given above, and with Hastings. Kern 1997 teaches that both pressure and flow

measurements are desirable Kern 1997, Ex. 1030, p. 616. Hastings teaches that "[i]t would be desirable to make use of both measurement techniques in order to provide the treating physician with as much diagnostic information as possible." Hastings, Ex. 1031, col. 1, ll. 56-621, ll. 56 - col. 2, l. 7.

290. Hastings teaches a combined pressure and flow sensor that ideally allows combined measurements to be made. Hastings, Ex. 1031, col. 2, ll. 7-17). Hastings teaches that blood flow measurements can be made using blood velocity measurements in a constant diameter lumen. Hastings, Ex. 1031, col. 12, ll. 13-16. It would thus have been obvious to add components for these sensors for the reasons given in paragraphs 209 to 212, above.

291. Regarding claim 2 specifically, Hastings teaches that there are at least two methods for measuring the severity of an arterial occlusion: pressure measurements (Hastings, Ex. 1031, col. 1, ll. 39-41) and blood flow measurements Hastings, Ex. 1031, col. 1, ll. 56-62. Blood flow measurements can be made using blood velocity measurements in a constant diameter lumen. Hastings, Ex. 1031, col. 12, ll. 13-16.

292. Claim 4 reads: "4. *The multipurpose host system of claim 2 wherein the diagnostic measurement user interfaces include a pressure measurement display interface wherein a ratio of a first measured pressure and a second measured pressure is rendered.*"

293. Regarding claim 4, Kern 1997 teaches in Fig. 2 teaches calculation and display of "Fractional Flow Reserve of myocardium" ("FFRmyo"). Kern 1997, Ex. 1030, p. 614 (top left) and 615, Fig. 2) According to Kern 1997: "The FFRmyo is computed as the ratio of distal mean coronary pressure and mean aortic pressure during maximal hyperemia...." Kern 1997, Ex. 1030, p. 615 (left column).

294. Claim 5 reads: *"5. The multipurpose host system of claim 2 wherein the diagnostic measurement user interfaces include a pressure measurement display interface wherein a pressure gradient between a first measured pressure and a second measured pressure is rendered."*

295. Regarding claim 5, Kern 1997 teaches in Figure 2 displaying a pressure measurement display having a gradient value between a first and second pressure. Kern 1997 states: "Fig. 2 Translesional pressure measurements made with a fiber optic guidewire (Radi, Inc.) were used to compute FFRmyo. The initial translesional pressure gradient (aortic [Pa] – Distal coronary [Pd]) is <3 mm Hg (left side) before inducing hyperemia." Kern 1997, Ex. 1030, p. 615, caption to Fig. 2.

296. Claim 6 reads: *"6. The multipurpose host system of claim 2 wherein the diagnostic measurement user interfaces include a flow measurement display*

interface wherein a ratio of flow velocity under high flow conditions and low velocity conditions is rendered."

297. Regarding claim 6, Kern 1997 in Fig. 1B teaches calculation and display of "Coronary Flow Reserve" ("CFR"). Kern 1997, Ex. 1030, p. 614, Fig. 1B. According to Kern 1997: "Coronary flow reserve (CFR) is computed as the ratio of maximal hyperemic to basal mean coronary flow velocity." Kern 1997, Ex. 1030, p. 614 (right column). It would further have been obvious to compute and display this value because Kern 1997 teaches that:

"With the introduction of sensor-tipped angioplasty guide wires, distal coronary flow velocity and pressure can be obtained in the cardiac catheterization laboratory, facilitating physiologically based decisions regarding the need for intervention. In the catheterization laboratory, physiologically significant stenoses can be characterized as having impaired post-stenotic coronary flow reserve <2.0 and pressure-derived fractional flow reserve <0.75 , both variables related strongly to positive ischemic perfusion imaging or stress testing results."

Kern 1997, Ex. 1030, Abst.

298. It would further have been obvious to combine the SmartFlow patent teachings with Kern 1997 and Hastings as a combination of known techniques by known methods leading to a predictable result. Hastings describes a standard sensor used for invasive cardiovascular diagnosis. Hastings, Ex. 1031, Abstract. Hastings can be combined with the SmartFlow patent teachings using standard

techniques (e.g. adding the sensor to the guidewire), to achieve only predictable results. The '994 patent describes neither how to implement sensors, nor any unpredictable results, and admits that different temperature sensors can be used predictably ('994 patent, Ex. 1001, col. 8, ll. 46-51).

4. Claim 19

299. Claim 19 reads: *"19. The multipurpose host system of claim 1 wherein the plurality of measurement processing components includes a temperature processing component and a blood pressure processing component."*

300. Regarding claim 19, Hastings teaches that in situ temperature measurements can be made to calibrate instrumentation. Hastings, Ex. 1031, col. 12, ll. 52-57. Hastings is concerned with CFR measurements. CFR measurements are useful in diagnosis as discussed above under claim 6 (paragraph 251). Svanerudh is also concerned with the calculation of CFR. Svanerudh teaches that temperature measurements can be used to calculate CFR in an improved fashion. Svanerudh Prov., Ex. 1033, p. 1, ll. 12-24, p. 9, ll. 7-13.

301. Regarding pressure, a measurement processing component is discussed above under paragraphs 289 to 291.

302. It would have been obvious to combine the SmartFlow patent teachings with Hastings for the reasons given in paragraphs 244. It would further have been obvious specifically to take and process temperature measurements in

order to allow correction for temperature drift of instrumentation as taught in Hastings (Hastings, Ex. 1031, col. 12, ll. 52-57), or to allow for calculation of CFR as taught by Svanerudh (Svanerudh Prov., Ex. 1033, p. 1, ll. 12-24, p. 9, ll. 7-13).

303. It would further have been obvious to combine the SmartFlow patent teachings with Hastings and Svanerudh under *KSR v. Teleflex*. Svanerudh represents a standard methodology used for the type of work described in the '994 patent. Svanerudh Prov., Ex. 1033, p. 6, ll. 16-24. Svanerudh can be combined with the SmartFlow patent teachings using standard techniques (e.g. adding the sensor to the guidewire and employing the calculations of Svanerudh in the PC of the Prucka Manual), to achieve only predictable results. The '994 patent does not describe any unpredictable results, and in fact admits that different temperature sensors can be used predictably. '994 patent, Ex. 1001, col. 8, ll. 46-51.

5. Claim 3

304. Claim 3 reads: "*3. The multipurpose host system of claim 2 wherein the plurality of measurement processing components includes an invasive device position sensor processing component.*"

305. Regarding claim 3, Ben-Haim teaches including a position sensor at the tip of an invasive guidewire. Ben-Haim, Ex. 1034, col. 3, ll. 14-27. It would have been obvious to combine the SmartFlow patent teachings with Ben-Haim. Ben-Haim is directed to systems for medical diagnosis using intravascular medical

catheters (Ben-Haim, col. 1, ll. 11-13), just like the SmartFlow patent teachings and the '994 patent. '994 patent, Ex. 1001, col. 1, ll. 26-31.

306. Ben-Haim teaches the use of a position sensor as required by claim 3. It would have been obvious to include a position sensor, because Hastings (see above, paragraphs 289 to 291 with respect to claim 2) teaches that positioning of the sensors at the correct point in the artery is important, and that positioning can be done in a variety of ways known in the art. Hastings, Ex. 1031, col. 12, ll. 52-57. Ben-Haim further teaches that positioning is standard practice, and that a position sensor can help reduce the need for X-Ray positioning techniques that cause radiation damage, and can help reduce the likelihood that collision with an arterial obstruction. Ben-Haim, Ex. 1034, col. 1, ll. 35-42, col. 3, ll. 14-27.

307. Furthermore, Ben-Haim teaches a standard position sensor used for the type of work described in the '994 patent. Ben-Haim, Ex. 1034, col. 3, ll. 14-27. Ben-Haim can be combined with the SmartFlow patent teachings using standard techniques (e.g. adding the sensor to the guidewire), to achieve only predictable results. The '994 patent describes neither how to implement sensors, nor any unpredictable results, and in fact admits that different temperature sensors can be used predictably. '994 patent, Ex. 1001, col. 8, ll. 46-51.

308. It further would have been obvious to process the position measurement in a processing component for the reasons given in paragraphs 209 to 212.

6. Claim 7

309. Claim 7 reads: "*7. The multipurpose host system of claim 1 wherein the plurality of measurement processing components includes a temperature processing component and an invasive device position sensor processing component.*"

310. Regarding claim 7, Hastings teaches that in situ temperature measurements can be made to calibrate instrumentation. Hastings, Ex. 1031, col. 12, ll. 52-57. Hastings is concerned with CFR measurements. CFR measurements are useful in diagnosis as discussed above in paragraph 297. Svanerudh is also concerned with the calculation of CFR. Svanerudh teaches that temperature measurements can be used to calculate CFR in an improved fashion. Svanerudh Prov., Ex. 1033, p. 1, ll. 12-24, p. 9, ll. 7-13. Regarding position, Ben-Haim teaches including a position sensor at the tip of a guidewire. Ben-Haim, Ex. 1034, col. 3, ll. 14-27.

311. It would have been obvious to include a temperature and positioning sensor, because Hastings teaches that when taking temperature measurements one should position the sensor in a variety of ways known in the art. Hastings, Ex.

1031, col. 12, ll. 37-41. Ben-Haim further teaches that positioning is standard practice, and that a position sensor can help reduce the need for X-Ray positioning techniques that cause radiation damage, and can help reduce the likelihood that collision with an arterial obstruction. Ben-Haim, Ex. 1034, col. 1, ll. 35-42, col. 3, ll. 14-27.

7. Claim 12

312. Claim 12 reads: "*12. The multipurpose host system of claim 11 wherein the generic graphical interface specification includes: a patient information display region; and a mode-specific data display region including graphical display components corresponding to data output rendered by specified ones of the measurement processing components associated with a current display mode.*"

313. Regarding claim 12, a generic graphical user interface specification is found in the SmartFlow patent teachings and Krauss 1999, as discussed under paragraph 235. Schoenberg teaches a graphical user interface having multiple display modes. Schoenberg, Ex. 1035, Figs. 2A – 3B. Each of these display modes represented by Figs. 2A-3B has patient information display regions at the top and bottom, indicating the bed number at the top and the date of admission to the hospital and the number of days in the hospital at the bottom. Because this region is constant across multiple modes (Figs. 2A-3B), it is indicative of a generic

graphical user interface having such a region. Furthermore, each mode has a middle region that is mode-specific, for example, a physician mode in Fig. 3A and a nurse mode in Fig. 3B. Schoenberg, Ex. 1035, col. 7, l. 58 – col. 8, l. 15.

314. Additionally, with respect to Fig. 2B, there is a subset of display regions. Each display region can be mode-selected to show the output of a specified sensor, such as "VITAL SIGNS, RESPIRATION, HEMATOLOGY and BLOOD GASES....FLUID BALANCE, FLUID INPUT and FLUID OUTPUT, or other flow chart parameters." Schoenberg, Ex. 1035, col. 7, ll. 28-32) One such display region is excerpted from the rest of Fig. 2B below:

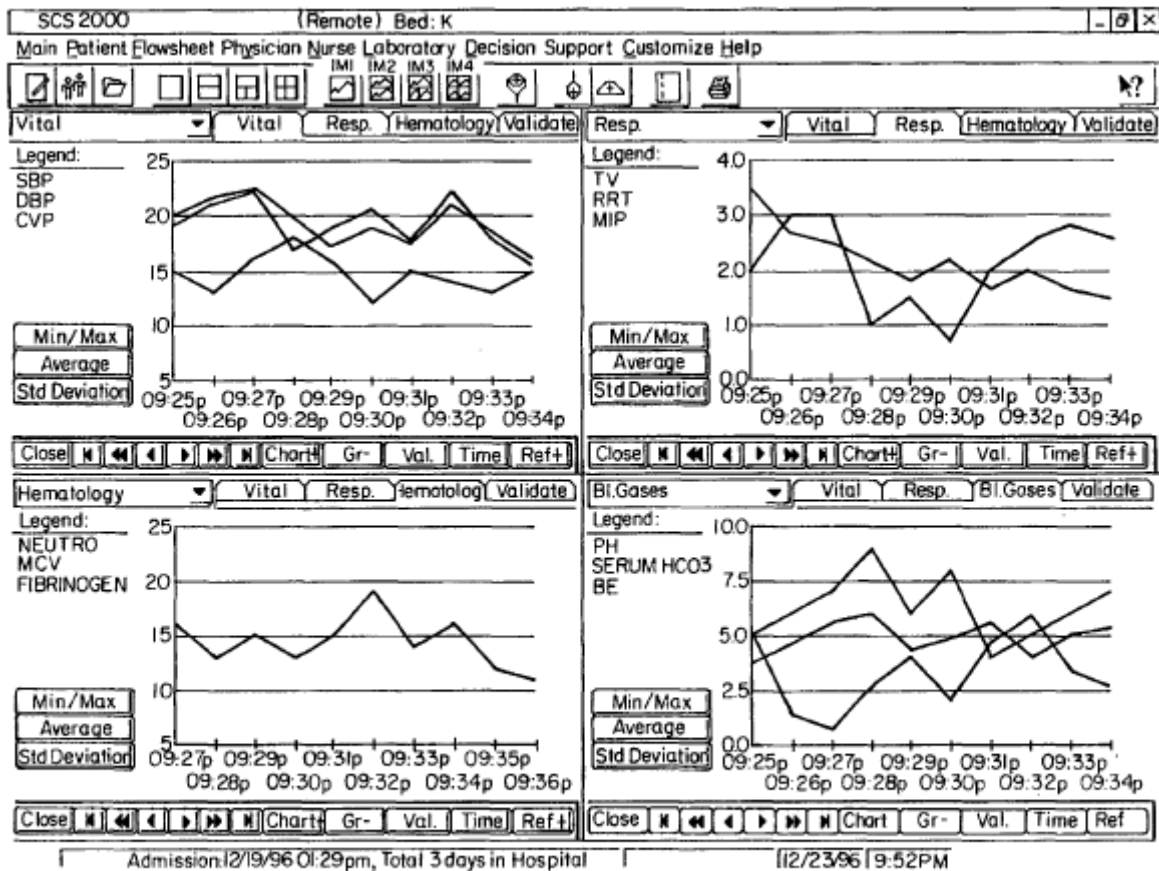


FIG. 2B

315. It would have been obvious to combine the SmartFlow patent teachings with Schoenberg. Schoenberg is analogous art because it is in the same field as the '994 patent and is reasonably related to the problems faced by the '994 patent's alleged inventors. Specifically, like the '994 patent, Schoenberg relates to a "medical information system receives patient data and information from various sources and displays such information in a variety of formats...." Schoenberg, Ex. 1035, Abstract. Furthermore, the SmartFlow patent teachings potentially provides numerous forms of information simultaneously. (e.g. Figure 17). Schoenberg

teaches that "[t]here exists a need for all data and information obtained from and about a patient in a hospital to be immediately and selectively accessible to various members of the medical team in a hospital in accordance with the function performed by those members." Schoenberg, Ex. 1035, col. 2, ll. 36-39.

Schoenberg addresses this by "provid[ing] a medical information system which presents multiple types of patient data simultaneously." Schoenberg, Ex. 1035, col. 2, ll. 63-65.

316. It would further have been obvious to have these display regions correspond to specified measurement processing components for the reasons given in paragraphs 209 to 212.

8. Claim 13

317. Claim 13 reads: "*13. The multipurpose host system of claim 12 wherein the generic graphical interface specification includes a first set of tabs for selecting one of a set of supported display modes, wherein each supported display mode includes a specified set of graphical display components rendered in the mode-specific data display region.*"

318. Regarding claim 13, Schoenberg teaches in Fig. 2B that each display region (graph) has a set of tabs labeled across the top "Vital", "Resp", "Hematology" and "Validate", for selecting the display of output of different sensors. Schoenberg, Ex. 1035, Fig. 2B. It would therefore have been obvious to

include the required tabs in the layout for a GUI element for the reasons given in paragraphs 313 to 315.

9. Claim 20

319. Claim 20 reads: *"20. The multipurpose host system of claim 12 wherein the generic graphical interface specification includes a system message display region."*

320. Schoenberg teaches that its system displays system messages on screen. For example, the system displays messages "to indicate that a particular data set is being updated". Schoenberg, Ex. 1035, col. 7, ll. 53-56. Schoenberg also includes system message display regions in Fig. 2B, for example, the time display region in the lower right-hand corner of the window. It would have been obvious (and indeed, necessary) to include an area for system messages within a layout for the GUI window, so that the Windows system would know where to display such messages. Furthermore, the inclusion of such a display region in a GUI layout would have been the use of known components in a known system having no unpredictable result, and accordingly, obvious.

10. Claim 15

321. Claim 15 reads *"15. The multipurpose host system of claim 12 wherein a pressure display mode supports rendering a fractional flow reserve value."*

322. Claim 15 is thus dependent on claim 12, but has essentially the same content as claim 4. This is because claim 4 requires a display interface where "*a ratio of a first measured pressure and a second measured pressure is rendered.*" This ratio can be a "fractional flow reserve" as specified in claim 15, and as taught by Kern 1997. Claim 15 is thus obvious over Kern 1997 in view of the references applied to claim 12, for the reasons given above in paragraphs 247 to 248.

11. Claims 16 and 17

323. Claim 16 reads: "*16. The multipurpose host system of claim 11 wherein the generic graphical user interface specification includes a touch screen interface for selecting display components.*"

324. Regarding claim 16, Snell teaches a touchscreen to allow display elements to be selected. The abstract of Snell states:

"An apparatus and a method for an improved user interface for communicating with implantable medical devices are described. An integrated digitizer display screen and a digitizer pen serve as the primary input devices to a tablet computer adapted to receive real-time and stored medical data. The pen is used to select programming options by tapping portions of the digitizer based on visual images on the display. Additionally, the pen may be used to manipulate the medical data through the use of gestures, or be used to enter free form annotations concerning the medical data. An additional aspect of the user interlace is the use of a questionnaire card to input information

into the tablet computer by tapping the pen on the marked answers on the questionnaire card."

Snell, Ex. 1036, Abstract.

325. The display elements as selectable according to Snell would need a location on screen. It would therefore have been obvious to include a touchscreen interface within the layout for the GUI elements.

326. Claim 17 reads: "*17. The multipurpose host system of claim 16 wherein a touch screen keyboard is supported by the generic graphical user interface specification.*"

327. Regarding claim 17, Snell teaches the use of a touchscreen keyboard which can be implemented in a medical device user interface:

"The use of explicit buttons arranged on the digitizer display screen 104 (FIG. 1) as in the configuration of a standard typewriter keyboard (QWERTY keyboard) is another known input scheme for text."

Snell, Ex. 1036, col. 4, ll. 44-48.

328. A touch screen keyboard would necessarily (and obviously) have had a layout within the graphical user interface, thus indicating that it is "supported" by the generic user interface specification.

329. It would have been obvious to include a standard input method such as the touchscreen of Snell into the generic graphical user interface specification

because the combination is one of known techniques, combined using known methods, to reach a predictable result. The '994 patent does not provide any teachings concerning how to include a touchscreen in a generic graphical user interface specification, nor does the '994 patent describe any unexpected results.

330. I note that Snell describes the touchscreen keyboard as a less preferred embodiment. However, in the relevant timeframe, touchscreen keyboards were well-known, and providing a touchscreen keyboard did not result in any technical effect. Rather, it was simply a interchangeable feature that the person of ordinary skill knew how to implement if desired.

12. Claim 14

331. Claim 14 reads: "*14. The multipurpose host system of claim 13 wherein a mode-specific display region displays a second set of display-mode specific tabs corresponding to a set of displays associated with a display mode corresponding to a selected one of the first set of tabs, thereby facilitating a hierarchical arrangement of supported graphical displays within the mode-specific display region.*"

332. Regarding claim 14, Snell teaches a display having vertical and horizontal tabs. The physician can choose between display modes using the vertical tabs, and within modes, can choose display options using the horizontal tabs. Snell explains:

"The screen display in FIG. 4 contains vertical tabs 202 and horizontal tabs 204. The physician or medical specialist uses the vertical and horizontal tabs 202 and 204 to move from one screen display to another screen display. Tapping one of the vertical tabs 202 triggers the button on the digitizer display screen 104 underneath the displayed image of the tab. This tapping will cause the tablet computer 100 (FIG. 1) to jump to the first screen display in a series of related screen displays....A highlighted vertical tab 206 and a highlighted horizontal tab 208 remind the physician or medical specialist of the context of the current screen display. In this case, the highlighted vertical tab 206 displays the tab labeled 'ECG Machine' and the highlighted horizontal tab 208 displays the tab labeled 'Parameters.'"

Snell, Ex. 1036, col. 15, ll. 32-54.

333. It would have been obvious to include a hierarchical display mode of Schoenberg together with tabs as taught by Snell. Snell teaches that ease of specific devices is "very important", because the physician may use the device for only a few hours a month, often where potentially distracting interactions with a patient is necessary. Snell, Ex. 1036, col. 3, ll. 26-40. To address this need, Snell teaches "an improved apparatus and improved method for viewing, manipulating and annotating both real-time and stored medical data" Snell, Ex. 1036, col. 5, ll. 50-52), for receiving "several channels of data". Additionally, the tabs of Snell were a common element of GUIs at the time of the alleged invention.

334. The inclusion of the relationship of Snell into the GUI of a Windows-based GUI of the SmartFlow patent teachings is the combination of known elements, using known methods, having no unpredictable result and is thus obvious under *KSR v. Teleflex*. The '994 patent does not teach how to create hierarchical display modes with tabs, nor does the '994 patent describe any unexpected results.

IX. Objective Evidence that the Claims are Obvious

335. Up to and before early 2002, many companies in a variety of medical monitoring applications were manufacturing systems identical to those claimed by the '994 patent. It is my opinion that at least three of these systems anticipated each and every limitation of claim 1 of the '994 patent, and are thus objective evidence ("simultaneous invention") that claim 1 was obvious.

336. I understand that instances of simultaneous invention are not required to qualify as prior art or be exactly simultaneous. Rather, instances of "simultaneous invention" should be reasonably close in time to the filing date of the application leading to the patent under review, such that the simultaneous invention tends to make demonstrate that the relevant industry was already moving in the direction specified by the patent claims. This tends to show that market forces were pushing the industry in the direction of the patent claims, independent of any contribution made by the "inventors" of the patent.

A. The Prucka CardioLab/Mac-Lab system as sold was an instance of simultaneous invention.

337. In this section of my Report, I explain how the Prucka CardioLab/Mac-Lab system anticipated claim 1 of the '994 patent, and thus is an instance of simultaneous (in this case, prior) invention. When I refer to the Prucka CardioLab/Mac-Lab system in the following section, I am referring to the versions of the system that ran on the Windows NT operating system, including versions bearing software version numbers from 5.0 up to and including 5.1C. To that end, I relied on documents provided by Sheri Prucka that related to these versions of the system, in addition to Prucka CardioLab/Mac-Lab system that I analyzed, which were manufactured in the relevant time frame, in addition to other evidence of the operation and functionality of the systems, as disclosed in Ex. 1053.

338. The GE Marquette Medical Systems, Inc. Prucka CardioLab models 2000, 4000, and 7000 and the Prucka Mac-Lab models 2000, 4000, and 7000 anticipated each limitation of claim 1 of the '994 patent.

339. The Prucka CardioLab/Mac-Lab system was a multipurpose host system.

340. The Prucka CardioLab/Mac-Lab system was a personal computer-based medical monitoring system.

341. The MAC-LAB System:

342. The MAC-LAB is a microprocessor based data acquisition system used during cardiac catheterization procedures. The MAC-LAB system, via various models of the GE Marquette TRAM module (K921669) and amplifier module, acquires patient data which may include surface ECG, invasive and non-invasive blood pressure, blood oxygen saturation via pulse oximetry, respiration and temperature. The MAC-LAB System joins together the TRAM module and amplifier module with computer processors, software, high resolution display monitors, power supply, laser printer, keyboard and mouse. Digital data is transmitted, via cable, from the TRAM module and/or amplifier module to the computer for processing. Major functions of the software include data acquisition and display, data storage, reporting of data, and transmission of data to other information systems via LAN.

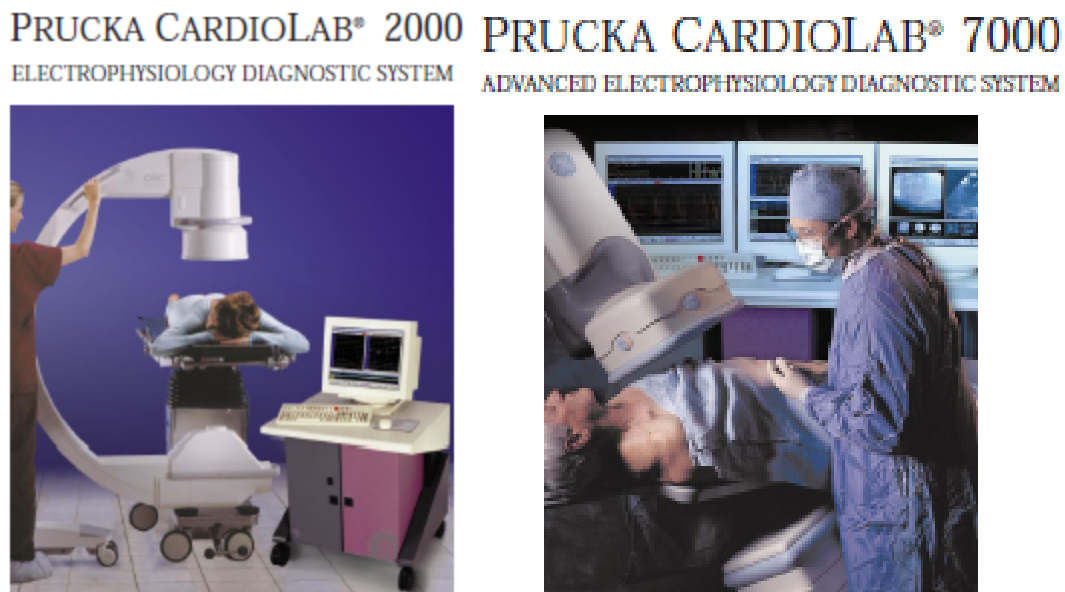
343. The CardioLab System

344. The CardioLab EP System is a microprocessor based data acquisition system used during electrophysiology procedures to acquire ECG, intracardiac signals, and pressure signals via amplifier module. Digital data is also acquired from other devices such as RF generators, fluoro video systems and the GE Marquette TRAM module. The ECG, intracardiac and pressure data are acquired by an amplifier that is connected to the patient by third-party devices such as ECG leadwires and catheters. The amplifier filters, amplifies, digitizes and transmits the

data to the computer via fiber optic cable. The computer stores the data on optical disks, displays the data on the video monitors, allows the user to perform basic signal measurements, and prints out waveforms on a laser printer or continuous paper recorder. Major functions of the software include data acquisition and display.

Prucka Decl., Ex. 1053, at SJM3070813.

345. I understand that the term "microprocessor" describes the central processing unit of a personal computer, such as the ones shown in the Prucka CardioLab marketing materials:



Prucka Decl., Ex. 1053, ¶ 5 & Exs. 4 & 6 thereof (corresponding to Exs. 1088 and 1090).

346. Additionally, the Prucka CardioLab/Mac-Lab system was "multipurpose" in that it was capable of word processing, spreadsheets, creating

power point presentations, reading .pdf documents, and other general-purpose uses. For example, in my research for the prior litigation and in preparation for this Report, I analyzed the software in a Prucka CardioLab/Mac-Lab system originally manufactured in March 2001 and a Prucka CardioLab/Mac-Lab system originally manufactured in May 2001.⁵ The systems came with copies of the Microsoft Office® suite of applications, including Word®, Excel®, PowerPoint®, as well as Adobe® Acrobat® Reader.

347. The Prucka CardioLab/Mac-Lab system was for invasive cardiovascular diagnostic measurement acquisition and display.

348. The purpose of the Prucka CardioLab was, *inter alia*, "to acquire . . . intracardiac signals, and pressure signals via amplifier module. . . . Major functions of the software included data acquisition and display." Prucka Decl., Ex. 1053, ¶ 6 & Ex. 2 thereof (corresponding to Ex. 1086). I understand "intracardiac signals" to refer to invasive cardiovascular signals. *See, e.g.*, Prucka Decl., Ex. 1053, ¶ 6. Prucka CardioLab also had as many as 224 catheter inputs. Prucka Decl., Ex. 1053, Ex. 3 thereof (corresponding to Ex. 1087). I understand that

⁵ Pursuant to the CardioLab Service Manual, the Prucka CardioLab computers I analyzed were manufactured in 2000 and 2001. *See* Prucka Decl. Ex. 8 (corresponding to Ex. 1092) at SJM03070844.

catheters are invasive devices used for cardiovascular measurements. *See, e.g.,* Prucka Decl., Ex. 1053, ¶ 6.

349. Additionally, the Prucka Mac-Lab "acquire[d] patient data which may include . . . invasive and non-invasive blood pressure . . ." and "[m]ajor functions of the software include[d] data acquisition and display." *Id.* & Ex. 2 (corresponding to Ex. 1086) at SJM03070813.

350. The Prucka CardioLab/Prucka Mac-Lab system incorporated a component based arrangement.

351. The Prucka CardioLab/Mac-Lab system comprised a variety of hardware components. For example, the Prucka CardioLab required a computer, an external amplifier, a monitor, a printer, and a workstation, as depicted below:



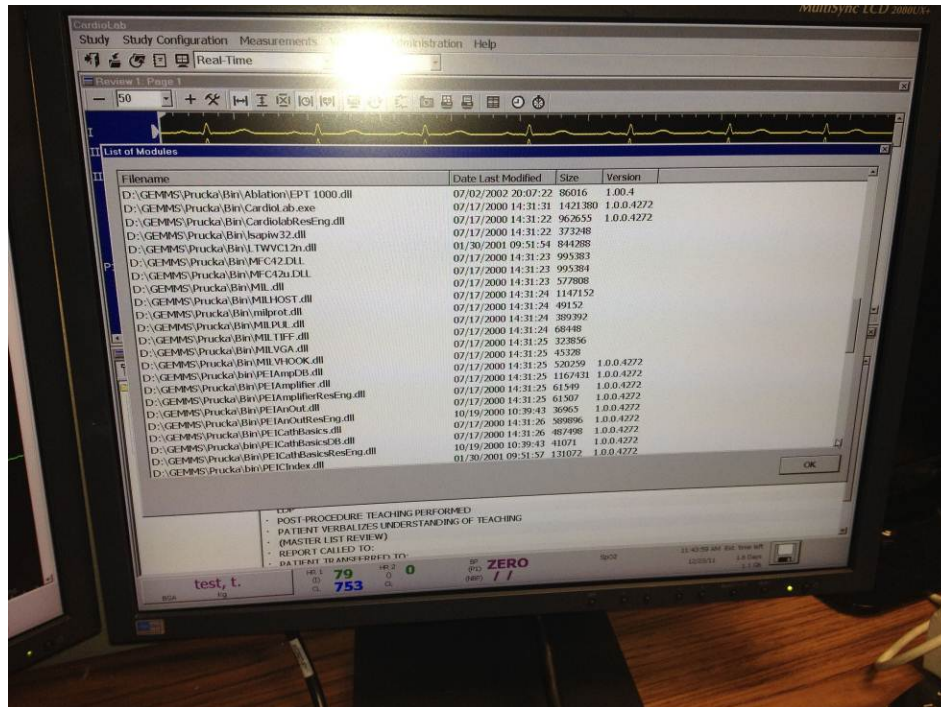
Prucka Decl., Ex. 1053, Ex. 3 thereof (corresponding to Ex. 1087).

352. The Prucka CardioLab/Mac-Lab system also had a component based software arrangement. Prucka Decl., Ex. 1053, ¶¶ 14-15. To verify this, I analyzed Prucka CardioLab/Mac-Lab computers originally manufactured in 2001,⁶ which were running software version 5.1.⁷ I also saw a Prucka CardioLab/Mac-Lab system hooked up to a simulator and I interviewed Phil Lonbeck, a former employee of Marquette Medical, and a person who has familiarity with the Prucka CardioLab/Mac-Lab system. A video of that session is provided as Ex. 1145. I determined that the user application in the Prucka CardioLab/Mac-Lab system was programmed using Microsoft Visual C++, as indicated in part by the following read-out from the Hex-Rays decompiler program that I used for analysis:

/* This file has been generated by the Hex-Rays decompiler.

⁶ See footnote 5, *supra*.

⁷ The Prucka CardioLab/Mac-Lab I analyzed has some files in it that were modified in June 2002, though many of the files were created in 2000. I understand that in or about June 2002, GE Marquette released version 5.1D, a minor bug-fix upgrade for the Prucka CardioLab/Mac-Lab system. Declaration of Sachin Vadodaria ("Vadodaria Lit. Decl") ¶ 6 (Ex. 1133); Prucka Decl. ¶ 23 (Ex. 1053). I understand that the functionality and architecture of the software and hardware in version 5.1D was unchanged from the original software version 5.1.



354. Attached are copies of the directories containing the applications and the drivers from the Prucka CardioLab/Mac-Lab system. The contents and creation date of the directory containing the drivers is provided as Ex. 1060 and the contents and creation date of the directory containing the applications is provided as Ex. 1061.

355. It should be noted that, in the Prucka CardioLab/Mac-Lab system, each file is distinctly named and generally corresponds to one logical software component or a family of related logical software components. As is typically the case for non-malicious software code, the individual components are named

descriptively, albeit typically with short names.⁸ The names of the functions within the application are also indicative of the use of object oriented programming, and further indicate that the main software application was written using C++, indicating an extensible, modular, component-based software architecture.

356. It should also be noted that in the Prucka CardioLab/Mac-Lab system I viewed and analyzed, the "Date Last Modified" for most of the software components is in July 2000, however not all components were modified on the same date.

357. An exemplary portion of the Prucka CardioLab/Mac-Lab software code that demonstrates the mechanism by which the system loads new software components follows:

```
//----- (1000291F) -----  
  
signed int __thiscall sub_1000291F(void *this, LPCWSTR  
hLibModule, LPCWSTR lpName, LPCWSTR lpType, int a5)  
  
{  
  
    HMODULE v5; // eax@1
```

⁸ This convention is held over from the MS-DOS and older Windows systems that required 8.3 compatible file names.

HMODULE v6; // edi@1

signed int v7; // esi@3

HRSRC v8; // eax@5

HRSRC v9; // esi@5

HGLOBAL v10; // ebx@8

DWORD v11; // eax@11

DWORD v12; // esi@11

const CHAR *v13; // edi@11

int v14; // eax@12

void *v15; // esp@12

const WCHAR *v16; // ebx@14

int v17; // eax@15

int v18; // esi@15

int v19; // eax@15

void *v20; // esp@15

int v22; // [sp+0h] [bp-14h]@12

int v23; // [sp+Ch] [bp-8h]@1

void *v24; // [sp+10h] [bp-4h]@1

```
HMODULE hLibModulea; // [sp+1Ch] [bp+8h]@1
```

```
v23 = 0;
```

```
v24 = this;
```

```
v5 = LoadLibraryExW(hLibModule, 0, 2u);
```

```
v6 = v5;
```

```
hLibModulea = v5;
```

```
if (v5 )
```

```
{
```

```
    v8 = FindResourceW(v5, lpName, lpType);
```

```
    v9 = v8;
```

```
    if (v8 && (v10 = LoadResource(v6, v8)) != 0 )
```

```
    {
```

```
        v11 = SizeofResource(v6, v9);
```

```
        v12 = v11;
```

```
        v13 = (const CHAR *)v10;
```

```
        if (*((__BYTE *)v10 + v11) )
```

```
        {
```



```

v14 = v11 + 4;

LOBYTE(v14) = (v12 + 4) & 0xFC;

v15 = alloca(v14);

v13 = (const CHAR *)&v22;

memcpy(&v22, v10, v12 + 1);

*((_BYTE *)&v22 + v12) = 0;

}

if (v13 )

{

v17 = strlenA(v13);

v18 = v17 + 1;

v19 = 2 * (v17 + 1) + 3;

LOBYTE(v19) = v19 & 0xFC;

v20 = alloca(v19);

v16 = (const WCHAR *)&v22;

LOWORD(v22) = 0;

MultiByteToWideChar(0, 0, v13, -1, (LPWSTR)&v22, v18);

}

```

```

else

{

    v16 = 0;

}

v7 = sub_10002A3D(&v23, v16, a5);

}

else

{

    if (GetLastError() )

        v7 = (unsigned __int16)GetLastError() | 0x80070000;

    else

        v7 = 0;

}

FreeLibrary(hLibModulea);

}

else

{

    if (GetLastError() )

```

```

        v7 = (unsigned __int16)GetLastError() | 0x80070000;

    else

        v7 = 0;

    }

    return v7;

}

```

358. In the above portion of software code, the highlighted function "LoadLibraryExW" is the Windows function used to dynamically load a software component. Microsoft, Windows Dev. Center, *LoadLibraryEx function* <http://msdn.microsoft.com/en-us/library/windows/desktop/ms684179%28v=vs.85%29.aspx> (last accessed April 17, 2013) (Ex. 1062). The use of the LoadLibraryExW function indicates a software application that supports dynamic loading of components based upon configuration information.

359. I have confirmed that these software components are user mode components by inspecting the portable executable headers and the import/export lists for these components in the software code for the Prucka CardioLab/Mac-Lab system that I viewed and analyzed. My inspection showed that all of these components are dependent on standard user mode operating system components,

which are frequently referred to as "libraries." Kernel mode components in Windows NT cannot rely on user mode operating system libraries. Therefore, I have concluded that the components shown in the prior table are all user mode components, and that the Prucka CardioLab/Mac-Lab system had a "component-based arrangement."

360. The Prucka CardioLab/Mac-Lab system comprised a component based arrangement. The software components in the Prucka CardioLab/Mac-Lab system were extensible and modular. For example, the software in the Prucka CardioLab/Mac-Lab system was primarily programmed using C++ because of the possibility of future growth provided by a modular, object oriented programming language. Prucka Decl., Ex. 1053, ¶ 14. The main application software was designed to have this future growth capability by using the modularity provided for in C++. *Id.* When Prucka Engineering and GE Marquette provided software upgrades, it provided the customer with a disk containing new software that replaced some or all of the software modules in the Prucka CardioLab and Prucka Mac-Lab main applications. *Id.* For example, when Prucka Engineering provided a fluoroscopy imaging feature for the Prucka CardioLab, it provided additional software modules that enhanced the main software application with that capability. *Id.* The extensibility and modularity of the main software application also allowed GE Marquette to readily incorporate the features of the TRAM from Marquette's

Mac-Lab system into the joint Prucka CardioLab/Mac-Lab system after the Prucka Engineering acquisition. *Id.*

361. I confirmed this extensibility and modularity with the Prucka CardioLab/Mac-Lab system I acquired. When I first acquired the system, it only contained the Prucka CardioLab version 5.1 software and other base software provided by the system. I had the system upgraded to include the Prucka Mac-Lab software. After upgrading the system, I analyzed the changes to the directories of the application software. I noticed that the added functionality for the Prucka Mac-Lab was enabled by the addition of new components that worked with the pre-existing Prucka CardioLab software. The software upgrade that added the Prucka Mac-Lab functionality to the Prucka CardioLab/Mac-Lab system I acquired did not require replacement of each of the existing components. Therefore, the addition of the Prucka Mac-Lab software (through the above-mentioned upgrade) indicated that the Prucka CardioLab computer was designed to be extensible and modular.

362. The Prucka CardioLab/Mac-Lab system included an external input signal bus interface for receiving data arising from cardiovascular diagnostic measurement sensors.

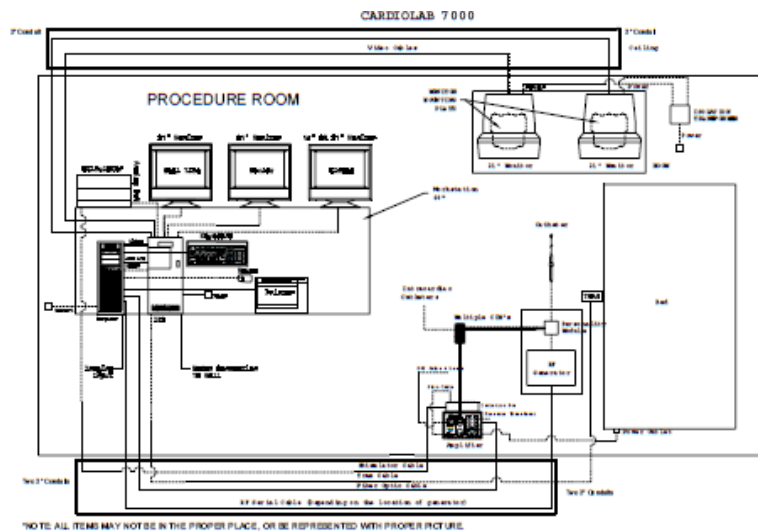
363. The Prucka CardioLab/Mac-Lab system received data from cardiovascular diagnostic measurement sensors. The Prucka CardioLab software required an external amplifier called the CardioLab II or CardioLab II Plus for data

acquisition. Prucka Decl., Ex. 1053, ¶ 7 & Ex. 8 (corresponding to Ex. 1092) at SJM03070854. The CardioLab II/II Plus amplifier had inputs for ECG, catheter/intercardiac, and blood pressure. Prucka Decl., Ex. 1053, Ex. 3 thereof (corresponding to Ex. 1087). The external amplifier filtered, amplified, digitized, further processed, and transmitted the data to the computer for use in the CardioLab application. Prucka Decl., Ex. 1053, ¶ 7 & Ex. 2 (corresponding to Ex. 1086) at SJM03070813.

364. The Prucka Mac-Lab software required an external amplifier called the TRAM for data acquisition. *Id.* The TRAM had inputs for surface ECG, invasive and non-invasive blood pressure, blood oxygen saturation via pulse oximetry, respiration and temperature. *Id.* The TRAM filtered, amplified, digitized, further processed, and transmitted the data to the computer for use in the Mac-Lab application. Prucka Decl., Ex. 1053, ¶ 7.

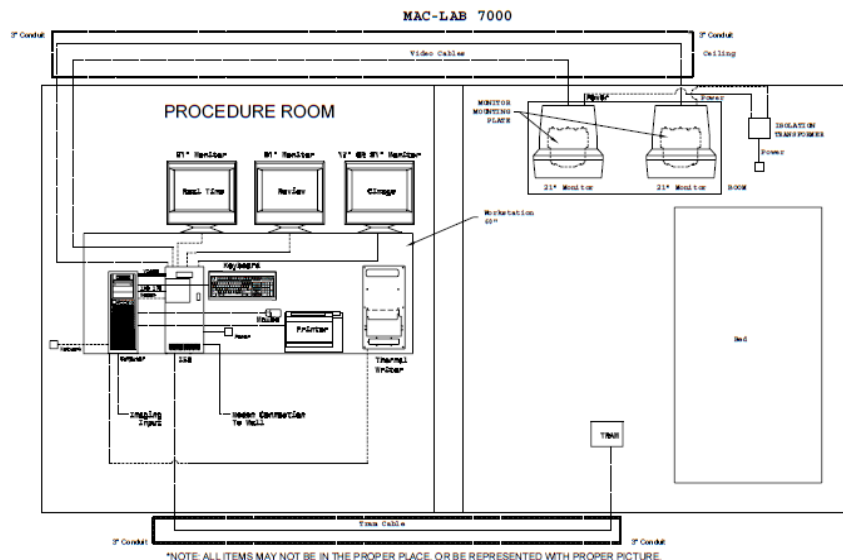
365. The Prucka CardioLab/Mac-Lab system also had an external signal bus interface for receiving this data. For example, after processing, signals from the cardiovascular diagnostic sensors were transmitted from the CardioLab II/II Plus to a COMMS ISA Expansion Card, which was mounted in the central processing unit (the "CPU"). Prucka Decl., Ex. 1053, ¶ 10 & Ex. 8 (corresponding to Ex. 1092) at SJM03070854. The CardioLab II/II Plus amplifier sent the signals to the COMMS ISA Expansion Card by means of a fiber optic cable. *Id.* A wiring

diagram from the Prucka CardioLab/Mac-Lab system configured to use the CardioLab software is as follows:



Prucka Decl., Ex. 1053, Ex. 8 (corresponding to Ex. 1092) at SJM03070867.

366. Signals from the TRAM were transmitted into the CPU using an RS422 PCI Expansion Card, which was mounted in the PCI card bus slot in the CPU. Prucka Decl., Ex. 1053, ¶ 9 & Ex. 8 (corresponding to Ex. 1092) at SJM03070853. The TRAM amplifier sent the signals to the RS422 PCI Expansion Card by means of an RS422 cable. *Id.* A wiring diagram from the Prucka CardioLab/Mac-Lab system configured to use Mac-Lab software is as follows:



Prucka Decl., Ex. 1053, Ex. 8 (corresponding to Ex. 1092) at SJM03070870.

367. Accordingly, the Prucka CardioLab/Mac-Lab system practiced and anticipated this limitation of claim 1 – i.e., it included an external input signal bus interface for receiving data arising from cardiovascular diagnostic measurement sensors.

368. The Prucka CardioLab/Mac-Lab system included a plurality of measurement processing components, that operate at a user mode level in the multipurpose host system, for facilitating receiving data of particular sensor types and rendering diagnostic measurement parameter values according to the received data.

369. As noted, the Prucka CardioLab/Mac-Lab system was written in C++ using the Microsoft Foundation Class, an object-oriented programming language

and component-based software engineering framework frequently used for writing component-based software applications. Additionally, as I explained in Section II.E of this Report, a conventional way of using the modularity allowed by C++ and MFC programming is to utilize separate components for processing as opposed to display. In accordance with its use of MFC, the Prucka CardioLab/Mac-Lab system does its data processing in certain software components, which are distinct from the software components used for display. Examples of the measurement processing components include PEIEventVeinousPres, PEIEventArterialPres, PEICathRTPresMeas, and PEICathGradientMeasData, each of which is a discrete class in C++.

370. These measurement processing components facilitated receiving data of particular sensor types. An exemplary portion of the analyzed software code from the Prucka CardioLab/Mac-Lab system that demonstrates the removal of data for the plurality of measurement processing components is set forth below:

```
int __thiscall PEINtAmpSignal::ReadDmaData(void *this, int a2, int
a3, int a4, unsigned int a5, double a6, int a7, int a8, double a9, int a10,
int a11)

{

    int v11; // eax@8

    int v13; // eax@11
```

```
int v14; // eax@13

int v15; // ST40_4@16

unsigned int v16; // ST70_4@20

int v17; // ST5C_4@28

int v18; // edx@28

int v19; // ST5C_4@32

signed int v20; // [sp+0h] [bp-9Ch]@21

int v21; // [sp+4h] [bp-98h]@1

signed int i; // [sp+14h] [bp-88h]@36

int j; // [sp+18h] [bp-84h]@29

int k; // [sp+18h] [bp-84h]@32

int v25; // [sp+1Ch] [bp-80h]@28

int v26; // [sp+1Ch] [bp-80h]@32

void *v27; // [sp+20h] [bp-7Ch]@28

__int16 v28; // [sp+24h] [bp-78h]@28

int v29; // [sp+28h] [bp-74h]@28

int v30; // [sp+3Ch] [bp-60h]@20

unsigned int v31; // [sp+44h] [bp-58h]@20
```

```

int v32; // [sp+48h] [bp-54h]@20

int v33; // [sp+4Ch] [bp-50h]@20

int v34; // [sp+50h] [bp-4Ch]@20

unsigned int v35; // [sp+54h] [bp-48h]@20

int v36; // [sp+58h] [bp-44h]@20

int v37; // [sp+5Ch] [bp-40h]@20

int v38; // [sp+60h] [bp-3Ch]@20

int v39; // [sp+64h] [bp-38h]@20

char v40; // [sp+68h] [bp-34h]@11

int v41; // [sp+7Ch] [bp-20h]@3

int v42; // [sp+80h] [bp-1Ch]@2

int v43; // [sp+84h] [bp-18h]@2

int v44; // [sp+88h] [bp-14h]@2

int v45; // [sp+8Ch] [bp-10h]@2

int v46; // [sp+98h] [bp-4h]@11


v21 = (int)this;

if ((* (int (**)(void))(*(_DWORD *)this + 52))() )

```

```

{

v42 = 0;

v43 = PEIAmpSignal::mg_pAmp;

v45 = a3;

v44 = 274;

if (PEIAmpSignal::mg_pAnInAmp )

{

v41 =
PEIBaseAmp::GetSignalByIndex(PEIAmpSignal::mg_pAnInAmp,
0);

if (v41 )

v44 = (*(int (__thiscall **)(int, _DWORD))(*(_DWORD *)v41
+ 204))(v41, 0);

}

if (a3 < v44 )

{

v43 = PEIAmpSignal::mg_pAmp;

(*(void (__thiscall **)(int, char *, signed int))(*(_DWORD
*)PEIAmpSignal::mg_pAmp + 96))((

```

```

    PEIAmpSignal::mg_pAmp,

    &v40,

    1);

v46 = 0;

v13 = PEIGenData::operator__(&v40, a3);

v45 = PEIGenData::operator unsigned_long(v13);

v42 = PEIBaseAmp::GetSignalByIndex(v43, a3);

if (v42 )

{

    if (!(* (int (__thiscall **)(int, _DWORD)))(* (_DWORD *)v42 +
192))(v42, 0) )

    {

        v14 = PEISigViewInfo::GetCurPage(0);

        if (!(* (int (__thiscall **)(int, int, _DWORD)))(* (_DWORD
*)v42 + 56))(v42, v14, 0) )

        {

            if (a3 < *(_WORD *)(v21 + 41164) || a3 >= *(_WORD *)(v21
+ 41164) + 5 )

            {

```

```

        v15 = *(_DWORD *)a2;

        v46 = -1;

        PEIGenData::_PEIGenData(&v40);

        return v15;

    }

}

}

}

}

v46 = -1;

PEIGenData::_PEIGenData(&v40);

}

else

{

    v43 = PEIAmpSignal::mg_pAnInAmp;

    v45 = a3 - v44;

    v42 =
PEIBaseAmp::GetSignalByIndex(PEIAmpSignal::mg_pAnInAmp, a3
- v44);

    if (v42 )

```

```

{

    if (!(* (int (__thiscall **)(int, _DWORD))(*(_DWORD *)v42 +
192))(v42, 0) )

    {

        v11 = PEISigViewInfo::GetCurPage(0);

        if (!(* (int (__thiscall **)(int, int, _DWORD))(*(_DWORD
*)v42 + 56))(v42, v11, 0) )

            return *(_DWORD *)a2;

    }

}

}

}

if (*(_DWORD *)(v21 + 92) >= a5 )

    return *(_DWORD *)a2;

v39 = PEIAmp::GetDmaChannelCount(v43);

v30 =
PEIAmp::GetNumOfSamplesPerDmaTransfer(PEIAmpSignal::mg_p
Amp);

v38 = PEIAmp::GetMaxDmaPage(PEIAmpSignal::mg_pAmp);

v33 = v39 * v30 * v38 - 1;

```

```

v32 = *(_DWORD*)(v21 + 41152);

v34 = 0;

(*(void (__thiscall **)(int, int *))*(_DWORD
*)PEIAmpSignal::mg_pAmp + 136))(PEIAmpSignal::mg_pAmp,
&v34);

v35 = *(_DWORD*)(v21 + 88) + (a4 - *(_DWORD*)(v21 + 92))
/ (unsigned int)v30;

v31 = (a4 - *(_DWORD*)(v21 + 92)) % (unsigned int)v30;

v16 = *(_DWORD*)(v21 + 88) + (a5 - *(_DWORD*)(v21 + 92))
/ v30;

v36 = v34 - v35;

v37 = v34 - v16;

if (((a5 - a4) & 0x80000000u) != 0 )

    v20 = 0;

else

    v20 = a5 - a4;

if (v36 < 3 * v38 / 4 && v36 >= 0 && v37 < 3 * v38 / 4 && v37
>= 0 && v20 > 0 )

{

```



```
v17 = (*(int (__thiscall **)(int, unsigned int))(*(_DWORD *)v43  
+ 144))(v43, v35);
```

```
v27 = operator new(2 * v20);
```

```
HIWORD(v18) = HIWORD(v45);
```

```
v25 = v17 + 2 * v45;
```

```
LOWORD(v18) = a3;
```

```
v28 = (*(int (__thiscall **)(int, int))(*(_DWORD *)v21 +  
20))(v21, v18);
```

```
v29 = v30 * (v38 - (signed int)v35 % v38) - v31;
```

```
if (v20 < v29 )
```

```
{
```

```
for (i = 0; i < v20; ++i )
```

```
    *((_WORD *)v27 + i) = v28 & *((_WORD *)v25 + 2 * (v33 &  
v39 * (v31 + i)));
```

```
}
```

```
else
```

```
{
```

```
for (j = 0; j < v29; ++j )
```

```
    *((_WORD *)v27 + j) = v28 & *(_WORD *)(v25 + 2 * (v33 &
v39 * (v31 + j)));
```

```
    v19 = (*(int (__thiscall **)(int, _DWORD))(*(_DWORD *)v43
+ 144))(v43, 0);
```

```
    v26 = v19 + 2 * v45;
```

```
    for (k = v29; k < v20; ++k )
```

```
        *((_WORD *)v27 + k) = v28 & *(_WORD *)(v26 + 2 * (v33 &
v39 * (k - v29)));
```

```
    PEIAmp::ReleaseDmaBuffer(v43);
```

```
}
```

```
    PEIAmp::ReleaseDmaBuffer(v43);
```

```
    PEINtAmpSignal::CopyBlock(v21, (int)v27, v20, 0, a7, a2, a8, a6,
v32, a9, a10);
```

```
    operator delete(v27);
```

```
}
```

```
}
```

```
    return *(_DWORD *)a2;
```

```
}
```

The key is:

```
if (PEIAmpSignal::mg_pAnInAmp )  
  
{  
  
    v41 =  
    PEIBaseAmp::GetSignalByIndex(PEIAmpSignal::mg_pAnInAmp,  
0);  
  
    if (v41 )  
  
        v44 = (*(int (__thiscall **)(int, _DWORD))(*(_DWORD *)v41  
+ 204))(v41, 0);  
  
}
```

This component from the Prucka CardioLab/Mac-Lab system software receives data of particular sensor types. For example, this portion of the above quoted software code:

```
if (PEIAmpSignal::mg_pAnInAmp )  
  
{  
  
    v41 =  
    PEIBaseAmp::GetSignalByIndex(PEIAmpSignal::mg_pAnInAmp,  
0);
```

```

        if (v41 )

            v44 = (*(int (__thiscall **)(int, _DWORD))(*(_DWORD *)v41
+ 204))(v41, 0);

        }

```

371. The code has the highlighted function "GetSignalByIndex." A similar routine is found many times in the larger portion of the code cited above. The software code accompanying this repeated function shows the removal of different kinds of data from different types of sensors based upon an index value that is specific to the type of sensor data being received. This data was then distributed to different components for processing based upon the type of data removed. Accordingly, the plurality of measurement processing components to which this data was fed facilitated the receipt of the data from particular sensor types.

372. The multiple processing components also facilitated rendering diagnostic parameter values based on the received sensor data, because the processing components fed the data to display components within the same application. For example, the Prucka CardioLab/Mac-Lab system software had, among others, the following dynamically linked libraries, which contained software components used for rendering and displaying data in the graphical user interface: PEICathBasics.dll, PEIGrid.dll, PEICoreDisp.dll, PEIImageViewer.dll, and PEIUI.dll. The file PEICoreDisp.dll referred to the "core display" designed by

"Prucka Engineering, Inc." ("PEI"). I was able to establish this by looking at the import list of operating system functions that it invokes. I found, for example, that in the import list, there is a function called CWnd::Create, which is invoked to create a window. The creation of a window is fundamental to a user interface in Windows NT, and therefore indicates that this "core display" component is the main display component, and that it provided the basic framework for the application display. Each individual software object then added a display element such as a dialog box, a menu item from the list of drop-down menus across the top of the window, an interactive button, or a subwindow.

373. The data used by these display components in the Prucka CardioLab/Mac-Lab system software was provided by the measurement processing components described above. I have identified display components by their implementation of the OnDraw and OnPaint methods; these are invoked by the Microsoft Foundation Class ("MFC") whenever the MFC framework has been advised that a window refresh is required.

374. Examples of the OnDraw method implementation are found in PEICoreDisp.dll (via the decompiled file PEICoreDisp.c, which is provided as Ex. 1063).

//----- (419150FB) -----

```

char __thiscall PEIRTSigViewHandler::HandleOnDraw(int this, int
a2)

{

    int v2; // eax@3

    int v4; // [sp+0h] [bp-14h]@1

    char v5; // [sp+4h] [bp-10h]@3


    v4 = this;

    if (PEISigViewHandler::HandleOnDraw(this, a2) )

    {

        *(_BYTE *)(v4 + 428) = 1;

        PEISigViewHandler::ResizeAllSVObjects((void *)v4);

    }

    PEIRTSigViewHandler::CreateTimeGrids((void *)v4, 0);

    v2 = sub_41907570((int)&v5, 0, 0, 0, 0);

    PEISigViewHandler::RedrawAllSVObjects(

        v4,

        *(_DWORD *)v2,

```

```

    *(_DWORD*)(v2 + 4),
    *(_DWORD*)(v2 + 8),
    *(_DWORD*)(v2 + 12),
    0);

return 1;

}

```

Id. at l. 17786:

```

//----- (419200B8) -----
char __thiscall PEISigViewHandler::HandleOnDraw(int this, int a2)
{
    int v2; // eax@3

    char result; // al@4

    void *v4; // [sp+0h] [bp-4h]@1

    v4 = (void *)this;

    if (*(_BYTE*)(this + 376) )

```

```

    *(_BYTE *)(this + 376) = 0;

LOBYTE(v2) = PEISigViewHandler::IsViewResized((void *)this);

if (v2 )

{

    (*(void (__thiscall **)(void *, _DWORD))(*(_DWORD *)v4 +
72))(v4, 0);

    (*(void (__thiscall **)(void *, _DWORD,
_DWORD))(*(_DWORD *)v4 + 172))(

        v4,

        *((_DWORD *)v4 + 20),

        *((_DWORD *)v4 + 2));

    PEISigViewHandler::ResizeAllSVObjects(v4);

    result = 1;

}

else

{

    result = 0;

}

return result;

```



```
}
```

Id. at l. 24731.

```
//----- (41917AB6) -----
```

```
char __thiscall PEIRVSigViewHandler::HandleOnDraw(int this, int  
a2)
```

```
{
```

```
int v2; // eax@3
```

```
int v4; // [sp+0h] [bp-18h]@1
```

```
char v5; // [sp+4h] [bp-14h]@3
```

```
int v6; // [sp+14h] [bp-4h]@2
```

```
v4 = this;
```

```
if (PEISigViewHandler::HandleOnDraw(this, a2) )
```

```
{
```

```
v6 = PEIRvAmp::GetAmpTotalDataNumPerChan(*(_DWORD  
*)(v4 + 404));
```

```
*(_BYTE *)(v4 + 453) = v6 <= 0;
```

```

    (*(void (__thiscall **)(int, signed int, _DWORD))(*(_DWORD
*)v4 + 228))(v4, 2, *(_BYTE *)(v4 + 454) == 0);

    *(_BYTE *)(v4 + 454) = 0;

}

v2 = sub_41907570((int)&v5, 0, 0, 0, 0);

PEISigViewHandler::RedrawAllSVObjects(

    v4,

    *(_DWORD *)v2,

    *(_DWORD *)(v2 + 4),

    *(_DWORD *)(v2 + 8),

    *(_DWORD *)(v2 + 12),

    0);

return 1;

}

```

Id. 1. 19420.

375. I can determine that each of these components is a display component because non-display components would have no need for an OnDraw or OnPaint

method. Each of these three OnDraw handlers are for separate display class components.

376. An OnPaint method is invoked in the base CardioLab.exe file (the decompiled version of which is provided as cardiolab.c in Ex. 1063:

```
int __thiscall sub_47E628(void *this)

{

    int v1; // eax@6

    int v2; // ST08_4@12

    int v3; // eax@12

    int v5; // [sp+0h] [bp-9Ch]@1

    char v6; // [sp+4h] [bp-98h]@12

    char v7; // [sp+14h] [bp-88h]@4

    RECT rc2; // [sp+68h] [bp-34h]@4

    struct tagRECT Rect; // [sp+78h] [bp-24h]@4

    int v10; // [sp+88h] [bp-14h]@1

    int v11; // [sp+8Ch] [bp-10h]@1

    int v12; // [sp+98h] [bp-4h]@4
```

```

v5 = (int)this;

v10 = *(_DWORD *)(sub_413130() + 32);

v11 = CWnd::GetTopLevelFrame(v5);

if (*(_BYTE *)(v5 + 1392) && v10 != v11 )

    sub_413C90(v11);

CPaintDC::CPaintDC(&v7, v5);

v12 = 0;

_LocaleUpdate::GetLocaleT(&rc2);

CDC::GetClipBox(&v7, &rc2);

sub_47F660((int)&v7, (LPPOINT)&rc2);

_LocaleUpdate::GetLocaleT(&Rect);

sub_413F30(v5, &Rect);

if (*(_BYTE *)(v5 + 1385) )

{

    if (ATL::CComVariant::Copy(&rc2) || (LOBYTE(v1) =
sub_47F5C0((int)&rc2), v1) )

        sub_47EFE7((void *)v5, *(HDC *)(v5 + 272), 1);

    else

```

```

    sub_47EFE7((void *)v5, *(HDC *)(v5 + 272), 0);

}

else

{

    *(_BYTE *)(v5 + 1385) = 1;

}

if (*(_BYTE *)(v5 + 1384) )

{

    wglMakeCurrent(*(HDC *)(v5 + 276), *(HGLRC *)(v5 + 264));

    _LocaleUpdate::GetLocaleT(&v6);

    sub_413F30(v5, (LPRECT)&v6);

    v2 = sub_414130((int)&v6);

    v3 = sub_414110((int)&v6);

    sub_47EA65((void *)v5, *(HDC *)(v5 + 272), v3, v2);

}

else

{

    wglMakeCurrent(*(HDC *)(v5 + 272), *(HGLRC *)(v5 + 268));

```

```

scene::render(v5 + 72);

SwapBuffers(*(HDC *)(v5 + 272));

}

if (*(_BYTE *)(v5 + 1392) && v10 != v11 )

    sub_413C90(v10);

CView::OnPaint(v5);

v12 = -1;

return CPaintDC::_CPaintDC(&v7);

}

```

Id. 1. 73474.

377. This decompiled code indicates that this display component is a display object (because it is invoking its own OnPaint method and, as stated, only display components would have an OnPaint method).

378. As I discussed in paragraph 38, above, a "view" class is typically derived from a "base" class. The base class contains the data, and the view class displays the data it receives from the base class. This typical structure is found in the Prucka CardioLab/Mac-Lab system software. For example, the OnPaint method above invokes the method labeled "sub_47EA65". This method in turn

uses data in the base class to compute the information that is to be displayed. See sub_47EA65 at line 73,643:

```
//----- (0047EA65) -----  
  
int __thiscall sub_47EA65(void *this, HDC hdc, signed int a3, signed  
int a4)  
  
{  
  
    double v4; // kr00_8@6  
  
    double v5; // kr08_8@7  
  
    unsigned __int64 v7; // [sp+8h] [bp-30h]@6  
  
    void *v8; // [sp+18h] [bp-20h]@1  
  
    double v9; // [sp+1Ch] [bp-1Ch]@5  
  
    double v10; // [sp+24h] [bp-14h]@5  
  
    double v11; // [sp+2Ch] [bp-Ch]@8  
  
  
    v8 = this;  
  
    scene::render((char *)this + 72);  
  
    if ((((_DWORD *)v8 + 74) <= a3 || ((_DWORD *)v8 + 75) <= a4)
```

```

    && (*((DWORD *)v8 + 74) >= a3 || (*((DWORD *)v8 + 75) >=
a4) )

{

    if (*((DWORD *)v8 + 74) <= a3 || (*((DWORD *)v8 + 75) > a4 )

    {

        if (*((DWORD *)v8 + 74) > a3 || (*((DWORD *)v8 + 75) <= a4
    )

        v11 = 1.0;

    else

        v11 = (double)(a4 / (*((DWORD *)v8 + 75));

    }

    else

    {

        v11 = (double)(a3 / (*((DWORD *)v8 + 74));

    }

}

else

{

    v9 = (double)(a3 / (*((DWORD *)v8 + 74));

```



```

v10 = (double)(a4 / *((_DWORD *)v8 + 75));

if (v9 <= v10 )

{

    v5 = (double)(a3 / *((_DWORD *)v8 + 74));

    *(double *)&v7 =
COERCE_DOUBLE(__PAIR__(HIDWORD(v9), LODWORD(v5)));

}

else

{

    v4 = (double)(a4 / *((_DWORD *)v8 + 75));

    *(double *)&v7 =
COERCE_DOUBLE(__PAIR__(HIDWORD(v10),
LODWORD(v4)));

}

v11 = *(double *)&v7;

}

BitBlt(

    *((HDC *)v8 + 68),

    *((_DWORD *)v8 + 72),

```

```

*((_DWORD *)v8 + 73),

*((_DWORD *)v8 + 74),

*((_DWORD *)v8 + 75),

*((HDC *)v8 + 69),

0,

0,

0xCC0020u);

return StretchDIBits(

    hdc,

    0,

    0,

    (signed __int64)((double)*((signed int *)v8 + 74) * v11),

    (signed __int64)((double)*((signed int *)v8 + 75) * v11),

    0,

    0,

    *((_DWORD *)v8 + 74),

    *((_DWORD *)v8 + 75),

    *((const void **)v8 + 345),

```

```

        (const BITMAPINFO *)((char *)v8 + 316),

        0,

        0xCC0020u);

}

// 4F812C: using guessed type int __thiscall
scene__render(_DWORD);

```

379. Ex. 1063.

380. The term "rendering" means "processing data into a different format including but not limited to drawing."

381. The display components in the Prucka CardioLab/Mac-Lab system processed diagnostic measurement parameter values into a different format including but not limited to drawing, a function of which was facilitated by the measurement processing components. Prucka Decl., Ex. 1053, ¶ 15. For example, in the Prucka CardioLab/Mac-Lab system I obtained, data was rendered on the screen using line graphs, an arterial tree view, and numerical values, as discussed in paragraphs 385-393, below. Further, the measurement processing components also facilitated rendering the data to disk for storage. Accordingly, the multiple processing components facilitated rendering the data.

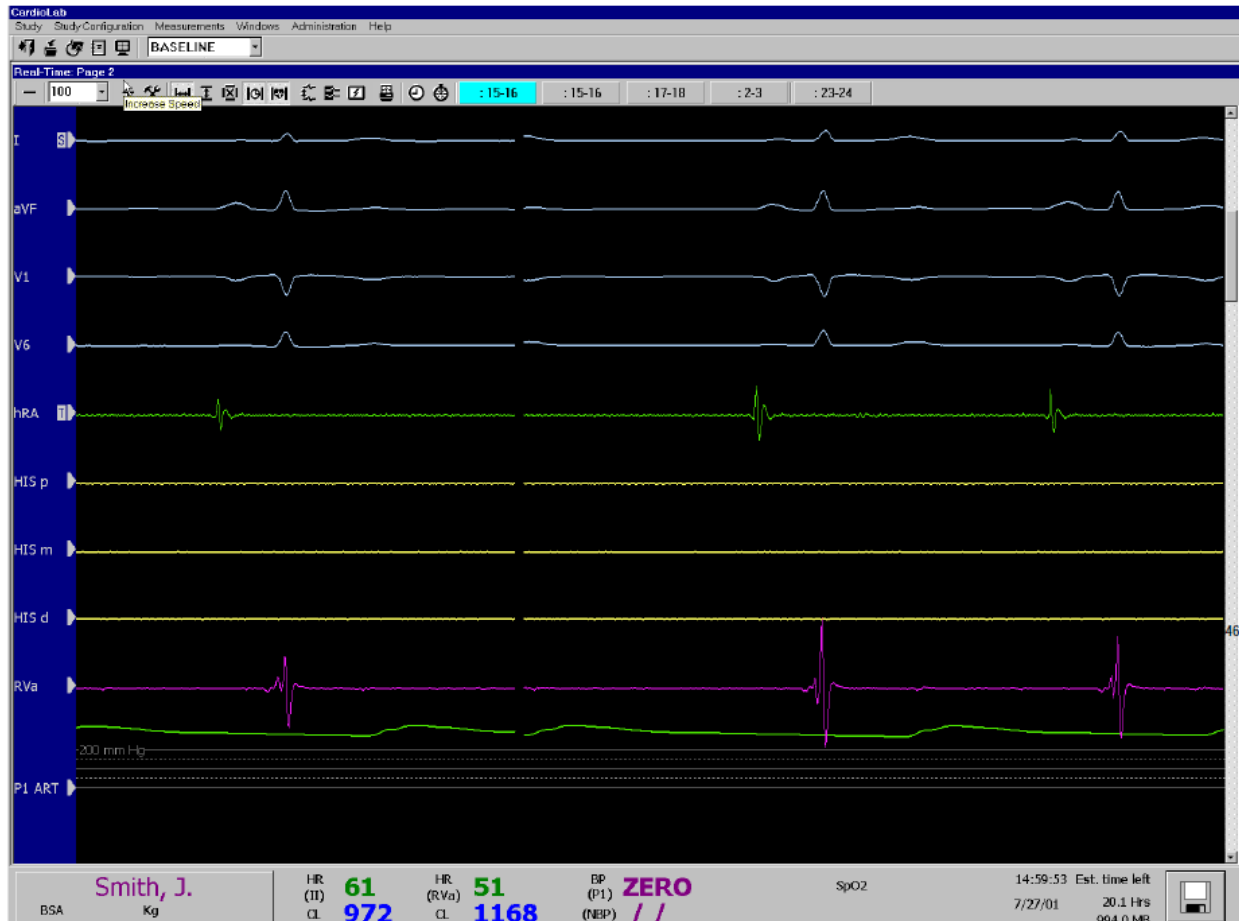
382. Accordingly, in the Prucka CardioLab/Mac-Lab system, the multiple display components received data from the multiple processing components, and the multiple processing components facilitated rendering diagnostic parameter values based on the received sensor data.

383. Therefore, the Prucka CardioLab/Mac-Lab system had a plurality of measurement processing components that operated at a user mode level in the multipurpose host system for facilitating receiving data of particular sensor types and for facilitating rendering diagnostic measurement parameter values according to the received data.

384. The Prucka CardioLab/Mac-Lab system included a multi-mode graphical user interface host comprising diagnostic measurement user interfaces including display components corresponding to data output rendered by specified ones of the plurality of measurement processing components.

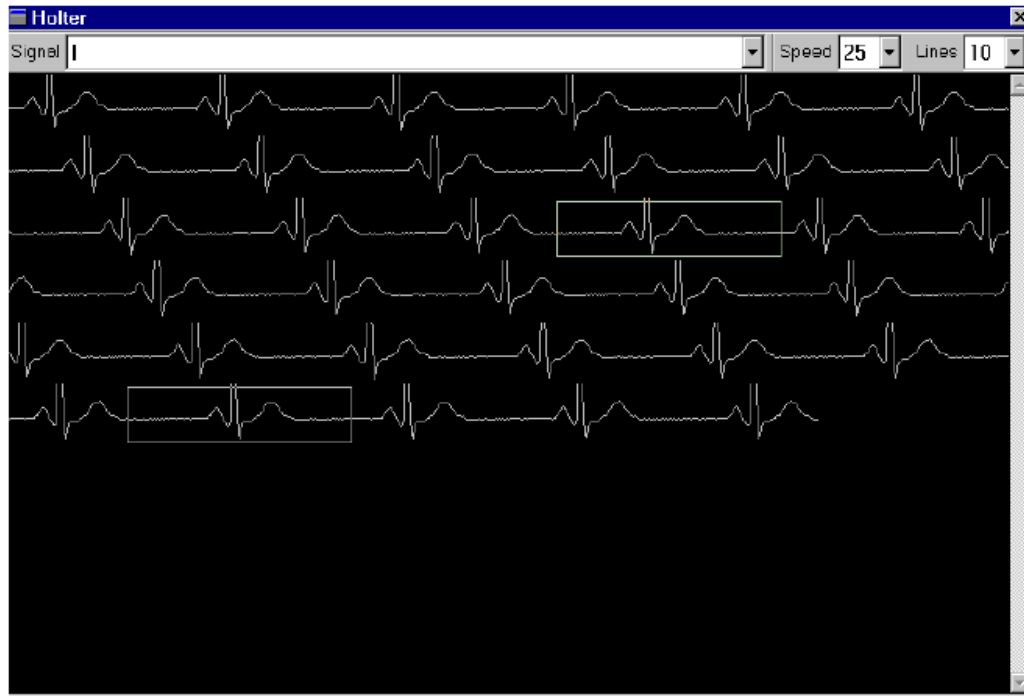
385. The Prucka CardioLab/Mac-Lab system had a human-computer interface with at least two distinct displays. For example, there were different displays for use in the CardioLab software and the Mac-Lab software depending on whether the user wanted to use the system for hemodynamic or electrophysiology purposes. Accordingly, the Prucka CardioLab/Mac-Lab system was a multi-mode system.

386. Further, each individual piece of software was multi-mode in that it provided a human-computer interface with at least two distinct displays including, for example, system or diagnostic displays. CardioLab software provided a screen for configuring each of the different types of sensors. A separate setup screen (the "review/setup mode" within the *Administration* menu) for ECG, Pressure, Stim, and two different sets of Intracardiac Blocks could be displayed. Prucka Decl., Ex. 1053, ¶ 12 & Ex. 9 (corresponding to Ex. 1093) at SJM03075573. Depending on what input sensor was selected, there were separate settings for gain, filtering, and other parameters per input device. Prucka Decl., Ex. 1053, ¶ 12 & Ex. 9 (corresponding to Ex. 1093) at SJM03075490. The user interface in the CardioLab software also had multiple diagnostic modes, including a mode that displayed graphs of different types of sensor data, including real-time intracardiac, ECG & pressure, heart rate, SPO₂ levels, NBP and patient data. Prucka Decl., Ex. 1053, ¶ 13 & Ex. 9 (corresponding to Ex. 1093). For example, in the screen shot captured below, the trace lines for hRA, HIS p, HIS m, HIS d, and RVa, represent inputs arising from catheters (see Prucka Decl., Ex. 1053, Ex. 9 (corresponding to Ex. 1093) at SJM03075464). The trace lines for I, aVF, V1, and V6 represent inputs arising from ECG sensors (see *id.* at SJM03075468). And the trace line for P1 ART represents a pressure sensor (see *id.* at SJM03075471).



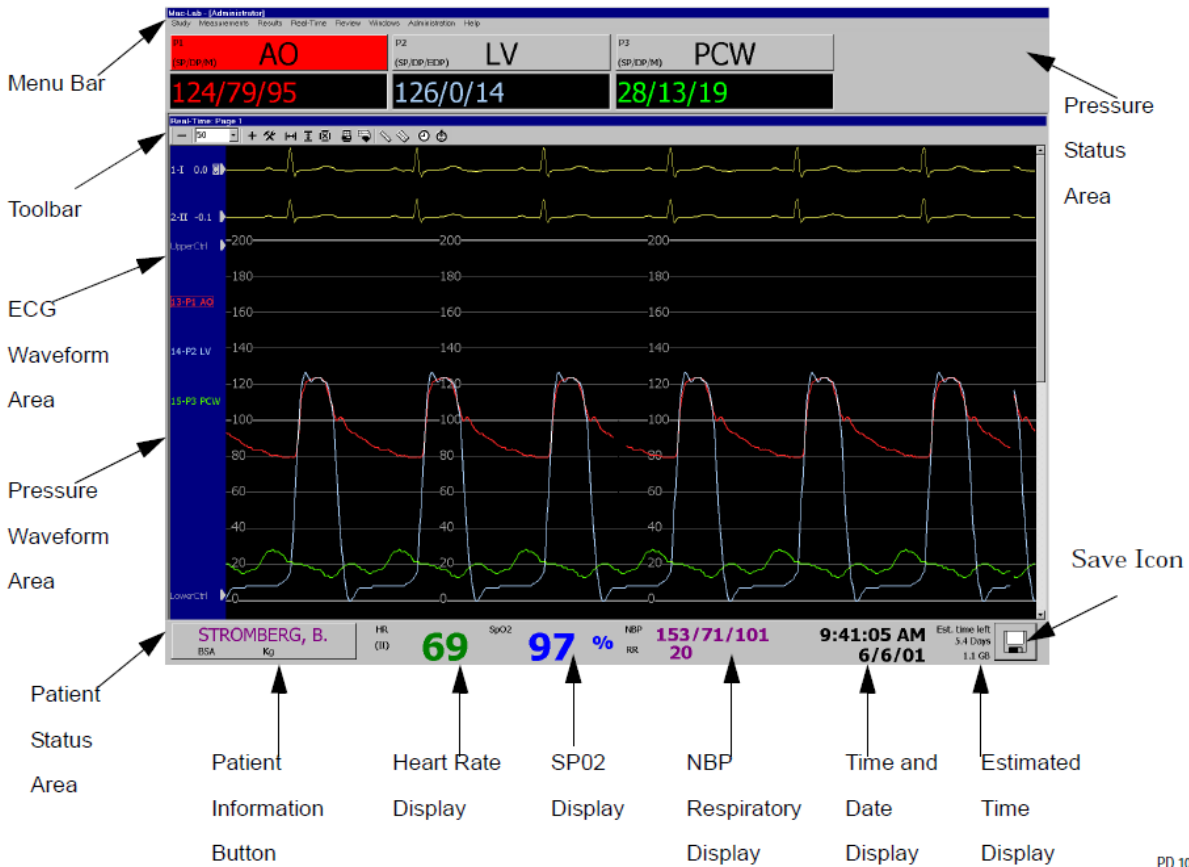
Id. at SJM03075486.

387. The CardioLab software also had a separate window for Holter analysis as shown below, which displayed heart beat information received from the sensors. These are just a few examples of multi-mode graphical displays that could be displayed.



Prucka Decl., Ex. 1053, ¶ 13 & Ex. 9 (corresponding to Ex. 1093) at SJM03075550; *see also* Ex. 3 (corresponding to Ex. 1087) at 11:11:50 – 11:13:37.

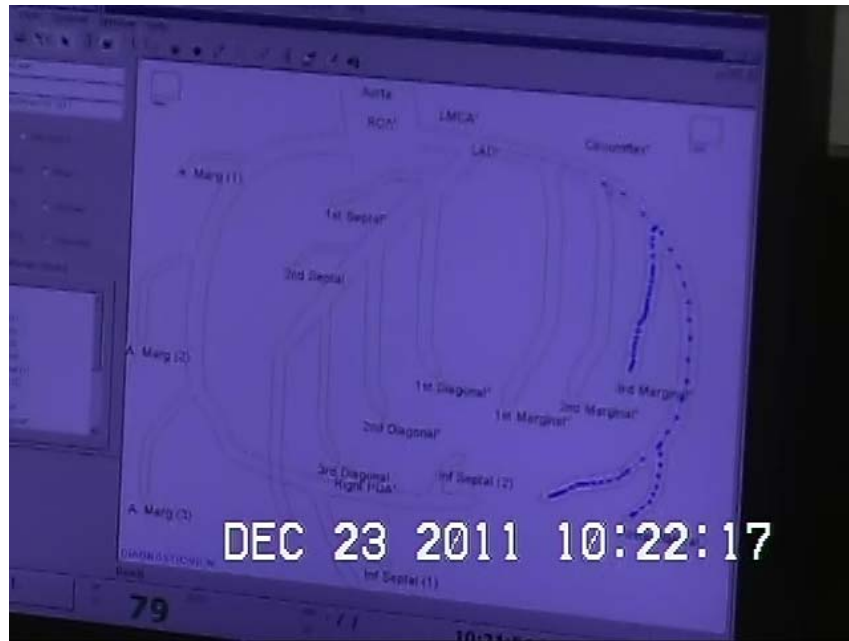
388. The Mac-Lab software also had a human-computer interface with multiple distinct displays including, for example, various system and diagnostic displays. The Mac-Lab software had a screen for configuring each of the different types of sensors. Prucka Decl., Ex. 1053, Ex. 10 (corresponding to Ex. 1094) at SJM03075213. The user interface in the Mac-Lab software also contained at least one diagnostic mode, which displayed graphs of different types of sensor data, including real-time intracardiac, ECG & pressure, heart rate, SPO₂ levels, NBP and patient data, as shown below:



PD 10015-21

Id. Ex. 10 (corresponding to Ex. 1094) at SJM03075201.

389. Mac-Lab software also had an arterial tree mode, as shown in the following photograph of the system I viewed:



See Ex. 3 (corresponding to Ex. 1087).

390. The multi-mode graphical user interface host in the Prucka CardioLab/Mac-Lab system included diagnostic measurement user interfaces, as shown in the preceding images.

391. As I explained and demonstrated in paragraph 372, above, the diagnostic measurement user interfaces in the Prucka CardioLab/Mac-Lab system included display components corresponding to data output rendered by specified ones of the plurality of measurement processing components.

392. Accordingly, the Prucka CardioLab/Mac-Lab system included a multi-mode graphical user interface host comprising diagnostic measurement user interfaces including display components corresponding to data output rendered by specified ones of the plurality of measurement processing components.

393. The Prucka CardioLab/Prucka Mac-Lab system practiced this claim limitation. The term "multi-mode graphical user interface host" means "software that supports a graphical user interface (including windows or portions of windows) for presenting at least two types of diagnostic information (including within the same window)". The Prucka CardioLab/Mac-Lab system practiced and anticipated this limitation of claim 1.

394. It is clear that the Prucka CardioLab/Mac-Lab system was programmed using C++, which is an object-oriented programming language. *See also, generally*, Prucka Decl., Ex. 1053, ¶ 14; *see also* paragraphs 352-354, above. Additionally, the Prucka CardioLab/Mac-Lab system had multiple distinct displays for presenting parameter values of multiple types, as shown in paragraphs 385-392, above.

395. Accordingly, the Prucka CardioLab/Mac-Lab system practiced and anticipated this limitation in claim 1 - *i.e.*, it included a multi-mode graphical user interface host comprising diagnostic measurement user interfaces including display components corresponding to data output rendered by specified ones of the plurality of measurement processing components.

396. The Prucka CardioLab/Mac-Lab system included one or more kernel mode drivers that extract processed sensor data from a peripheral interface card

providing a hardware interface for one or more invasive diagnostic measurement devices.

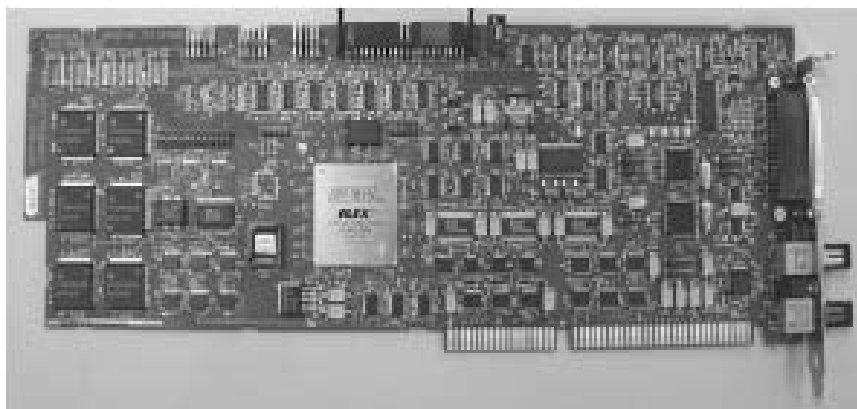
397. The Prucka CardioLab/Mac-Lab system used one or more invasive diagnostic measurement devices as the basis of its sensor data. Prucka Decl., Ex. 1053, Ex. 2 (corresponding to Ex. 1086) at SJM03070813. For example, the system had as many as 224 catheter inputs. *Id.* ¶ 6 & Ex. 3 (corresponding to Ex. 1087). I understand that catheters are invasive devices used for cardiovascular measurements. The system also had invasive blood pressure inputs on the TRAM. Prucka Decl., Ex. 1053, Ex. 2 (corresponding to Ex. 1086) at SJM03070813.

398. The Prucka CardioLab/Mac-Lab system then processed the sensor data from the invasive diagnostic measurement devices. The CardioLab software required an external amplifier called the CardioLab II/II Plus for data acquisition. *Id.* Ex. 8 (corresponding to Ex. 1092) at SJM03070854. The CardioLab II/II Plus amplifier had inputs for ECG, catheter/intercardiac, and blood pressure. *Id.* Ex. 3 (corresponding to Ex. 1087). The external amplifier filtered, amplified, digitized, further processed, and transmitted the data to the computer. Prucka Decl., Ex. 1053, ¶ 7 & Ex. 2 (corresponding to Ex. 1086) at SJM03070813.

399. The Mac-Lab software required an external amplifier called the TRAM for data acquisition. *Id.* The TRAM had inputs for ECG, invasive and non-invasive blood pressure, blood oxygen saturation via pulse oximetry,

respiration, and temperature. *Id.* The external amplifier filtered, amplified, digitized, further processed, and transmitted the data to the computer. *Id.*

400. The Prucka CardioLab/Mac-Lab system also had peripheral interface cards that provided a hardware interface to one or more invasive diagnostic measurement sensors. The COMMS ISA Expansion Card "communicate[d] with the CardioLab II/II Plus amplifier (via 2 ST connectors on the bracket)".



Prucka Decl., Ex. 1053, ¶ 10 & Ex. 8 (corresponding to Ex. 1092) at SJM03070854.

401. The RS422 PCI Expansion Card "allow[ed] the computer to communicate with the TRAM RAU module."



Id. & Ex. 8 (corresponding to Ex. 1092) at SJM03070853.

402. The Prucka CardioLab/Mac-Lab system also included one or more kernel mode drivers that extracted the processed sensor data from the peripheral interface cards.

403. The file called PEI.sys was one of the one or more kernel mode drivers that was used to extract data from the COMMS ISA Expansion Card. I determined this based on the file extension, the location of the file, and the import list in the file, in addition to by eliminating those drivers that were either device drivers provided standard with Windows NT or other known software components. I determined that the PEI.sys driver in the Prucka CardioLab/Mac-Lab system executed in kernel mode because it imported functions from the Windows kernel and hardware abstraction layer, such as ExAllocatePoolWithTag, KeGetCurrentThread, and KfRaiseIrql.

404. I determined that the file called PEISerial10.sys in the Prucka CardioLab/Mac-Lab system was used to extract data from the RS422 PCI Expansion Card.⁹ The RS422 protocol is a serial communications protocol. I

⁹ There was also a third kernel mode device driver called PEIMGANT40.sys, which I understand may have been used to extract data from the fluoroscopy imaging video card in the Prucka CardioLab/Mac-Lab system.

determined that the PEISerial10.sys is the driver that corresponds to the RS422 PCI Expansion Card based on the file extensions, the location of the file, and the import list in the file, in addition to the name of the driver (which indicates that it is a serial driver) and references within the driver to variables that are suggestive of serial communications, such as RxFIFO and TxFIFO. Additionally, I noticed that the software was written so that if it could not find the appropriate hardware device, it would return an error named,

"STATUS_SERIAL_NO_DEVICE_INITED," as follows:

```
if (v3->DeviceObject )
{
    v3->DriverUnload = (PDRIVER_UNLOAD)PEISerialUnload;
    v3->MajorFunction[9] =
(PDRIVER_DISPATCH)PEISerialFlushBuffers;
    v3->MajorFunction[4] = (PDRIVER_DISPATCH)PEISerialWrite;
    v3->MajorFunction[3] = (PDRIVER_DISPATCH)PEISerialRead;
    v3->MajorFunction[14] =
(PDRIVER_DISPATCH)PEISerialDeviceControl;
    v3->MajorFunction[0] = (PDRIVER_DISPATCH)PEISerialCreate;
    v3->MajorFunction[2] = (PDRIVER_DISPATCH)PEISerialClose;
    v3->MajorFunction[18] = (PDRIVER_DISPATCH)sub_14034; //
```

This was

not IRP_MJ_DEVICE_CHANGE in NT 4.0 (need to look it up to confirm)

```
v3->MajorFunction[5] =
```

```

(PDRIVER_DISPATCH)PEISerialQueryInformation;
    v3->MajorFunction[6] =
(PDRIVER_DISPATCH)PEISerialSetInformation;
}
else
{
    v2 = 0xC0000150u;          //
    STATUS_SERIAL_NO_DEVICE_INITED
}

```

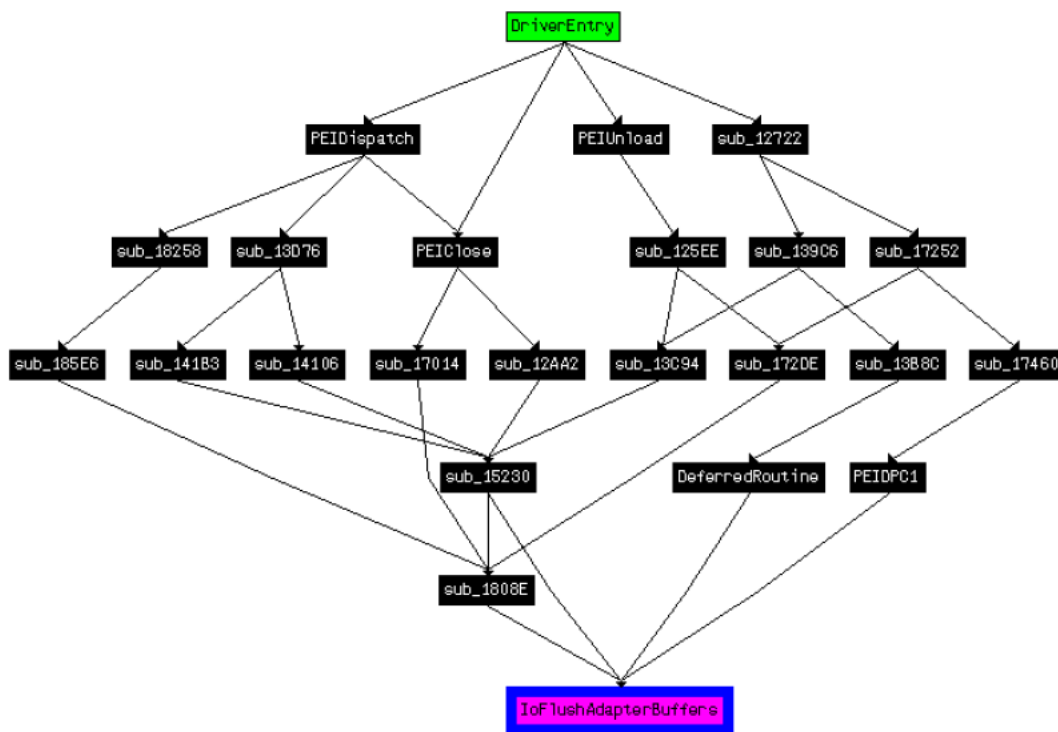
405. The invocation of STATUS_SERIAL_NO_DEVICE_INITED error indicates that the driver is a serial device driver.

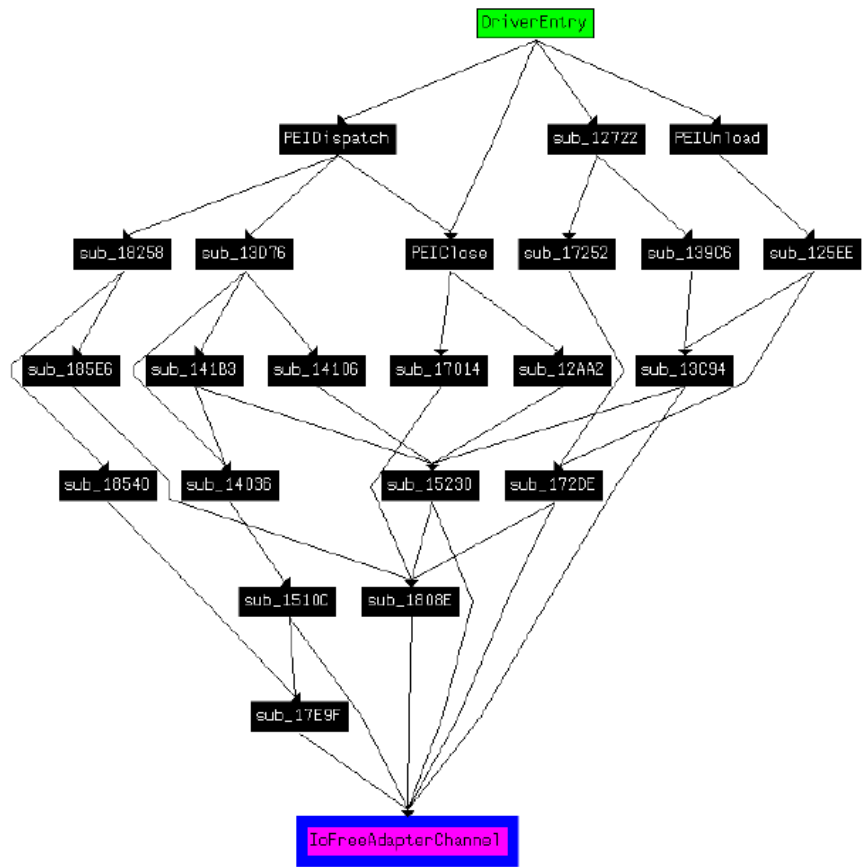
406. I determined that the PEISerial10.sys driver in the Prucka CardioLab/Mac-Lab system executed in kernel mode because it imported functions from the Windows kernel and hardware abstraction layer, such as ExAllocatePoolWithTag, KeDelayExecutionThread, and KfRaiseIrql.

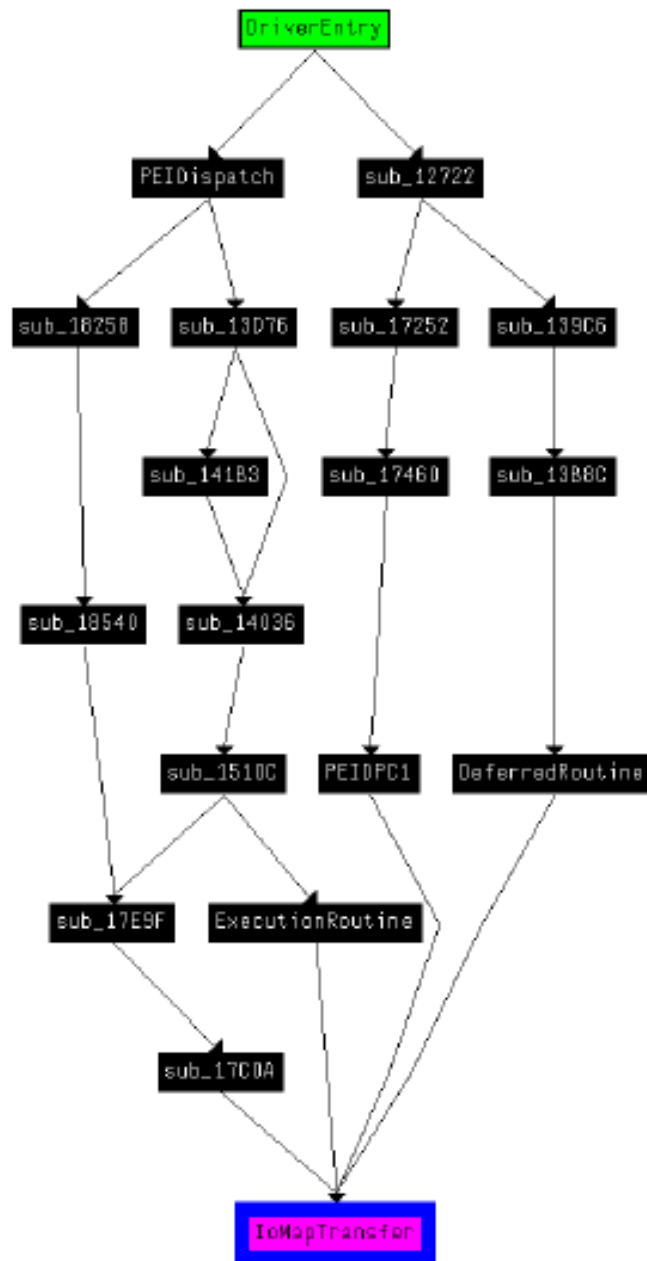
407. Having determined the kernel mode drivers that were used to communicate with the COMMS ISA Expansion Card and the RS422 PCI Expansion Card, I then analyzed the function of the drivers using the IDA Pro 6.2 forensic analysis tool and Hex-Rays Decompiler plugin version 1.6.

408. The PEI.sys driver in the Prucka CardioLab/Mac-Lab system used direct memory access ("DMA") to extract information from the COMMS ISA Expansion Card into the main Prucka CardioLab application. The use of direct

memory access is indicated by both the use of IoMapTransfer, which was a key Windows NT function used to initiate DMA, and IoFlushAdapterBuffers and IoFreeAdapterChannel, which were key Windows NT functions used for completing DMA. These functions within Windows NT were used to extract data from a hardware device, such as the COMMS ISA Expansion Card. The use of these functions is demonstrated in the following charts, which were created within IDA Pro 6.2 as a visual representation of the flow of control within the PEI.sys driver.







409. The routine labeled sub_128E8 within the driver was responsible for extracting the data from the buffers and communicating that data to the components in the main application. That decompiled routine is copied below:

```
//----- (000128E8) -----
NTSTATUS __stdcall sub_128E8(int a1, PIRP Irp)
```

```

{
    ULONG v2; // eax@1
    unsigned int v3; // eax@4
    int v4; // eax@7
    int v5; // eax@8
    int v6; // eax@9
    int v7; // eax@10

    v2 = Irp->Tail.Overlay.PacketType;
    Irp->IoStatus.Information = 0;
    if (*( _BYTE *)v2 )
    {
        if (*( _BYTE *)v2 == 2 )
        {
            sub_11A08(a1, Irp);
        }
        else
        {
            if (*( _BYTE *)v2 == 14 )
            {
                v3 = *( _DWORD *)(v2 + 12);
                if (v3 == -1214774272 )
                    return sub_129E4(*( _DWORD *)(a1 + 40), Irp);
                if (v3 == -1214774268 )
                    return sub_12A08(*( _DWORD *)(a1 + 40), Irp);
                if (v3 == -1214774264 )
                    return sub_12A08(*( _DWORD *)(a1 + 40), Irp);
            }
        }
    }
}

```

```

    v4 = (v3 >> 16) - 47010;
    if (!v4 )
        return sub_13D76(a1, Irp);
    v5 = v4 - 10;
    if (!v5 )
        return sub_18258(a1, Irp);
    v6 = v5 - 20;
    if (!v6 )
        return sub_18952(a1, Irp);
    v7 = v6 - 40;
    if (!v7 )
        return sub_167F2(a1, Irp);
    if (v7 == 10 )
        return sub_1690E(a1, Irp);
    }
}
}
else
{
    sub_119AE(a1, Irp);
}
Irp->IoStatus.Information = 0;
Irp->IoStatus.Status = -1073741822;
IoCompleteRequest(Irp, 0);
return -1073741822;
}

```

410. This routine shows a series of discreet subroutines, by which discreet components of the main application retrieve specific types of data in the Prucka CardioLab/Mac-Lab system. Accordingly, the PEI.sys driver extracted information from the COMMS ISA Expansion Card for the main user application.

411. I also analyzed the function of the PEISerial10.sys driver and determined that it extracted data from the RS422 PCI Expansion Card. The PEISerial10.sys driver implements the standard Windows serial interface, which allows a software application to communicate with any standard serial device attached to a computer running a Windows operating system. The following portion of software code reads the data from the serial device, such as the RS422 PCI Expansion Card:

```
int __stdcall PEISerialReadSubroutine(int DeviceExtension)
{
    int v1; // ebx@1
    int v2; // esi@1
    int v3; // eax@2
    int v4; // eax@3
    int v5; // eax@4
    KIRQL v6; // al@4
```

```

void (__fastcall *v7)(PKSPIN_LOCK, KIRQL); // edi@4

unsigned int v8; // esi@4

unsigned __int64 v9; // qax@6

unsigned __int8 v10; // zf@6

unsigned __int8 v11; // sf@6

unsigned __int8 v12; // of@6

int v13; // eax@9

int v14; // eax@13

unsigned __int64 v15; // qax@24

int v16; // esi@29

int v17; // eax@29

int v18; // ecx@37

int v19; // eax@39

int v20; // edi@47

int v21; // edi@51

unsigned int buffer; // [sp+Ch] [bp-3Ch]@4 - Best be a buffer of
0x14 bytes or longer, since we memcpy into it further along.

int v24; // [sp+10h] [bp-38h]@13

```

```
unsigned int v25; // [sp+14h] [bp-34h]@13

int SynchronizeContext; // [sp+20h] [bp-28h]@1

size_t v27; // [sp+24h] [bp-24h]@25

char v28; // [sp+28h] [bp-20h]@38

LARGE_INTEGER DueTime; // [sp+2Ch] [bp-1Ch]@24

int v30; // [sp+34h] [bp-14h]@44

KIRQL Irql[4]; // [sp+38h] [bp-10h]@29

int v32; // [sp+3Ch] [bp-Ch]@32

char v33; // [sp+42h] [bp-6h]@4

char v34; // [sp+43h] [bp-5h]@4

char v35; // [sp+44h] [bp-4h]@4

char v36; // [sp+45h] [bp-3h]@4

KIRQL NewIrql; // [sp+46h] [bp-2h]@25

char v38; // [sp+47h] [bp-1h]@1

char DeviceExtension_3; // [sp+53h] [bp+Bh]@4
```

```
v38 = 0;
```

```
v1 = DeviceExtension;
```

```

SynchronizeContext = DeviceExtension;

v2 = DeviceExtension + 228;

while (1 )

{

    v3 = *(_DWORD *)(*(_DWORD *)v2 + 96);

    if (*(_BYTE *)v3 != 3 )

    {

        v4 = sub_14E60(v1);

        goto LABEL_31;

    }

    v5 = *(_DWORD *)(v3 + 4);

    DeviceExtension_3 = 0;

    v35 = 0;

    v36 = 0;

    v34 = 0;

    v33 = 0;

    *(_DWORD *)(v1 + 300) = v5;

    KeInitializeTimer((PKTIMER)(v1 + 936));

```



```

KeInitializeTimer((PKTIMER)(v1 + 976));

v6 = KfAcquireSpinLock((PKSPIN_LOCK)(v1 + 516));

memcpy(&buffer, (const void *)(v1 + 388), 0x14u);//

v7 = KfReleaseSpinLock;

KfReleaseSpinLock((PKSPIN_LOCK)(v1 + 516), v6);

v8 = buffer;

if (buffer )

{

    if (buffer == -1 )

        goto LABEL_13;

    v33 = 1;

    v9 = sub_10E50(buffer, 10000i64);

    *(_DWORD *)(v1 + 72) = v9;

    v12 = __OFSUB__(HIDWORD(v9), *(_DWORD *)(v1 + 100));

    v10 = HIDWORD(v9) == *(_DWORD *)(v1 + 100);

    v11 = HIDWORD(v9) - *(_DWORD *)(v1 + 100) < 0;

    *(_DWORD *)(v1 + 76) = HIDWORD(v9);

```

```
    if (v11 ^ v12 || (unsigned __int8)(v11 ^ v12) | v10 &&  
*_(_DWORD *)(v1 + 72) < *(_DWORD *)(v1 + 96) )
```

```
    v13 = v1 + 80;
```

```
else
```

```
    v13 = v1 + 88;
```

```
    *(_DWORD *)(v1 + 144) = v13;
```

```
}
```

```
if (v8 != -1 )
```

```
{
```

```
    v14 = v24;
```

```
    if (v24 || v25 != v24 )
```

```
{
```

```
    DeviceExtension_3 = 1;
```

```
    goto LABEL_24;
```

```
}
```

```
    goto LABEL_25;
```

```
}
```

```
LABEL_13:
```

```

v14 = v24;

if (v25 )

{

    if (v25 != -1 )

    {

LABEL_17:

        if (v24 == -1 )

        {

            if (v25 == -1 )

                goto LABEL_25;

            DeviceExtension_3 = 1;

            v36 = 1;

            v34 = 1;

            v14 = 0;

        }

        else

        {

            DeviceExtension_3 = 1;

```

```
v36 = 1;
```

```
}
```

```
LABEL_24:
```

```
v15 = sub_10E50(*(_DWORD *)(v1 + 300), (unsigned int)v14);
```

```
DueTime = (LARGE_INTEGER)sub_10E50(v25 + v15, -  
10000i64);
```

```
goto LABEL_25;
```

```
}
```

```
}
```

```
else
```

```
{
```

```
if (v24 )
```

```
goto LABEL_17;
```

```
v35 = 1;
```

```
}
```

```
LABEL_25:
```

```
NewIrql = KfAcquireSpinLock((PKSPIN_LOCK)(v1 + 516));
```

```
v27 = sub_14B6E(v1);
```

```

    if (!v35 )

    {

        if (*(_DWORD *)(v1 + 300) && (!v36 || !*(_DWORD
*)*)(_DWORD *)(v1 + 228) + 28)) )

            break;

    }

    KeSynchronizeExecution(*(PKINTERRUPT *)(v1 + 192),
(PKSYNCHRONIZE_ROUTINE)sub_14CA5,
&SynchronizeContext);

    KfReleaseSpinLock((PKSPIN_LOCK)(v1 + 516), NewIrql);

    v19 = *(_DWORD *)(v1 + 228);

LABEL_41:

    *(_DWORD *)(v19 + 24) = 0;

    if (!v38 )

    {

        v32 = 0;

        goto LABEL_43;

    }

LABEL_44:

```

```

v2 = v1 + 228;

sub_1514D(v1 + 228, v1 + 196, (int)&v30, 1);

if (!v30 )

    return v32;

}

v16 = v1 + 228;

*(_DWORD *)(*(_DWORD *)(*(_DWORD *)(v1 + 228) + 96) +
16) = 0;

IoAcquireCancelSpinLock(Irql);

v17 = *(_DWORD *)(v1 + 228);

if (*(_BYTE *)(*(_DWORD *)(v1 + 228) + 36) )

{

    IoReleaseCancelSpinLock(Irql[0]);

    KfReleaseSpinLock((PKSPIN_LOCK)(v1 + 516), NewIrql);

v4 = -1073741536;

*(_DWORD *)(*(_DWORD *)v16 + 24) = -1073741536;

*(_DWORD *)(*(_DWORD *)v16 + 28) = 0;

LABEL_31:

```

```

    if (!v38 )

    {

        v32 = v4;

LABEL_43:

        v38 = 1;

        goto LABEL_44;

    }

    goto LABEL_44;

}

if (v34 )

{

    if (*(_DWORD *)(v17 + 28) || *(_DWORD *)(v1 + 300) !=
*_DWORD *)(*(_DWORD *)(v17 + 96) + 4) )

        RtlAssert(

            "(!Extension->CurrentReadIrp->IoStatus.Information) &&
(Extension->NumberNeededForRead ==
IoGetCurrentIrpStackLocation( Extension->CurrentReadIrp )-
>Parameters.Read.Length)",

            &word_143A6,

```

```

    0x202u,

    0);

v18 = *(_DWORD *)v16;

*(_DWORD *)(v1 + 300) = 1;

*(_DWORD *)(*(_DWORD *)v18 + 96) + 4 = 1;

}

KeSynchronizeExecution(*(PKINTERRUPT *)(v1 + 192),
(PKSYNCHRONIZE_ROUTINE)sub_14D61,
&SynchronizeContext);

if (v28 )

{

    IoReleaseCancelSpinLock(Irq[0]);

    KfReleaseSpinLock((PKSPIN_LOCK)(v1 + 516), NewIrql);

    v19 = *(_DWORD *)v16;

    goto LABEL_41;

}

if (DeviceExtension_3 )

{

    v20 = *(_DWORD *)(*(_DWORD *)v16 + 96) + 16;

```



```

if (*(_BYTE *)v20 & 4 )

    RtlAssert("!(*_arg4 & _refType)", ".\\read.c", 0x22Du, 0);

    *(_DWORD *)v20 |= 4u;

    KeSetTimer((PKTIMER)(v1 + 936), DueTime, (PKDPC)(v1 +
584));

    v7 = KfReleaseSpinLock;

}

if (v33 )

{

    v21 = *(_DWORD *)(*(_DWORD *)v16 + 96) + 16;

    if (*(_BYTE *)v21 & 8 )

        RtlAssert("!(*_arg4 & _refType)", ".\\read.c", 0x23Cu, 0);

        *(_DWORD *)v21 |= 8u;

        KeQuerySystemTime((PLARGE_INTEGER)(v1 + 104));

        KeSetTimer((PKTIMER)(v1 + 976), *(LARGE_INTEGER
*)*(_DWORD *)v1 + 144), (PKDPC)(v1 + 616));

        v7 = KfReleaseSpinLock;

}

*(_BYTE *)(*(_DWORD *)(*(_DWORD *)v16 + 96) + 3) |= 1u;

```

```

IoReleaseCancelSpinLock(Irql[0]);

v7((PKSPIN_LOCK)(v1 + 516), NewIrql);

if (!v38 )

    v32 = 259;

return v32;

}

```

412. The highlighted portion of the above software code: **memcpy(&buffer, (const void *)(v1 + 388), 0x14u);** copies the data from memory on the RS422 PCI Expansion Card. Accordingly, the PEISerial10.sys driver extracts the data from a peripheral interface card in the Prucka CardioLab/Mac-Lab system.

413. The Prucka CardioLab/Mac-Lab system practiced this claim limitation. The term "kernel mode drivers" means "software components that execute on the most privileged level in a computer, which permit a computer system to communicate with a device" The term "processed sensor data" means "sensor data that has been systematically modified, including by means of analog to digital conversion". A "peripheral interface card" is "a printed circuit board or adapter that can be plugged into a computer to provide added functionality or new

capability, which provides specialized services that are not built into the computer, and that interfaces with peripherals."

414. The Prucka CardioLab/Mac-Lab system practiced this limitation in claim 1. For example, the Prucka CardioLab/Mac-Lab system had code that facilitated communicating commands and data between interface hardware and a set of one or more user mode processes, as described in paragraphs 403-412, above.

415. Additionally, the Prucka CardioLab/Mac-Lab system applied appropriate mathematical operations to the sensor data before it was extracted by the kernel mode drivers. Prucka Decl., Ex. 1053, ¶ 7. For example, after the analog sensor signals for the CardioLab software were processed with analog processes, the signals were converted to digital signals in the CardioLab II/II Plus amplifier using a twelve-bit analog to digital converter. Prucka Decl., Ex. 1053, ¶ 7. Because the Prucka CardioLab used sixteen-bit words, the twelve-bit output of the analog to digital converters had to be mathematically treated for compatibility. *Id.* This was done by multiplying the output of the analog to digital converter times the binary number "1000." *Id.* The four least significant bits were then filled with marker data. *Id.* Accordingly, the sensor data in the Prucka CardioLab was "processed" before it was sent to the peripheral interface card. This digital signal processing was done prior to transmission of the digital signal values from the

CardioLab II/II Plus amplifier to the COMMS ISA Expansion Card. Accordingly, the PEI.sys kernel mode driver in the CPU extracted *processed* sensor data when the CardioLab software was in use.

416. After the analog sensor signals for the Mac-Lab software were processed with analog processes, the signals were converted to digital signals in the TRAM. The digital signals were then further processed using appropriate mathematical operations such as digital filtering in a Motorola 6800 series digital signal processor in the TRAM module.¹⁰ *Id.* The digital filtering process used addition and multiplication of numerical values. *Id.* This digital signal processing was done prior to transmission of the digital signal values from the TRAM to the RS422 PCI Expansion Card. Accordingly, the PEISerial10.sys kernel mode driver

¹⁰ In a Prucka CardioLab/Mac-Lab system I viewed and analyzed, I disassembled the TRAM module and found a microchip bearing a Motorola logo and the part number MC68HC000FN16. *See* December 23, 2011 Video provided as Exhibit 1145 at 11:43:54:



The MC68HC000FN16 was a microprocessor used for digital signal processing. Motorola, Addendum to M68000 User Manual (Aug. 7, 1997) at 2.

in the CPU extracted processed sensor data when the Mac-Lab software was in use.

417. Therefore, The Prucka CardioLab/Mac-Lab system practiced and anticipated each limitation of claim 1 of the '994 patent.

418. The Prucka CardioLab and Prucka Mac-Lab each qualify as prior art both under 35 U.S.C. §§ 102(a) & (b).

419. I understand from my discussions with counsel that a device is prior art under 35 U.S.C. § 102(b) if it was in public use or on sale in the United States more than one year prior to the date of the application for patent. Accordingly, a device would be prior art under § 102(b) if it practiced each limitation of claim 1 of the '994 patent and was in public use or on sale in the United States prior to May 20, 2001.

420. I also understand from my discussions with counsel that a device is prior art under 35 U.S.C. § 102(a) if it practiced each limitation of the patent and was known or used by others in the United States or was patented or described in a printed publication in the United States or a foreign country before the invention was made.

421. The Prucka CardioLab was in public use and was on sale in the United States prior to May 20, 2001 (which is one year prior to the May 20, 2002 102(a) priority date for the '994 patent). It was also known and used by others in

the United States prior to May 20, 2002. For example, the FDA granted 510(k) clearance to the Windows NT-based Prucka CardioLab and Prucka Mac-Lab on June 27, 2000. Prucka Decl., Ex. 1053, ¶ 4 & Ex. 2 (corresponding to Ex. 1086). That same year, GE Medical Systems published full color brochures describing the features of the Prucka CardioLab for purposes of selling the systems. Prucka Decl., Ex. 1053, ¶ 5 & Exs 3-6 (corresponding to Exs. 1087-1090). By December, GE Medical Systems sent its customers a letter announcing:

422. Due to the release of the new Mac-Lab 7000/4000/2000 in July 2000, GE Medical Systems has discontinued any further development of the Mac-Lab Classic product line as of December 31, 2000.

423. Prucka Decl., Ex. 1053, ¶ 17 & Ex. 12 (corresponding to Ex. 1096).

424. Additionally, GE Marquette issued quotes to U.S. hospitals for sales of Prucka CardioLab/Mac-Lab systems running version 5.0 software on the Windows NT operating system in early 2000. Prucka Decl., Ex. 1053, ¶ 18 & Ex. 13 (corresponding to Ex. 1097); Vadodaria Lit. Decl., Ex. 1133, ¶ 3. In 2000, GE Marquette sold CardioLab version 5.0 systems running Windows NT to both Stanford University and the Mayo Clinic. Vadodaria Lit. Decl., Ex. 1133, ¶ 2.

425. Prucka CardioLab/Mac-Lab systems running version 5.1 software, the second generation of Windows NT-based systems, were in production no later than the middle of 2001. *Id.* For my investigation in this case and the preparation of

this report, I acquired a Prucka CardioLab/Mac-Lab system computer running version 5.1 software, which was the software version available on Windows NT compatible systems as of 2001. I have analyzed Prucka CardioLab/Mac-Lab system computers made in 2000 and 2001.

426. In 2001, GE Marquette introduced an additional networking feature for the 7000 series of Prucka CardioLab/Mac-Lab systems called CardioLink. Prucka Decl., Ex. 1053, ¶ 16 & Ex. 11 (corresponding to Ex. 1095). A press release dated May 8, 2001 indicated that CardioLink was already in use at the University of Iowa and Beaver Medical Center in Pittsburgh as of that date, which further indicates that Prucka CardioLab 7000 systems were in use at those facilities as of May 8, 2001. GE Medical Systems, *New Tools Enhance Productivity and Accuracy in Electrophysiology Workflow and Diagnosis* (May 8, 2001), available at <http://www.thefreelibrary.com/GE+Tuesday+Technology+Brief+May+8%2c+2001.-a074300729> (last visited April 17, 2013) (Ex. 1077).

427. A Microsoft case study written about the CardioLab further states, citing a GE Medical Systems employee, that "[CardioLab was] was upgraded to Microsoft Windows NT in 2001." See Microsoft, *GE Healthcare Empowers Professionals with Windows Embedded-based Recording Systems Smart Design Helps Improve Workflow and Productivity* (Feb. 17, 2011) available at

http://65.55.12.249/canada/casestudies/Case_Study_Detail.aspx?casestudyid=4000009230 (last visited April 17, 2013) (Ex. 1076) ("[CardioLab and Mac-Lab] were upgraded to Microsoft Windows NT in 2001.").

428. The publication dates for relevant documentation further supports these dates of first use or first knowledge. For example, the first version of the CardioLab/Mac-Lab® 2000/4000/7000 System Service Manual was first published on June 15, 2000. Prucka Decl., Ex. 1053, Ex. 8 (corresponding to Ex. 1092) at SJM03070831. The first version of the Prucka CardioLab 2000/4000/7000 Operator's Manual for Software Version 5.1 was published August 9, 2001. Prucka Decl., Ex. 1053, Ex. 9 (corresponding to Ex. 1093) at SJM03075409. The first version of the Prucka Mac-Lab® 2000/4000/7000 Operator's Manual for Software Version 5.1 was first published July 10, 2001. Prucka Decl., Ex. 1053, Ex. 10 (corresponding to Ex. 1094) at SJM03075169. The Prucka CardioLab/Mac-Lab 7000 CardioLink Operator's Manual was first published March 19, 2001. Prucka Decl., Ex. 1053, Ex. 11 (corresponding to Ex. 1095).

429. By May 2002, prior to the '994 patent filing date, Prucka CardioLabs running Windows NT were widely used. For example, on May 9, 2002, GE issued a press release indicating that Prucka CardioLab 7000/CardioLink system was then "currently deployed at nearly 60 sites in the U.S." GE Medical Systems, *GE Medical Systems Information Technologies Introduces First and Only Networking*

Tool for CardioLab Electrophysiology Data (May 9, 2002), available at <http://www.thefreelibrary.com/GE+Medical+Systems+Information+Technologies+Introduces+First+and+Only...-a085587742> (last visited April 17, 2013). (Ex. 1075)

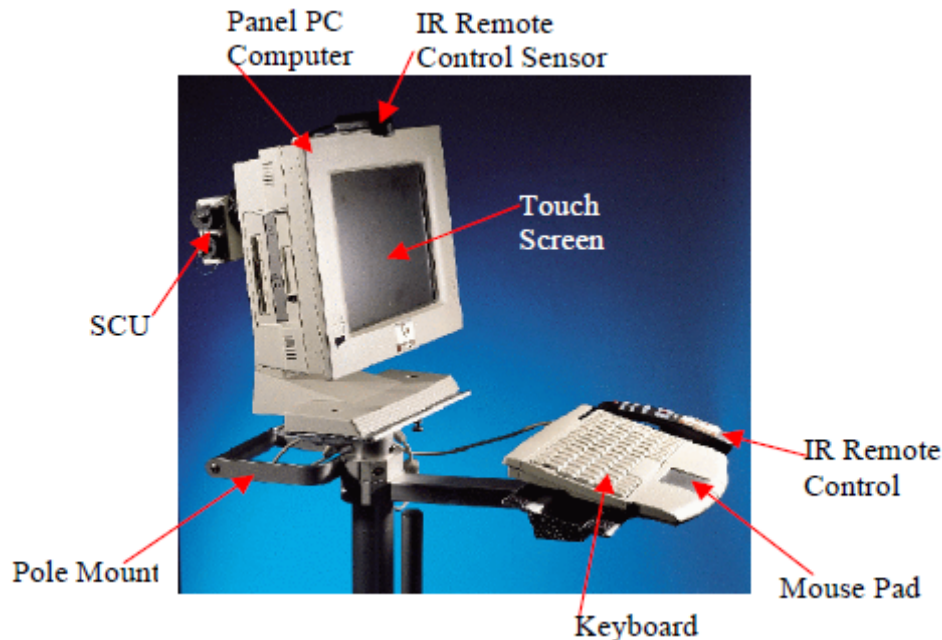
430. Accordingly, the Prucka CardioLab was in public use or was on sale in the United States before May 20, 2001, and was known or used by others in the United States prior to May 20, 2002. Thus, the Prucka CardioLab was prior art to the '994 patent under both 35 U.S.C. §§ 102(a) & (b). Because the Prucka CardioLab practiced and anticipated each limitation in claim 1 of the '994 patent, claim 1 of the '994 patent is invalid as anticipated under 35 U.S.C. § 102.

B. SmartFlow System

431. In this section of my report, I explain how both the SmartFlow system and the '669 Patent practiced and anticipated each limitation of claim 1 of the '994 patent. I will first address the SmartFlow system itself, and will then follow with my comments about the '669 Patent.

432. The SmartFlow system was a multipurpose host system.

433. The SmartFlow system was a personal computer-based medical monitoring system. Barak Lit. Decl., Ex. 1134, ¶ 6 ; Martin Decl., Ex. 1058, ¶ 3 & Ex. 3 (corresponding to Ex. 1100). For example, the personal computer used a Microsoft Windows operating system. Barak Lit. Decl., Ex. 1134, ¶ 6 & Exs. 4-6 (corresponding to Exs. 1083 and 1137-1138).



Barak Lit. Decl., Ex. 1134, Ex. 8 at SJM03072838.

434. As shown, the SmartFlow system contained a screen, a keyboard, a mouse pad, and a remote control. *Id.*

435. The SmartFlow system was used for invasive cardiovascular diagnostic measurement acquisition and display.

436. The SmartFlow system was "an advanced generation system for real time analysis of intravascular pressure measurement and to evaluate the hemodynamic status of diseased arteries and to provide further clinical information in the diagnosis and treatment of both coronary and peripheral artery diseases." Martin Decl., Ex. 1058, Ex. 3. It also provided display capabilities for invasive cardiovascular diagnostic measurements. For example, the SmartFlow system had a Measure mode, which displayed multiple graphs with separate soft-keys for the

user to choose between proximal, distal, vasodilation, and coronary wedge pressure measurements, as shown below:

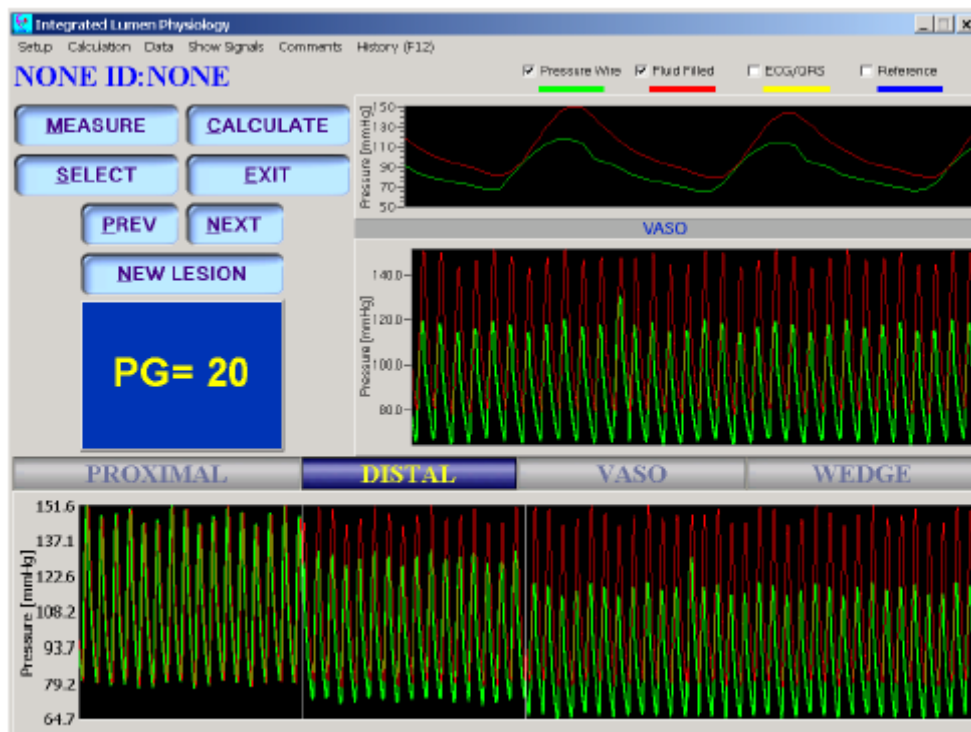


Fig 15: The measure mode window.

Barak Lit. Decl., Ex. 1134, ¶ 8 & Ex. 8 (corresponding to Ex. 1027) at SJM03072853.

437. The SmartFlow system also had a CFR/FFR plot mode, which displayed the CFR and FFR values acquired per heart beat, as shown below:

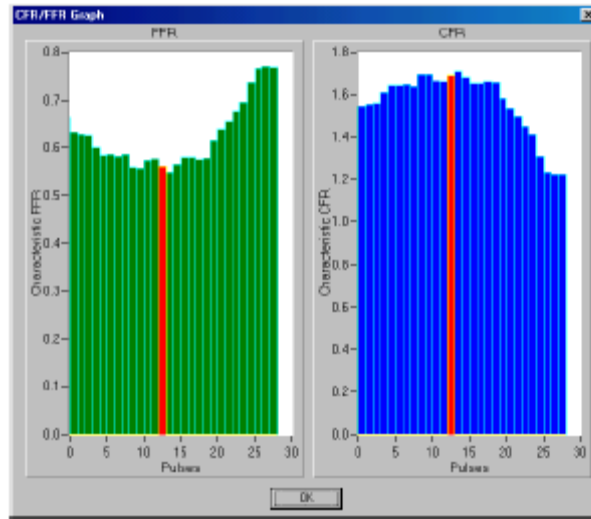


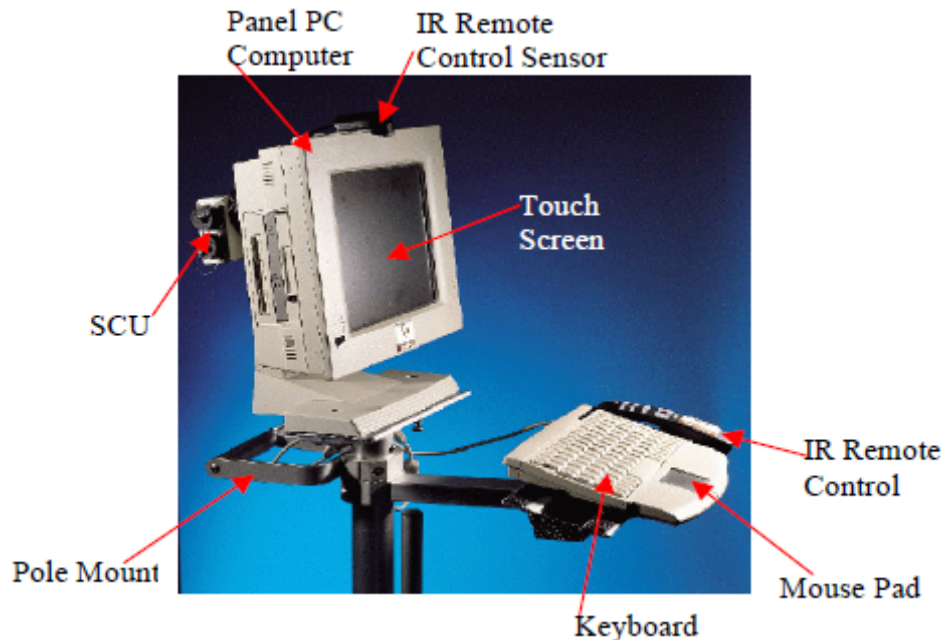
Fig 20: A graphic presentation of the CFR and FFR values per beat.

Barak Lit. Decl., Ex. 1134, ¶ 8 & Ex. 8 at SJM03072856.

438. The SmartFlow system therefore practiced and anticipated this limitation of claim 1 – *i.e.*, it was used for invasive cardiovascular diagnostic measurement acquisition and display.

439. The SmartFlow system incorporated a component based arrangement.

440. The SmartFlow system included a variety of hardware components. For example, the SmartFlow system had a Panel PC Computer (referred to by Florence Medical as the SmartFlow Integrated Processor, or "SFIP," Barak Lit. Decl., Ex. 1134, ¶ 6) (Ex. 1134), a Signal Conditioning Unit ("SCU"), a pole mount, a keyboard, an infrared remote control, and a mouse pad:



Barak Lit. Decl., Ex. 1134, Ex. 8 at SJM03072838.

441. A bill of materials showing a number of components that were assembled into the SFIP computer itself is attached to the Barak Lit. Decl., Ex. 1134, as Exhibit 5 (corresponding to Ex. 1137), and the Product Tree for the SFIP is attached to the Barak Lit. Decl., Ex. 1134, as Exhibit 6 (corresponding to Ex. 1138).

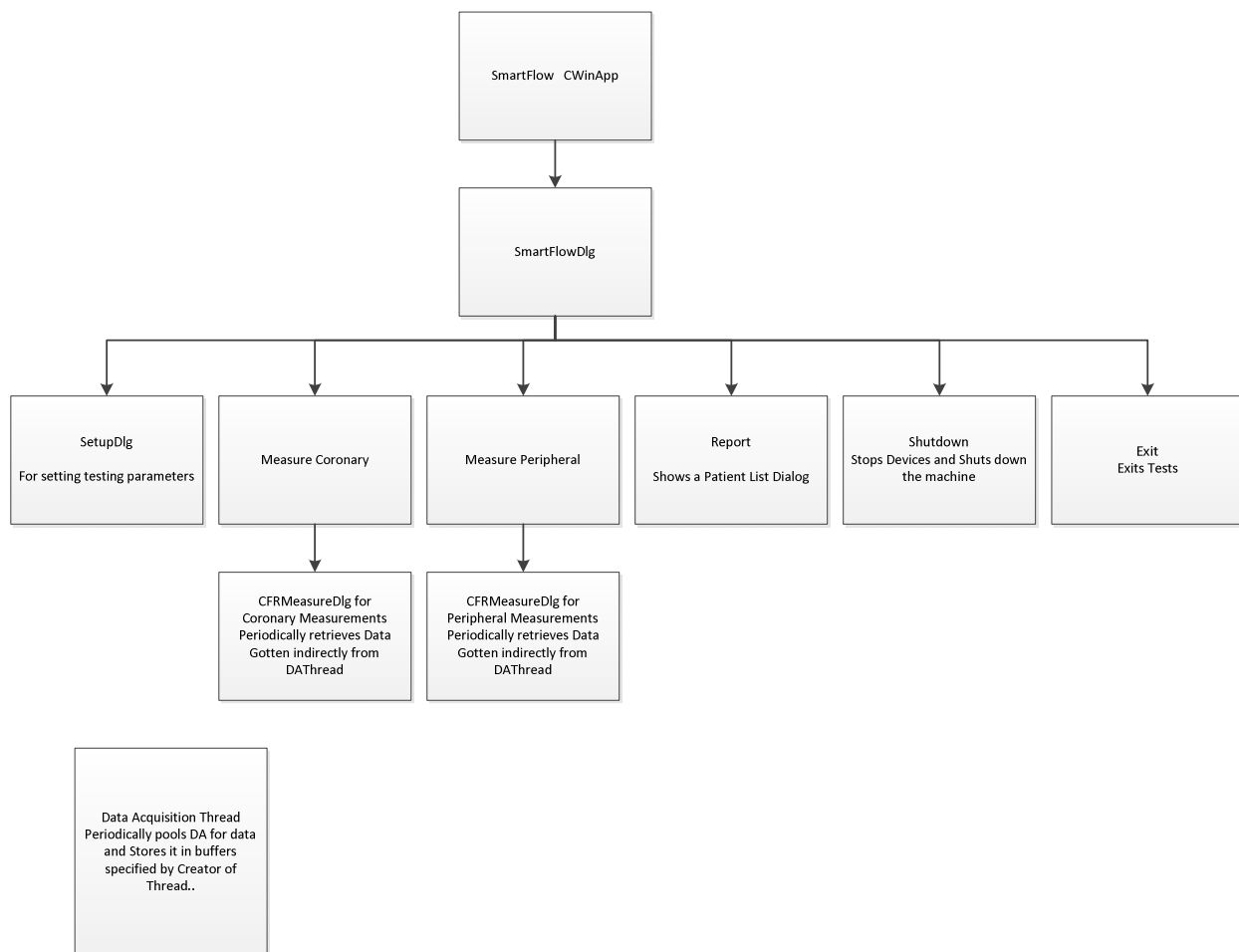
442. The SmartFlow system also included a variety of software components. To determine this, I analyzed the source code attached to the Barak Lit. Decl., Ex. 1134, as Exs. 19 & 20 (corresponding to Exs. 1142-1143). It is clear from the source code I acquired that the SmartFlow system uses the modular decomposition that is normal for software in the relevant time period. For example, the "HWSetup" directory (which I understand to be the "hardware setup"

directory) contains clear and distinct components for setting up the channel, gain, scan rate, transducer, and vascular method.¹¹ The "CFRMeasure" directory also consisted of numerous components, including yes/no dialog boxes (prompting the user for a yes/no answer to specific questions), a query to the user as to the correct CFR calculation method to use, the module for displaying the CFR measurement, and various components for controlling different graphical elements on the screen (including buttons, display axes, labels, graphical plot information, multi-lesion display information, information save operations, signal information, and a web browser interface control).

443. The SmartFlow system included a component based arrangement. A "component based arrangement" means "software arrangement that is extensible, modular." The software components in the SmartFlow system were extensible, modular because they were designed in the standard fashion to achieve this – they

¹¹ Although the source code attached to the Barak Lit. Decl., Ex. 1134, is version 5.1.0.9, which I understand was released after May 20, 2002, the majority of the software components have "Date Modified" dates denoting their creation between 1999 and 2001. Accordingly, it is my opinion that SmartFlow systems originally offered for sale in the relevant time period (*i.e.*, prior to May 20, 2002) satisfied this claim limitation.

used the standard Microsoft tools of the time period, utilizing the C++ object oriented methodology, encapsulating distinct functionality into class libraries and exploiting the inherent nature of Windows programming to enable this model of programming. I have prepared the following diagram based on my analysis of the software code provided to me, which demonstrates some of the components (called "classes" in C++) that were in the SmartFlow system user mode application.



444. Each of the above boxes represents a separate user mode component (C++ class) in the SmartFlow system, each of which can be found in the directories

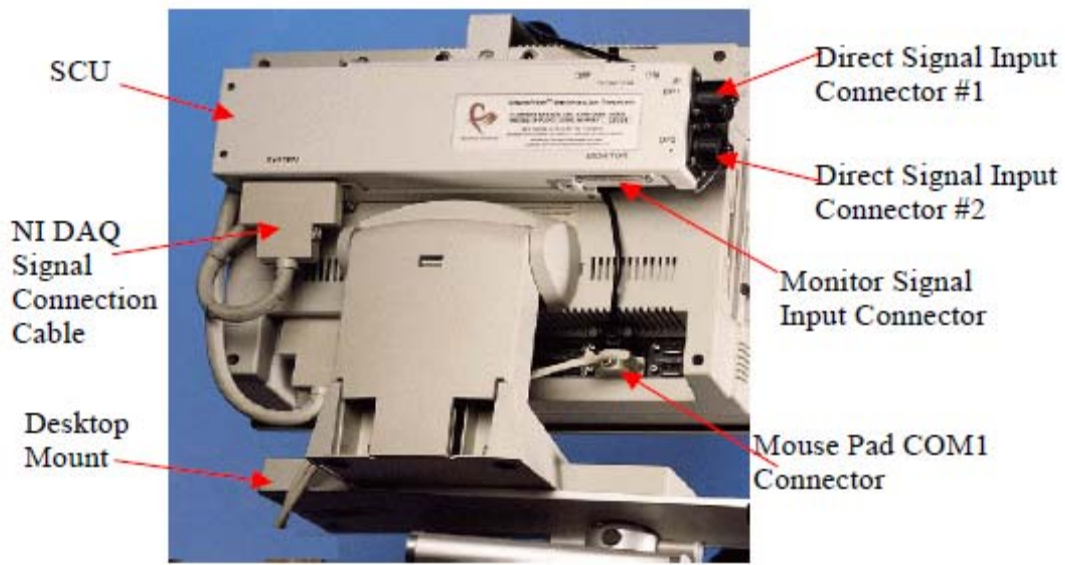
in the source code disk. *See* Barak Lit. Decl., Ex. 1134, Ex. 19 (corresponding to Ex. 1142).

445. Accordingly, the SmartFlow system therefore practiced and anticipated this limitation of claim 1.

446. The SmartFlow system included an external input signal bus interface for receiving data arising from cardiovascular diagnostic measurement sensors.

447. The SmartFlow system received data from cardiovascular diagnostic measurement sensors. The SmartFlow system was "designed to characterize a stenosis within a blood vessel, based on pressure input raw data It receive[d] its input data by connection to a variety of standard medical monitors as well as directly from pressure measurement devices . . ." Barak Lit. Decl., Ex. 1134, Ex. 8 (corresponding to Ex. 1027) at SJM03072839. "It accept[ed] inputs from a variety of standard medical monitors and pressure sensing units and prepare[d] these signals for use by the SmartFlow™ Intravascular Processor." *Id.* Ex. 8 at SJM03072840.

448. The SmartFlow system also had an external signal bus interface for receiving this data. For example, the SmartFlow system had direct input connections on the SCU and the SFIP connected to the SCU by means of a peripheral interface card and the NI DAQ Signal Connection Cable.



Barak Lit. Decl., Ex. 1134, Ex. 8 (corresponding to Ex. 1027) at SJM03072838.

449. The SmartFlow system therefore practiced and anticipated this limitation of claim 1 - *i.e.*, it included an external input signal bus interface for receiving data arising from cardiovascular diagnostic measurement sensors.

450. The SmartFlow system included a plurality of measurement processing components, that operate at a user mode level in the multipurpose host system, for facilitating receiving data of particular sensor types and rendering diagnostic measurement parameter values according to the received data.

451. As noted, the SmartFlow system was written in C++ using the Microsoft Foundation Class, an object-oriented programming language and component-based software engineering framework frequently used for writing component-based software applications. Additionally, as I explained in Section

II.E of this Report, a conventional way of using the modularity allowed by C++ and MFC programming is to utilize separate components for processing as opposed to display. In accordance with the MFC, the SmartFlow system does its data processing in certain software components, which are distinct from the software components used for display.

452. Based on my review of the source code attached to the Barak Lit. Decl., Ex. 1134,, I determined that the SmartFlow system had a plurality of measurement processing components that operated at a user mode level in the multipurpose host system. These processing components facilitated receiving data of particular sensor types and rendering diagnostic measurement parameter values according to the received data. For example, in the SmartFlow software directory CPBDBuffDAQ, the file DAQDBdlg.cpp contains multiple processing components that facilitate receiving data of particular sensor types. One of the processing components is GetDataFromChannels. Line 404 of that processing component receives data from the different, particular types of sensors connected to the SmartFlow system, including "pressure wire" and "fluid filled wire" as two of the particular types of sensors.

```
//// calc mean values //////////////////////////////////////  
int nn=0;  
double meanPW=0, meanFF=0;  
if(i==0) { //pressure wire
```

```

        for(int j=0;j<CHPNTS &&
nn<CHPNTS_REDUCED;j+=m_iPlotJump) {
            meanPW+=
                (float)(m_dfVoltstmp[j]*pcSetupDat-
>GetGainAdjust()[0]+pcSetupDat->GetOffset()[0]);
            nn++;
        }
        if(nn>0) {
            meanPW/=nn;
            m_pcCfrFfrApp-
>m_fMeanPW=(float)meanPW;
        }
    }
    nn=0;

    if(i==2) { fluid filled wire
        for(int j=0;j<CHPNTS &&
nn<CHPNTS_REDUCED;j+=m_iPlotJump) {
            meanFF+=
                (float)(m_dfVoltstmp[j]*pcSetupDat-
>GetGainAdjust()[2]+pcSetupDat->GetOffset()[2]);
            nn++;
        }
        if(nn>0) {
            meanFF/=nn;
            m_pcCfrFfrApp->m_fMeanFF=(float)meanFF;
        }
    }

```

```

    }
    //////////////////////////////////////

```

453. Barak Lit. Decl., Ex. 1134, Ex. 19 (corresponding to Ex. 1142) at \SmartFlow5.1.0.9\CPBDBuffDAQ\DAQDBDlq.cpp line 446 (emphasis added).

454. The same processing component, GetDataFromChannels, processes the data by, for example, by adding and dividing the sensor data in the portion of code that follows:

```

for(int j=0;j<CHPNTS &&
nn<CHPNTS_REDUCED;j+=m_iPlotJump) {
    meanPW+=
        (float)(m_dfVoltstmp[j]*pcSetupDat-
>GetGainAdjust()[0]+pcSetupDat->GetOffset()[0])

```

455. Barak Lit. Decl., Ex. 1134, Ex. 19 (corresponding to Ex. 1142) at \SmartFlow5.1.0.9\CPBDBuffDAQ\DAQDBDlq.cpp line 446 (emphasis added).

456. The same processing component, GetDataFromChannels, facilitated rendering the diagnostic measurement parameter values according to the received data. The processing component GetDataFromChannels called a method called "SetData," which sets "data for signal popup window." *Id.* line 527

("m_cRunApp->m_cSignalWndPlotData.SetData(i, CHPNTS, m_iPlotJump, m_dfVoltstmp); //**data for signal popup window**" (emphasis added)). The term "rendering" means "processing data into a different format including but not

limited to drawing." This processing component GetDataFromChannels facilitated rendering the data in the "signal popup window".

457. A separate processing component, the method called "OnTimer" in the class CFRMeasureDLG, facilitated receiving the following types of data: distal, proximal, hyperemic, and wedge, which can be seen in the following portion of software code:

```
case 1: //Measure
    {
        if(m_bIsButtonsEnabled)
        {
            //m_cbutMeasure.SetValue(FALSE);
            if(CPBDAQ_IsDAQrunning())
            {
                EnableButtons(FALSE);
                ///m_cbutSelect.SetEnabled(TRUE);
            //    CheckPBlength();
            int progrrange =10;
            int progrstep =1;
            HWND
            hDAQwnd=CPBDAQ_GetDAQhwnd();
            switch(m_NstateCount)
            {
                case 1:
                case 2:
                    {
```

```

                                progrrange=m_pcSetupDat-
>GetTimeOfAcquisition()*40;
                                progrstep=progrrange/(m_pcSetupDat-
>GetTimeOfAcquisition()*4);
m_iDataBufLength=m_pcSetupDat-
>GetScanRate(SCANRATE_CFR)*m_pcSetupDat-
>GetTimeOfAcquisition();
                                if(m_NstateCount==2)
m_pcProgressWnd=new
CProgresWndDlg(progrrange,progrstep,"Please Wait, Acquiring
Distal
Data");
                                else m_pcProgressWnd=new
CProgresWndDlg(progrrange,progrstep,"Please Wait, Acquiring
Proximal
Data");
                                }
                                break;
case 3:
                                {
                                int k_distvaso=m_pcSetupDat-
>GetKdistVaso();
                                progrrange=m_pcSetupDat-
>GetTimeOfAcquisition()*40*k_distvaso;
                                progrstep=(progrrange/(m_pcSetupDat-
>GetTimeOfAcquisition()*4))/k_distvaso;
m_iDataBufLength=m_pcSetupDat-

```

```

>GetScanRate(SCANRATE_CFR)*m_pcSetupDat-
>GetTimeOfAcquisition()*k_distvaso;

        m_pcProgressWnd=new
CProgresWndDlg(progrrange,progrstep,"Please Wait, Acquiring
Hyperemic
Data");

        }
        break;
    case 4:
        {
            int k_distvaso=m_pcSetupDat-
>GetKdistVaso();

            progrrange=m_pcSetupDat-
>GetTimeOfAcquisition()*40 *k_distvaso;

            progrstep=(progrrange/(m_pcSetupDat-
>GetTimeOfAcquisition()*4))/k_distvaso;
m_iDataBufLength=m_pcSetupDat-
>GetScanRate(SCANRATE_CFR)*m_pcSetupDat-
>GetTimeOfAcquisition()*k_distvaso;

            m_pcProgressWnd=new
CProgresWndDlg(progrrange,progrstep,"Please Wait, Acquiring
Wedge
Data");

            // progrrange=m_pcSetupDat-
>GetTimeOfAcquisition()*40;

            // progrstep=progrrange/(m_pcSetupDat-
>GetTimeOfAcquisition()*4);

```

```

        //
        m_iDataBufLength=m_pcSetupDat-
        >GetScanRate(SCANRATE_CFR)*m_pcSetupDat-
        >GetTimeOfAcquisition();

        }

        break;

    }

    //    m_pcProgressWnd=new
    CProgresWndDlg(progrrange,progrstep,"Please
    Wait, Acquiring Data");

    CALCCFR_OnStartMeasure(m_iDataBufLength);
    //
    //
    CALCCFR_SetMeasureParam(m_NstateCount, 1,
    m_enmSelectedArtery,
    m_enmSelectedArtDiametr, m_enmKindOfMesure);
    //
    //
    CALCFR_OnStartMeasurePullBack(m_fStep, m_fLength,
    m_fSpeed);
    //
    CALCFR_OnCaptureMeasurePullBack(m_fStep, m_fLength,
    m_iDataBufLength);

    ::SendMessage(hDAQwnd,WM_USERMSG_STARTDATACOLLE
    CT,m_iDataBufLength,

```



```

        m_iDataBufLength);
    }
    // comment out 2.06.2003
    /*if(!CPBDAQ_IsCardPresent() &&
(m_NstateCount==3 ))//||
m_NstateCount==4))
    {
        m_cwKnobSelectMeasure.EnableWindow(FALSE);
        if(!CFRFFR_IsTakeVasoEveryPoint() ||
m_iMultLesCurrentLes==m_iNumOfLesions+2 ||
IS_ASK_OPPER_AT_EVERY_VASO_PNT){
            CSaveOperationDlg
saveopper(m_enmSelectedOperation);
            if(m_pcMultLesScreen)
m_pcMultLesScreen->EnableWindow(FALSE);
            if(saveopper.DoModal()==IDOK)
            {

m_enmSelectedOperation=saveopper.GetSelectedOperation();
            }
            if(m_pcMultLesScreen)
m_pcMultLesScreen->EnableWindow(TRUE);
        }
    }*/
    if(!CPBDAQ_IsCardPresent())
OnNextMeasurementGraph();
    if(m_pcMultLesScreen) m_pcMultLesScreen-

```

```

>EnableWindow(FALSE);
    }
    break;
}

```

458. Barak Lit. Decl., Ex. 1134, Ex. 19 (corresponding to Ex. 1142) at \cfrmeasure\cfrmeasuredlg.cpp at 1191 (emphasis added).

459. I understand that the data referred to as proximal, distal, hyperemic and wedge in the SmartFlow system was received from multiple sensors of particular sensor types.

460. The same processing component, OnTimer, processed the sensor data. For example, the following portion of software code from the OnTimer processing component indicates in the first line that it "Calculate[s]" the sensor data:

```

        case 2: //Calculate
            {

//

        if(IsItMultipleLesion() && m_pcMultLesScreen !=NULL) {

            delete m_pcMultLesScreen;

            m_pcMultLesScreen=NULL;

        }

```

```

UpdateWindow();

EnableButtons(FALSE);

m_bIsItAdvancedCalculation=FALSE;

//replot to select
positions according to the last option.

PlotCollectedData();

// save before
calculating:

if(!CALCCFR_IsDataSavedAlready()) {

char
flbuf[444], flmsg[444];

CFRFFR_StoreCurrentLeasionData(flbuf, flmsg, TRUE);

CString
csFlbnm="SmartFlow Physiology Measurement, ";

```

```
csFlbnm+=flmsg;
```

```
}
```

```
CALCCFR_Calc(m_icfr, m_iffr, m_enmKindOfMeasure,  
m_fFFRtarget, m_fFFRtarget2);
```

```
WriteMsg("Please
```

```
Wait, Calculating ...");
```

```
SetCursor(AfxGetApp()->LoadStandardCursor(IDC_WAIT));
```

```
AfxGetApp()-
```

```
>DoWaitCursor(1);
```

```
m_pWaitWindDlg=new CWaitWindDlg(NULL, this);
```

```
//////////
```

```
DisplayKaorticnotch();
```

```
break;
```

461. Barak Lit. Decl., Ex. 1134, Ex. 19 (corresponding to Ex. 1142) at \cfrmeasure\cfrmeasuredlg.cpp at 1284 (emphasis added).

462. The same processing component, OnTimer, facilitated rendering the diagnostic measurement parameter values according to the received data. The processing component OnTimer facilitated rendering the sensor data into a variety of different formats. One type of different format was drawing the data onto graphs or other display formats in the graphical user interface, as occurred in the "repaint" and "signal popup window" methods. An example is demonstrated in the following portion of SmartFlow software source code:

```
case 5: //Prev
    {
//
        if(IsItMultipleLesion())
        {
            if(!m_pcMultLesScreen)
            {
                if(m_iMultLesCurrentLes>1 &&
m_iMultLesCurrentLes<=m_iNumOfLesions+1)
m_iMultLesCurrentLes--;
                if(m_NstateCount==3)
m_iMultLesCurrentLes=m_iNumOfLesions-1;
                if(CFRFFR_IsTakeVasoEveryPoint() &&
m_NstateCount==2) m_iMultLesCurrentLes=m_iNumOfLesions;
                OpenMultLesionMeasureScr();
            }
        }
    }
```

```

        }
        else
        {
            switch(m_NstateCount) {
                case 1: //PROX. point, go to WEDGE
                    m_NstateCount=4;

                    m_cwKnobSelectMeasure.GetActivePointer().SetValuePairIndex(m_
NstateCount);

                    m_iMultLesCurrentLes=m_iNumOfLesions+2;
                    break;
                case 2: //DIST. point, go to prev. point
                    HYPEREMIC
                    if(m_iMultLesCurrentLes>2)
                        m_NstateCount++;
                    else m_NstateCount=1; // prox

                    if(!CFRFFR_IsTakeVasoEveryPoint() &&
m_iMultLesCurrentLes>2)
                        m_NstateCount--;

                    m_cwKnobSelectMeasure.GetActivePointer().SetValuePairIndex(m_
NstateCount);

                    if(m_iMultLesCurrentLes==m_iNumOfLesions+2)
                        m_iMultLesCurrentLes=m_iNumOfLesions+1;

```

```

                                if(m_iMultLesCurrentLes>1)
m_iMultLesCurrentLes--;
                                break;
                                case 3: //HYPEREMIC. point, go to
same point DISTAL
                                m_NstateCount--;

                                m_cwKnobSelectMeasure.GetActivePointer().SetValuePairIndex(m_
NstateCount);

                                if(m_iMultLesCurrentLes==m_iNumOfLesions+2)
m_iMultLesCurrentLes=m_iNumOfLesions+1;
                                break;
                                case 4: //WEDGE. point, go to same
point HYPEREMIC
                                m_NstateCount--;

                                m_cwKnobSelectMeasure.GetActivePointer().SetValuePairIndex(m_
NstateCount);

                                break;
                                }
                                HWND hw=m_pcMultLesScreen-
>GetSafeHwnd();

                                ::SendMessage(hw,WM_USERMSG_MULTLES_GOTO_MESPNT,
m_iMultLesCurrentLes, m_NstateCount);
                                }

```

```

    }
    else //single lesion:
    {
        OnPrevNextState(0);
        PlotCurentSignal(m_NstateCount);
    }
    //////////////////////////////////////
    break;

```

463. Barak Lit. Decl., Ex. 1134, Ex. 19 (corresponding to Ex. 1142) at \cfrmeasure\cfrmeasuredlg.cpp at 1439.

464. Additionally, the following portion of SmartFlow software source code in the same processing component, OnTimer, "repaints" the screen after the data has been received, providing another example of rendering.

```

case 15: //Repaint
{
    //AfxMessageBox(" ");
    OnPaint() ;
    //UpdateWindow();
    break;

```

465. Barak Lit. Decl., Ex. 1134, Ex. 19 (corresponding to Ex. 1142) at \cfrmeasure\cfrmeasuredlg.cpp at 1591 (emphasis added).

466. The measurement processing components also rendered or facilitated rendering the data by processing it into an archival format for saving patient-specific data to disk. The files named casedata.cpp, lesiondata.cpp, patientdata.cpp and saveoperationdlg.cpp performed this type of rendering.

467. Accordingly, the SmartFlow system contains multiple processing components that facilitate receiving data of particular sensor types and facilitate rendering diagnostic parameter values according to that received data.

468. The SmartFlow system included a multi-mode graphical user interface host comprising diagnostic measurement user interfaces including display components corresponding to data output rendered by specified ones of the plurality of measurement processing components.

469. The SmartFlow system had a human-computer interface with multiple distinct displays, such as, for example, various system and diagnostic displays. As stated above, the SmartFlow system had multiple distinct displays, including a Setup mode display for entry of patient information, adjusting parameters, and registering the ID of the pressure wire being used.

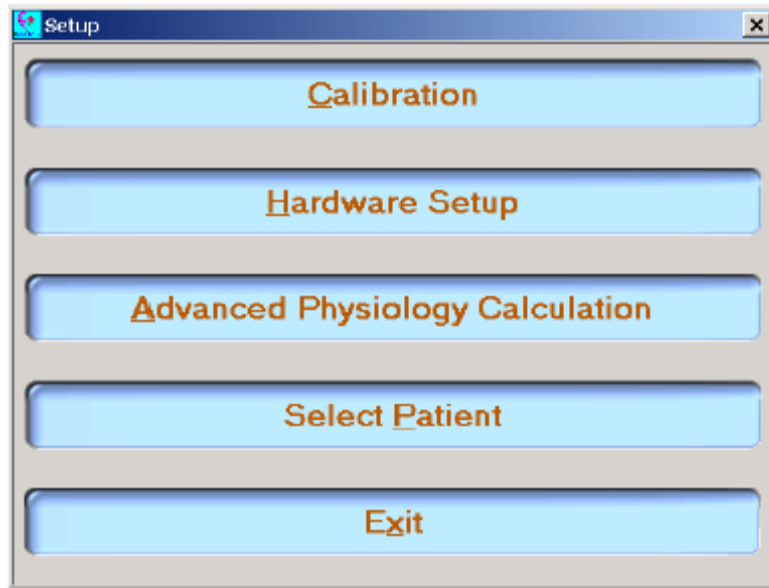


Fig 6: SetUp Screen options

Barak Lit. Decl., Ex. 1134, ¶ 8 & Ex. 8 (corresponding to Ex. 1027) at SJM03072846-50.

470. The Measure mode display in the SmartFlow system contained multiple graphs and separate soft-keys for proximal, distal, vasodilation, and coronary wedge pressure measurements.

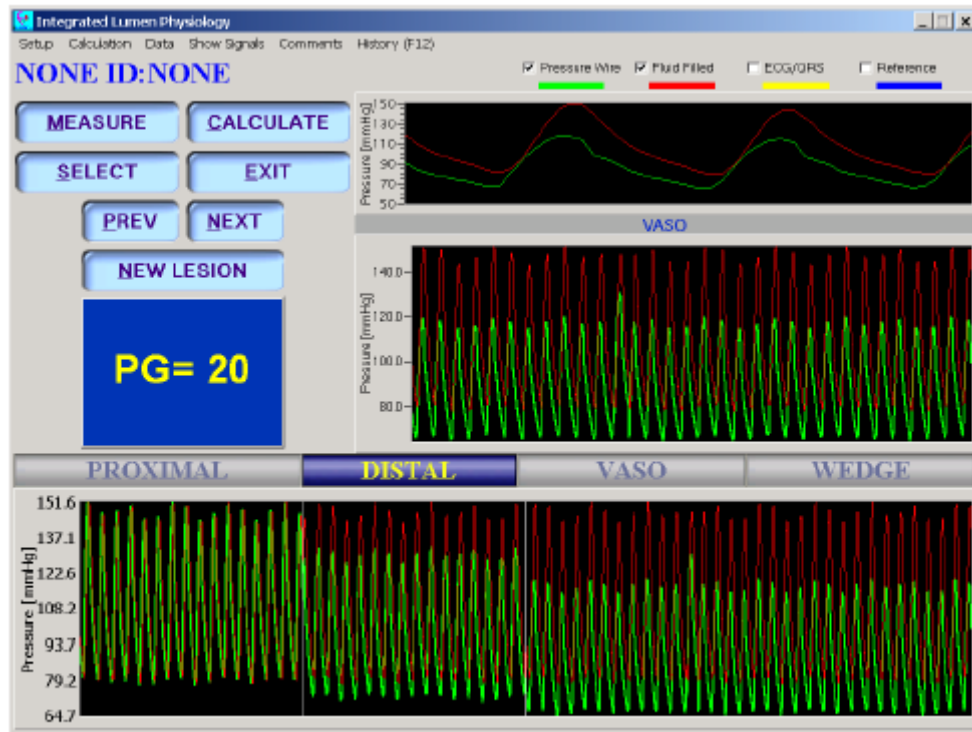


Fig 15: The measure mode window.

Barak Lit. Decl., Ex. 1134, ¶ 8 & Ex. 8 (corresponding to Ex. 1027) at SJM03072853.

471. The CFR/FFR plot display in the SmartFlow system showed CFR and FFR values per heart beat.

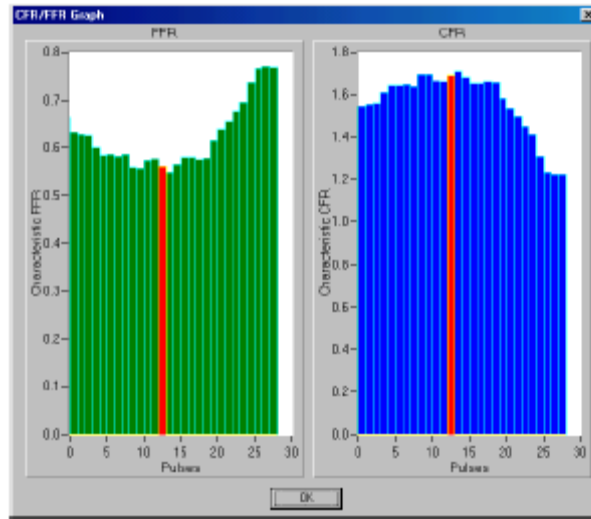
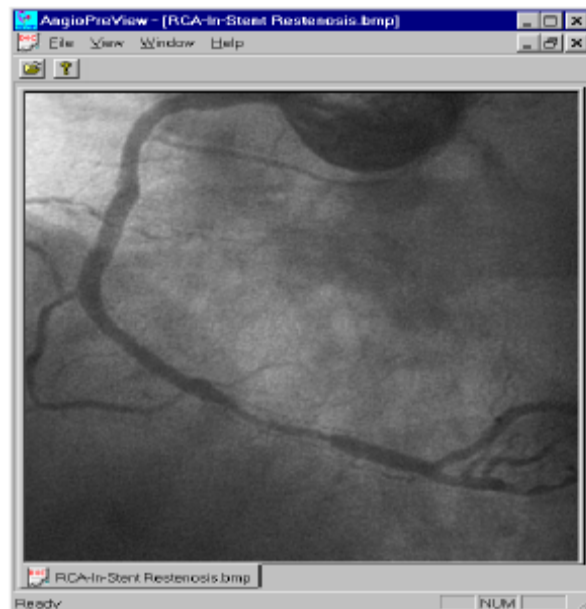


Fig 20: A graphic presentation of the CFR and FFR values per beat.

Barak Lit. Decl., Ex. 1134, ¶ 8 & Ex. 8 (corresponding to Ex. 1027) at SJM03072856.

472. Additionally, the angio picture preview in the SmartFlow system displayed photographic images.



Barak Lit. Decl., Ex. 1134, ¶ 8 & Ex. 8 (corresponding to Ex. 1027) at SJM03072862.

473. The multi-mode graphical interface host in the SmartFlow system thus included diagnostic measurement user interfaces, as depicted in the preceding images. The diagnostic measurement user interfaces in the SmartFlow system included display components corresponding to data output rendered by specified ones of the plurality of measurement processing components. For example, in the file named `cfrmeasuredlg.cpp`, there was a display component called `PlotCollectedCurrentData`, which displayed data that had been collected.

BarakDecl. Ex. 19 (Ex. 1142) at

`\SmartFlow5.1.0.9\CFRmeasure\cfrmeasuredlg.cpp\` at line 4839. Additionally, in the file named `MultlesionscreenDLG.cpp`, there was a display component called `DrawLesions`, which rendered lesion data in the graphical user interface.

BarakDecl. Ex. 19 (Ex. 1142) at

`\SmartFlow5.1.0.9\CFRmeasure\MultlesionscreenDLG.cpp\` at line 232.

474. The term "multi-mode graphical user interface host" means "software that supports a graphical user interface (including windows or portions of windows) for presenting at least two types of diagnostic information (including within the same window)".

475. For example the SmartFlow system code contains numerous "user interface mode objects." The source code attached to the Barak Lit. Decl., Ex. 1134, (Ex. 1134) as Ex. 19 (corresponding to Ex. 1142) consists of 320 distinct

files, most of which represent distinct "user interface mode objects" such as buttons, display axes, labels, graphical plot information, multi-lesion display information, information save operations, signal information, and a web browser interface control. Barak Lit. Decl., Ex. 1134, Ex. 19 (Ex. 1142) at SmartFlow5.1.0.9\CFRmeasure. Additionally, the SmartFlow system had multiple distinct displays for presenting diagnostic parameter values of multiple types, as shown in paragraphs 469-472, above.

476. The SmartFlow system included one or more kernel mode drivers that extract processed sensor data from a peripheral interface card providing a hardware interface for one or more invasive diagnostic measurement devices.

477. The SmartFlow system used one or more invasive diagnostic measurement devices as the basis of its sensor data. "The SmartFlow™ Intravascular Processor provides the physician with pressure derived lesion length and location, using a simple pressure wire pullback across the lesion of interest." Barak Lit. Decl., Ex. 1134, Ex. 8 (corresponding to Ex. 1027) at SJM03072834.

478. The SmartFlow system then processed the sensor data from the invasive diagnostic measurement devices. Signals from invasive diagnostic sensors (such as the Radi PressureWire™) were processed by the SmartFlow's Signal Conditioning Unit (SCU) with gain and bandwidth filtering. Specifically, the owner's manual for the SmartFlow system shows that the SCU provided "Gain

– 100" and "Bandwidth – 2000 Hz" for the "Direct Pressure Inputs." Barak Lit. Decl., Ex. 1134, Ex. 8 (corresponding to Ex. 1027) at SJM03072842.

479. The SmartFlow system also had a peripheral interface card, which provided a hardware interface for one or more invasive diagnostic measurement devices. Specifically, it used a PCI-6023E card manufactured by National Instruments, Inc ("National Instruments"). Barak Lit. Decl., Ex. 1134, ¶ 7 & Ex. 5 (corresponding to Ex. 1137) at SJM03073109 (Ex 1028).

480. The SmartFlow system also included a kernel mode driver that extracted the processed sensor data from the PCI card. The source code attached to the Barak Lit. Decl., Ex. 1134, as Ex. 19 (Ex. 1142) included the interface to the kernel mode drivers provided by National Instruments for use in interfacing with the PCI card. *See Id.* at \SmartFlow5.1.0.9\NI_DAQ. I acquired the driver that was provided by National Instruments for the PCI card Florence Medical used in the SmartFlow in the relevant timeframe, which I determined was NIDAQNT.sys. I acquired the NIDAQNT.sys driver update package version 4.9.0, which had a modified date of September 17, 1996. I also acquired the complete 6.9.1 driver distribution package, which contains a later version of the NIDAQNT.sys driver, along with appropriate documentation, user mode interface components that corresponded with specific versions of the drivers, and other accessories. The modified date for the 6.9.1 driver distribution package was April 30, 2001. I used

each of these drivers as the basis of my analysis because they were available in the relevant timeframe (*i.e.*, prior to May 20, 2002). A copy of both the NIDAQNT.sys driver version 4.9.0 and the complete 6.9.1 driver distribution package are provided as (Exhibit 1065).

481. I determined that the NIDAQNT.sys driver used direct memory access ("DMA") to extract information from the PCI-6023E PCI card for the main SmartFlow system application. The use of direct memory access is indicated by both the use of IoAllocateAdapterChannel, which was a key Windows NT function used to initiate DMA, and IoFreeAdapterChannel, which was a key Windows NT function used for completing DMA. These functions within Windows NT were used to extract data from a hardware device, such as the PCI-6023E PCI card that was used by Florence Medical on the SmartFlow system. Additionally, the data sheet provided for the PCI-6023E PCI card from National Instruments confirms, in discussing the output rates, that the card used DMA for data transfers:

¹ 10 kS/s typical when using the single DMA channel for analog output. 1 kS/s maximum when using the single DMA channel for either analog input or counter/timer operations. 1 kS/s maximum for PCMCIA DAQCard devices in all cases.

482. National Instruments, Inc., *Low-Cost E Series Multifunction DAQ – 12 or 16-Bi, 200 kS/s, 16 Analog Inputs, (available at*

http://www.ni.com/pdf/products/us/4daqsc202-204_ETC_212-213.pdf) (*last accessed April 17, 2013*) (Ex. 1074).

483. Because the card itself used DMA for data transfer, the supporting device driver necessarily used DMA for data extraction.

484. I determined that the NIDAQNT.sys driver used in the SmartFlow system executed in kernel mode because it imported functions from the Windows kernel and hardware abstraction layer, such as ExAllocatePoolWithTag, KeInsertQueueDpc, and KfRaiseIrql. Accordingly, the NIDAQNT.sys driver was a kernel mode driver that extracted information from the PCI-6023E PCI Card for the main user application.

485. In the SmartFlow system, sensor data from the pressure sensing guidewires (such as the Radi PressureWire™) entered the SCU, was processed in the SCU, entered the SmartFlow PCI (peripheral interface) card, which provided a hardware interface for one or more invasive diagnostic measurement devices, and was then extracted from the PCI card into the CPU by a kernel mode driver. Thus, every limitation of claim 1 of the '994 patent was present in the SmartFlow system.

486. The term "kernel mode drivers" means "software components that execute on the most privileged level in a computer, which permit a computer system to communicate with a device" The term "processed sensor data" means "sensor data that has been systematically modified, including by means of analog

to digital conversion". A "peripheral interface card" is "a printed circuit board or adapter that can be plugged into a computer to provide added functionality or new capability, which provides specialized services that are not built into the computer, and that interfaces with peripherals."

487. The SmartFlow system practiced these limitations of claim 1. For example, the SmartFlow system applied appropriate mathematical operations to the sensor data before it was extracted by the kernel mode drivers. The SmartFlow PCI card accepted analog signals from pressure sensors in the form of changing voltage values. This type of signal, in which a change in amplitude is reflected through a change in voltage, is referred to as an "analog" signal. The signals then underwent a process called "analog to digital conversion," or "digital sampling." As explained in one text on digital signal processing:

488. 5.2 Analog-to-Digital Conversion

489. One of the most important ways of producing a sequence is by sampling a continuous-time or analog waveform. An analog-to-digital (A/D) converter is a device that operates on the analog waveform to produce a digital output consisting of a sequence of numbers each of which approximates a corresponding sample of the input waveform. Figure 5. 1 shows a simple

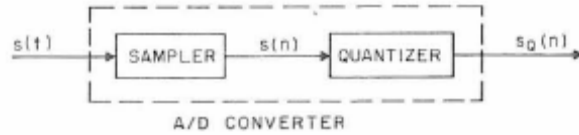


Fig. 5.1 Block diagram of A/D converter.

490. block diagram for the A/D converter that is conceptually represented as a two-stage process. In the first stage the sequence $s(n) = s(t)|_{t=nT}$ is created where $s(n)$ is expressed to infinite precision. In the second stage the numerical equivalent of each sample of $s(n)$ is expressed by a finite number of bits giving the sequence $s_Q(n)$. In actual A/D converters there are not two separate stages but instead a single stage with input $s(t)$ and output $s_Q(n)$. The difference signal $e(n) = s(n) - s_Q(n)$ is called quantizing noise or A/D conversion noise.

491. As discussed in Chapter 2, the analog signal $s(t)$ must be bandlimited in order for the quantized output $s_Q(n)$ to be a meaningful representation of $s(t)$. Thus generally a presampling analog lowpass filter precedes the A/D converter. The desirable characteristics of this analog filter are at least 40-dB loss for frequencies greater than half the sampling frequency and as little in-band ripple as possible in the signal baseband. Practical experience has shown that eighth-order (48-dB/octave slopes) lowpass filters are adequate for most speech processing applications.

492. Depending on the detailed way in which $s(n)$ is quantized, different distributions of quantization noise may be obtained. If the smallest quantization

step used in the digital representation of $s_Q(n)$ is called Q , then the relation between $s_Q(n)$ and $s(n)$ for *rounding* is given in Fig. 5.2. Since there

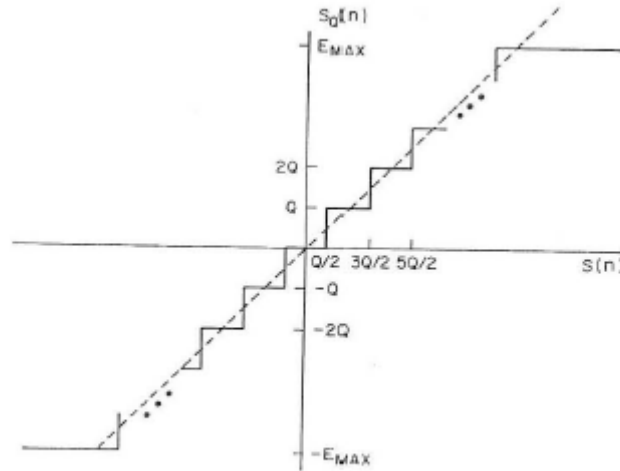


Fig. 5.2 Quantizer characteristic with rounding.

are only a finite number of quantization levels, all signals either exceeding the largest ($E_{\max}$) or falling below the smallest ($-E_{\max}$) quantization level are rounded to these numbers. Generally such overflows or underflows are avoided by judicious choice of the quantization step Q and by careful scaling of the analog input waveform. With these exceptions, it is clear from Fig. 5.2 that the error signal satisfies the relation

$$-\frac{Q}{2} \leq e(n) \leq \frac{Q}{2}$$

for all n . Under certain not overly restrictive assumptions it can be shown that the distribution of the error signal is uniform. Thus the probability distribution of the quantization error for rounding is as shown in Fig. 5.3.

493. To illustrate the process of rounding in an A/D converter, Fig. 5.4 shows an analog waveform $s(t)$ and the resulting digital samples $s_Q(n)$ that are obtained. At the bottom of Fig. 5.4, the quantization error (on a magnified scale) is shown for each of the samples. This quantization error resembles a random sequence as predicted by the argument above.

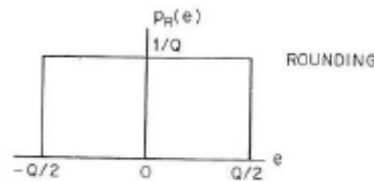


Fig. 5.3 Probability density function for roundoff error.

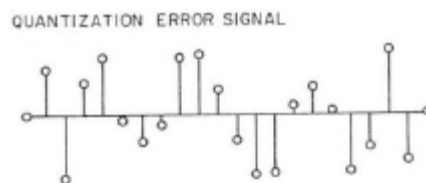
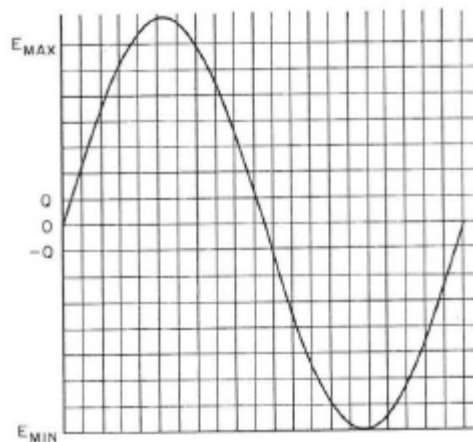


Fig. 5.4 Quantization error for a sine wave.

* * *

494. The importance of understanding A/D conversion and gaining insight into the distribution of the A/D conversion noise is that one can represent the

quantized input to a digital system [with impulse response $h(n)$] as the infinite precision input $[s(n)]$ plus the quantization error $[e(n)]$ as shown in Fig. 5.7. Based on linearity, one can process the sequences $s(n)$ and $e(n)$ independently. Thus the output sequence (assuming infinite precision processing) can be represented as

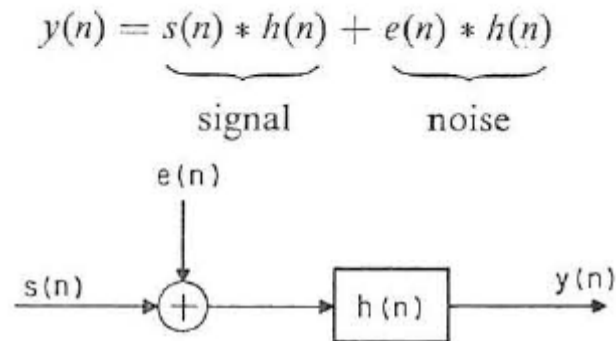


Fig. 5.7 Linear model of quantization noise in an LTI system.

495. In this manner one can define a signal and a noise output of the filter and thus obtain a measure of signal-to-noise ratio of the processed sequence. Of course the processing in the digital filter is performed with finite precision; thus the ideas above must be modified somewhat to account for this effect. However, the idea of representing a quantized signal as the sum of an unquantized signal plus a noise term is basic to most of the theory of finite word length effects.

496. Before proceeding to a discussion of the various types of arithmetic that are used in digital filter realizations, it is worthwhile emphasizing a small but important point about analog-to-digital conversion noise. All analog signals are corrupted by some form of noise—i.e., they have a finite signal-to-noise ratio. For example, a speech signal transmitted over an ordinary telephone line has a signal-

to-noise ratio of about 36 dB. Thus the use of a large number of bits to represent such a signal does *not* give a digital signal with a higher signal-to-noise ratio than the analog signal. Hence even though the quantization step size Q may be very small relative to the peak signal, the lower-order bits of the sequence are merely providing a good representation of the noise inherent in the analog signal. This implies that, depending on the exact signal, increasing the number of bits in the A/D converter beyond a certain point merely increases the accuracy by which an analog noise is represented. Thus the word length of an A/D converter is dictated by the signals being converted. Lawrence Rabiner & Bernard Gold, *THEORY AND APPLICATION OF DIGITAL SIGNAL PROCESSING*, 296-300 (1975) (Ex. 1073).

497. In other words, the process of quantizing, used in digital sampling, means taking the value of a voltage level and changing the signal by *rounding* it to the nearest pre-assigned numerical voltage level. Accordingly, the definition of the term "quantizing" describes a mathematical operation:

quan•tize (kwon'tiz), v.t., **-tized**, **-tiz-ing**. **1.** *Math., Physics.* to restrict (a variable quantity) to discrete values rather than to a continuous set of values. . . .

THE RANDOM HOUSE DICTIONARY OF THE ENGLISH LANGUAGE, 1579
(2d ed. Unabridged 1987) (emphasis in original) (Ex 1052).

quan•tize (-tiz) vt, **-tized**, **-tiz-ing** [QUANT(UM) + -IZE] **1.** *Math.* to express in multiples of a basic unit . . .

WEBSTER'S NEW WORLD DICTIONARY, 1162 (2d College Ed. 1984)
(emphasis in original). (Ex. 1072).

498. Similarly, rounding is a mathematical operation. For example
rounding is:

round•ing (round'ing) . . . **6. Math.** **a.** the process of replacing a number by another number of approximately the same value but having fewer digits: *To the nearest dollar, the rounding of \$27.68 yields \$28.* **b.** a similar process that specifies one of various rules. Generally, the number is first truncated to one or two digits more than is desired; then the last one or two digits are adjusted in a specified way in order to reflect the magnitude of the original number. In rounding the final digits, 0-4 are simply dropped, 6-9 are dropped after the preceding digit is increased by 1, and 5 is handled in various ways depending on the surrounding digits and the particular convention being followed. Cf. **truncate** (def. 2). [1545-55; ROUND¹ + -ING², -ING¹]

THE RANDOM HOUSE DICTIONARY OF THE ENGLISH LANGUAGE, 1676
(2d ed. Unabridged 1987) (emphasis in original) (Ex. 1052).

rounding [MATH] Dropping or neglecting decimals after some significant place. Also known as truncation. { raúnd•in }

MCGRAW-HILL DICTIONARY OF SCIENTIFIC AND TECHNICAL TERMS,
1835 (6th ed. 2003) (emphasis in original) (Ex. 1051).

499. The '994 patent itself describes the process of analog to digital conversion (digital sampling) as a form of "processing." For example, the '994 patent describes one embodiment of the claimed invention, and states that "[e]xemplary *processes* performed by the DSP include: A/D and D/A conversions . . ." See col. 5 ll. 34-36 (emphasis added) (Ex 1001). I understand "A/D conversion" to refer to the process of analog to digital conversion, or quantizing.

500. As stated, the analog sensor data signals entered the SmartFlow system, were processed in the SCU using analog processing, and then were digitally sampled on the PCI card using an analog to digital converter that applied the mathematical operation of quantizing. Accordingly, the sensor data in the SmartFlow system was "processed" before it was sent to the peripheral interface card. Accordingly, the kernel mode drivers in the SmartFlow CPU extracted processed sensor data from the PCI card.

501. The SmartFlow system also had code that facilitated communicating commands and data between interface hardware and a set of one or more user mode processes, as described above. That driver code extracted the processed sensor data from the PCI card.

502. Therefore, in SmartFlow system, sensor data from the pressure sensing guidewires entered the SCU, was processed in the SCU, entered the PCI (peripheral interface) card, which provided a hardware interface for the pressure

sensing guidewires, was further processed in the PCI card, and then was extracted from the PCI card into the CPU by a kernel mode driver. Thus, the SmartFlow system practiced each limitation of claim 1 of the '994 patent.

503. The SmartFlow system qualifies as prior art both under 35 U.S.C. §§ 102(a) & (b).

504. I understand from my discussions with counsel that a device is prior art under 35 U.S.C. § 102(b) if it was in public use or on sale in the United States more than one year prior to the date of the application for patent. Accordingly, a device would be prior art under § 102(b) if it practiced each limitation of claim 1 of the '994 patent and was in public use or on sale in the United States prior to May 20, 2001.

505. I also understand from my discussions with counsel that a device is prior art under 35 U.S.C. § 102(a) if it practiced each limitation of the patent and was known or used by others in the United States or was patented or described in a printed publication in the United States or a foreign country before the invention was made.

506. The SmartFlow system was in public use and was on sale in the United States prior to May 20, 2001. It was also known and used by others in the United States prior to May 20, 2002. For example, during pre-clinical evaluations, various U.S.-based doctors operated the SmartFlow system in test situations,

including Morton Kern, M.D., then of St. Louis University, Amir Lerman, of the Mayo Clinic, Martin Leon, M.D., then of Lenox Hill Heart & Vascular Institute in New York, Christopher White, M.D., of Ochsner Clinic in Louisiana, Dr. Kenny Rosenfield, currently of Massachusetts General Hospital, and Marvin Slepian, M.D., of the University of Arizona. In compliance with regulations from the United States Food and Drug Administration ("FDA"), Florence Medical also supplied some of these and other doctors with SmartFlow system systems prior to FDA clearance to sell the SmartFlow system in the United States. Martin Decl., Ex. 1058, ¶ 4; Barak Lit. Decl., Ex. 1134, ¶ 10. These pre-clinical evaluations occurred prior to FDA 510(k) clearance for the SmartFlow system, which was received May 14, 2001. Martin Decl., Ex. 1058, ¶ 6 & Ex. 5 (corresponding to Ex. 1102)

507. In October 1999, Florence Medical displayed and demonstrated the SmartFlow system in the booth Endosonics (a predecessor of Volcano) had at the Transcatheter Cardiovascular Therapeutics ("TCT") congress in Washington D.C. Barak Lit. Decl., Ex. 1134, ¶ 12. At that congress, Florence Medical also demonstrated the SmartFlow system in a "Live Case" demonstration, during which a SmartFlow system was off-site at a hospital in use by physicians, and the procedure was projected onto screens in real time into a conference room at the convention hall. Barak Lit. Decl., Ex. 1134, ¶ 12.

508. The following year, in October 2000, Florence Medical had a booth at the TCT congress, in which it displayed and demonstrated the SmartFlow system. Martin Decl., Ex. 1058, ¶ 5; & Ex. 4; Barak Lit. Decl., Ex. 1134, ¶ 13.

509. As of 2001, Florence Medical had obtained the domain name www.florencemedical.com and maintained a web page at that site, where it advertised the SmartFlow system. Martin Decl., Ex. 1058, ¶ 7.

510. Following FDA clearance in mid-May 2001, Florence Medical engaged in a marketing campaign to promote SmartFlow system in the United States. To that end, Florence Medical developed advertising materials, including brochures, a clinical package, the above-referenced data sheet, and case presentations, and mailed them to doctors in the United States. Martin Decl., Ex. 1058, ¶ 7 & Exs. 6 & 7 (corresponding to Exs. 1103-1104).

511. Additionally, both before and after clearance by the FDA, Florence Medical took the SmartFlow system and the above-referenced marketing materials to several trade shows, where its employees and officers met with doctors, researchers, and others in the industry. Martin Decl., Ex. 1058, ¶ 8. At trade shows, Florence Medical also frequently displayed a poster or posters in the poster section of the convention hall. *Id.* & Ex. 8. Florence Medical also met with doctors to discuss the SmartFlow system, at which it offered the SmartFlow system for sale. At each of the tradeshow that Florence Medical attended, its employees

and officers met with U.S.-based doctors and others in the industry for this purpose.¹²

512. For example, Mr. Martin attended the American College of Cardiology ("ACC") conference in Orlando, Florida from March 17-21, 2001. At that conference, he met with Dr. Kern, Florence Sheehan, M.D. of the University of Washington Medical School, and Matt Prucka of GE Medical. He discussed the SmartFlow system with each of them. Martin Decl., Ex. 1058, ¶ 9.

513. Florence Medical also had a booth at the EuroPCR (Paris Course of Revascularization) conference in Paris in May 2001. Martin Decl., Ex. 1058, ¶ 10 & Ex. 9 (corresponding to Ex. 1106); Barak Lit. Decl., Ex. 1134, ¶ 18. While in Paris for the EuroPCR conference, Mr. Martin again met with doctors and others from the United States, including representatives from General Electric, Siemens, and Boston Scientific, to discuss the SmartFlow system system. Martin Decl., Ex. 1058, ¶ 10.

514. From June 4-7, 2001, Florence Medical attended the Advanced Endovascular conference in New York City, at which Mr. Martin met with

¹² As noted in paragraph 507, during one scientific meeting, Florence Medical exhibited in the booth of Endosonics, a U.S.-based company, and predecessor to Volcano. Barak Lit. Decl., Ex. 1134, ¶ 4.

multiple doctors from the U.S. in an attempt to sell a SmartFlow system, including Dr. Marco Costa from the University of Miami and Elezar Edelmann, M.D. from Harvard Medical School. Martin Decl., Ex. 1058, ¶ 11.

515. In September 2001, Florence Medical attended the European Society of Cardiology ("ESC") convention in Stockholm, Sweden. Martin Decl., Ex. 1058, ¶ 12. Also in September 2001, Dr. Barak from Florence Medical met with U.S. doctors Chris White and Kenny Rosenfield and discussed the SmartFlow system with them in Washington D.C. while setting up for the TCT conference, which was later cancelled because of the events of September 11, 2001. Barak Lit. Decl., Ex. 1134, ¶ 19.

516. In October 2001, Mr. Martin presented Florence Medical at the annual meeting of Massachusetts Medical Device Industry Council ("MassMEDIC") and various venture capitalists and corporate development officers of many of the larger medical device manufacturers. At the MassMEDIC meeting, Mr. Martin handed out brochures for the SmartFlow system. Martin Decl., Ex. 1058, ¶ 13.

517. Also in October 2001, Florence Medical received 510(k) clearance from the FDA to sell SmartFlow Multiple Lesion in the United States. SmartFlow Multiple Lesion was based on the same SmartFlow device and architecture, but provided the additional ability to diagnose multiple lesions in a coronary artery. Martin Decl., Ex. 1058, ¶ 14 & Ex. 10 (corresponding to Ex. 1107).

518. In November 2001, Florence Medical attended the American Heart Association meeting in Anaheim, California and promoted the SmartFlow system. Martin Decl., Ex. 1058, ¶ 15.

519. In February 2002, Florence Medical attended the Joint Interventional Meeting ("JIM") conference in Rome, where Mr. Martin again met with U.S.-based doctors and representatives of U.S.-based medical device manufacturers. Martin Decl., Ex. 1058, ¶ 16; Barak Lit. Decl., Ex. 1134, ¶ 21(Ex 1028). At the JIM conference, Florence Medical also participated in a "live case" demonstration, in which the SmartFlow system was demonstrated live on computer monitors in a convention hall. Martin Decl., Ex. 1058, ¶ 16.

520. In March 2002, Florence Medical had booth #1661 at the ACC conference in Atlanta where it marketed the SmartFlow system. Martin Decl. Ex. 1058, ¶ 17. In advance of the 2002 ACC conference, Florence Medical issued a press release announcing the introduction of the SmartFlow Multiple Lesion. *Id.* & Exs. 11 & 12 (corresponding to Exs. 1108-1109). Also in March 2002, Dr. Martin Leon, then of Lenox Hill Hospital in New York, indicated that he had used the SmartFlow Multiple Lesion device, stating, "In our hands, the SmartFlow Multiple Lesion device enabled us to quickly identify the offending stenotic lesion in the presence of multiple lesions." *Id.*

521. It was the customary business practice of Mr. Martin, the CEO of Florence Medical, and Dr. Chen Barak, the Vice President of Operations, to market the SmartFlow system at all industry events that they attended by handing out brochures, clinical packages, and having meetings with U.S.-based doctors. Martin Decl., Ex. 1058, ¶ 18; Barak Lit. Decl., Ex. 1134, ¶ 22. Florence Medical also sent mailings containing marketing materials, including an announcement of its attendance at the conferences, to select U.S.-based doctors taken from the roster of attendees in advance of the conferences. Martin Decl., Ex. 1058, ¶ 18.

522. Moreover, the SmartFlow system was in public use no later than May 8, 2001.

523. Accordingly, the SmartFlow system was in public use or on sale in the United States before May 20, 2001, and was known or used by others in the United States prior to May 20, 2002. Thus, the SmartFlow system was prior art under both 35 U.S.C. §§ 102(a) & (b). Because it practiced each limitation in claim 1 of the '994 patent, claim 1 of the '994 patent is invalid as anticipated under 35 U.S.C. § 102.

C. EP-WorkMate system

524. The EP-MedSystems EP-WorkMate system version 3.0 anticipated each limitation of claim 1 of the '994 patent.

525. EP Medical was founded in March 1993 as a manufacturer of medical devices for use in the cardiac electrophysiology marketplace. Declaration of Charles Bryan Byrd (the "Byrd Decl.") ¶ 2. I understand that electrophysiology deals with electrical functions of the heart. In 1994, EP Medical acquired a product called the EP-Easy from a think tank based at the University of Michigan. *Id.* EP Medical renamed the product the EP-WorkMate and, in 1996, changed its company name to EP MedSystems, Inc. ("EP MedSystems"). *Id.* In October 1995, Bryan Byrd of EP MedSystems installed the first EP-WorkMate system at Rush North Shore Hospital in Chicago. *Id.* ¶ 11. In May 2000, at the 21st Annual Scientific Sessions held by the North American Society of Pacing and Electrophysiology in Washington D.C., EP MedSystems launched version 3.0 of the EP-WorkMate system running Microsoft's Windows NT operating system. *Id.* & Exs. B & C (corresponding to Exs. 1117-1118).

526. As described in the User's Guide for version 3.0 of the EP-WorkMate system:

EP-WorkMate is a fully computerized system for capturing and measuring physiological data in the clinical electrophysiology laboratory. It provides digital signal acquisition and display of those electrical signals on high resolution monitors.

An Electrophysiology system is used to monitor and analyze surface ECGs and intra-cardiac signals. Catheters are guided into the heart

and intra-cardiac electrical signals are acquired from the electrodes on the indwelling catheters from various heart locations. These signals are then used to measure irregular cardiac electrical activity that may occur in the heart. A cardiac stimulator is used to send an electrical pulse via a catheter to the heart. These electrical pulses are called pacing protocols. Pacing protocols are performed in order to reproduce any life threatening arrhythmias under the safest possible conditions for the patient. Treatment of an arrhythmia is then initiated with radio frequency ablations, drug therapy, or placement of cardiac devices that will control the heart's rhythm.

The system is also intended for recording electrocardiograms from multiple sites during clinical assessment of electrophysiological interventions. The EP-WorkMate provides on-line measurements of intra-channel intervals, and presents these measurements on the display monitor. Retrieval and display are provided without interruption of the multi-channel acquisition process. An optical disk drive (MO) or digital audio tape (DAT) is used for archival storage of digitized data.

The standard hardware configuration consists of three 21" high resolution display monitors, a Pentium™ processor, 2 Gbyte hard disk drive, 1.44 Mbyte floppy disk drive (3.5"), 2.6- 5.2 Gbyte Optical (MO) drive or 4 Gbyte Digital Audio Tape (DAT) tape drive, TI 34020 Advanced Graphics Coprocessor, 76 channel analog-to-digital (A/D) signal conditioning data acquisition module (56 intra-cardiac) with integrated pressure/marker interface, catheter interface box, EP-3 integrated stimulator, and HP LaserJet 6 series printer. Available

options include: 24 intra-cardiac channel signal conditioning unit (44 total), 120 intra-cardiac channel signal conditioning unit (140 total), 32 channel strip chart recorder, cine capture, and additional slave monitors.

Menu driven software is incorporated for data acquisition, analysis, interval posting, instant retrieval with waveform markers and intervals displayed, marker editing capability, and personal form generation. EP-WorkMate has an integrated patient data base system for rapid access to all patient records. Unique features of the system include: On-line interval measurement, two channel integrated stimulator, a choice of scrolling or wiper-mode display during real-time signal acquisition, a comprehensive database with a query function, integrated RF graphics, and a user configured report editor with fax capabilities.

527. Byrd Decl., Ex. 1067, Ex. F (corresponding to Ex. 1121) at SJM03074956-57 (emphasis added). In this section of my report, I explain how the Windows version of the EP-WorkMate system (version 3.0) anticipated each limitation of claim 1 of the '994 patent. When I refer to the EP-WorkMate system, I am referring to version 3.0 unless otherwise noted.

528. The EP-WorkMate system was a multipurpose host system for invasive cardiovascular diagnostic measurement acquisition and display.

529. In essence, the EP-WorkMate system was a personal computer-based medical monitoring system that obtained invasive cardiovascular diagnostic

measurements and displayed them on the computer screen. As explained in the EP-WorkMate® User's Guide, it was a "a computer based electrocardiographic recording and monitoring system designed for efficient capture, display and retrieval of surface and intra-cardiac signals during cardiac electrophysiology studies." Byrd Decl., Ex. 1067, ¶ 3 & Ex. F (corresponding to Ex. 1121) at SJM03071652. Moreover, Mr. Byrd explained that the EP-WorkMate system used one or more invasive diagnostic measurement devices as the basis of its sensor data. Byrd Decl., Ex. 1067, ¶¶ 4, 8.



Byrd Decl., Ex. 1067, Ex. F (corresponding to Ex. 1121) at SJM03071653.

530. The EP-WorkMate system incorporated a component based arrangement.

531. The EP-WorkMate system included an arrangement of hardware components. As depicted in the preceding image, the EP-WorkMate system incorporated a computer module, a printer, a keyboard, an external Signal Conditioning Unit ("SCU"), and computer monitors.

532. The EP-WorkMate system also included a variety of software components. To determine this, I analyzed the software source code attached to the Byrd Decl., Ex. 1067, as Ex. Q (corresponding to Ex. 1132). It is clear from the source code that the EP-WorkMate system used the modular decomposition that is normal for software in the relevant time period. For example, the source code contains CHolterView, which is in the file named HolterView.cpp, CRfView, which is in the file named rfview.cpp, and CMonitorView, which is in the file named MonitorView.cpp, among numerous other software components contained within the approximately 263 C++ source code files that included the EP-WorkMate application. Byrd Decl., Ex. 1067, Ex.Q (corresponding to Ex. 1132).

533. The EP-WorkMate system included a component based arrangement. A "component based arrangement" means "software arrangement that is extensible, modular."

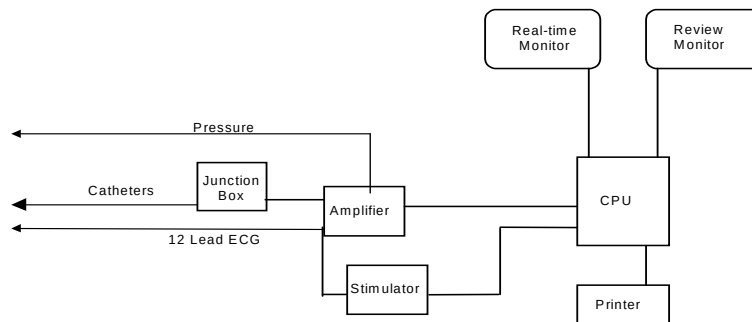
534. The software components in the EP-WorkMate system were extensible and modular. For example, the software in the EP-WorkMate system was primarily programmed using C++ because of the possibility of future growth provided by a modular, object oriented programming language. The main application software was designed to have this future growth capability by using the modularity provided for in C++. When EP MedSystems provided software upgrades, it provided the customer with a disk containing new software that replaced some or all of the previously existing software modules in the EP-WorkMate main application. Byrd Decl., Ex. 1067, ¶ 7.

535. For example, the EP-WorkMate software code is structured with 263 different files comprising the user mode application, so that individual files can be replaced or new components can be added by creating new files without the need to replace the other components.

536. The EP-WorkMate system included an external input signal bus interface for receiving data arising from cardiovascular diagnostic measurement sensors.

537. The EP-WorkMate system received data from cardiovascular diagnostic measurement sensors such as ECG cables, catheters and invasive pressure transducers. Byrd Decl., Ex. 1067, ¶ 4 & Ex. F (corresponding to 1121) at SJM03071655.

538. The EP-WorkMate system also had an external signal bus interface for receiving the cardiovascular diagnostic sensor data. For example, the ECG cables, catheters and pressure sensors were plugged into adaptor cables and junction boxes that plugged into the SCU. Byrd Decl., Ex. 1067, ¶ 4 & Ex. F (corresponding to Ex. 1121) at SJM03071655. The central processing unit ("CPU") had an Ethernet card in it, and the SCU sent the sensor data to the CPU by means of an Ethernet cable. *Id.* A wiring diagram from the EP-WorkMate system is shown as follows:



Byrd Decl., Ex. 1067, ¶ 4 & Ex. F (corresponding to Ex. 1121) at SJM03071654 (I understand that the "Amplifier" in the above diagram is the SCU).

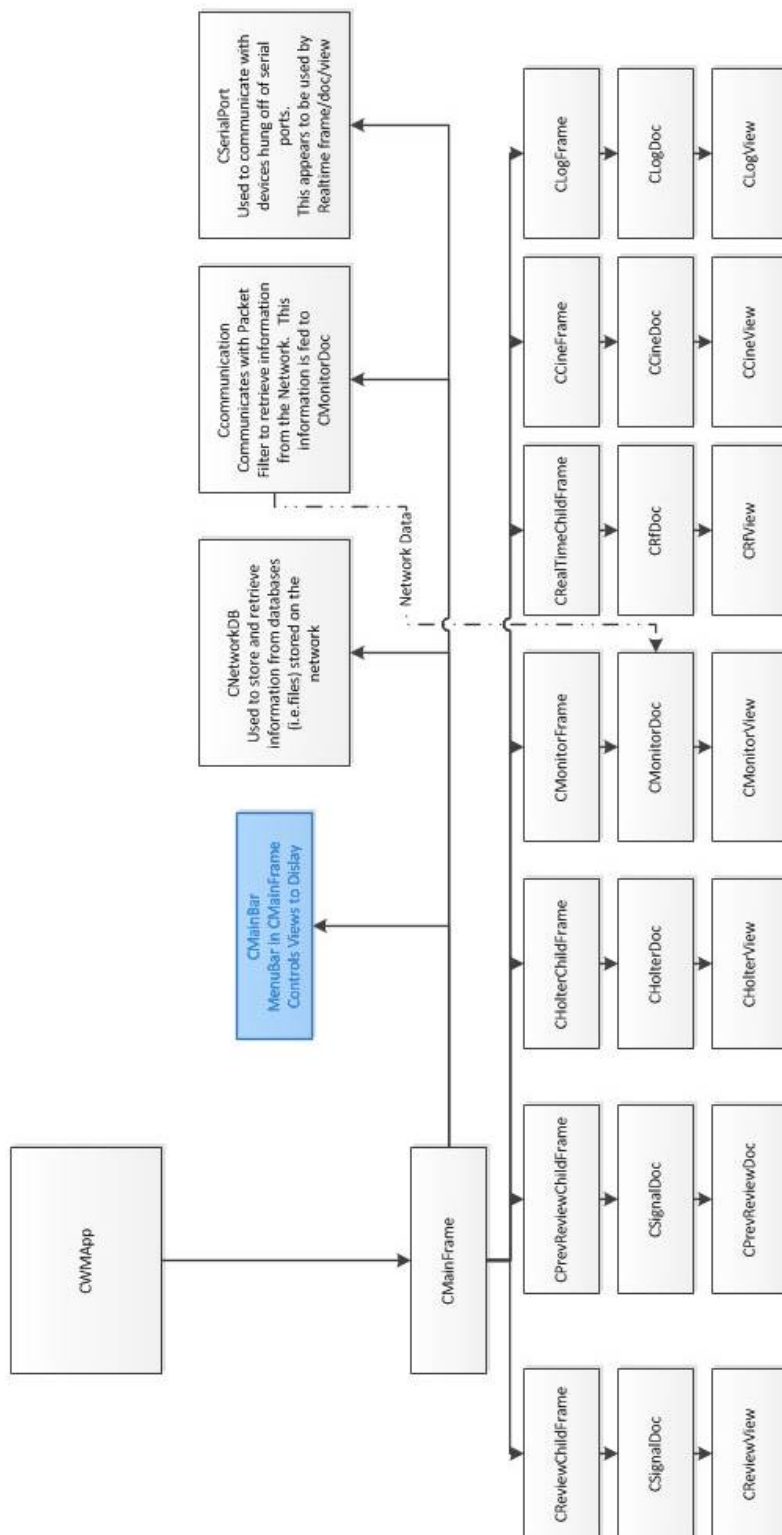
539. Accordingly, the EP-WorkMate system practiced this limitation of claim 1.

540. The EP-WorkMate system included a plurality of measurement processing components, that operate at a user mode level in the multipurpose host

system, for facilitating receiving data of particular sensor types and rendering diagnostic measurement parameter values according to the received data.

541. As noted, the EP-WorkMate software code was written in C++ and used the Microsoft Foundation Class, an object-oriented programming language and component-based software engineering framework frequently used for writing component-based software applications. Additionally, as I explained in Section II.E of this Report, a conventional way of using the modularity allowed by C++ and MFC programming is to utilize separate components for processing as opposed to display. In accordance with the MFC, the EP-WorkMate system does its data processing in certain software components, which are distinct from the software components used for display.

542. I prepared the following diagram based on my analysis of the EP-WorkMate software code attached to the Byrd Decl., Ex. 1067, as Ex. Q (corresponding to Ex. 1132). This diagram demonstrates many of the significant components (called "classes" in C++) that were in the EP-WorkMate user mode application:



543. Each of the boxes in this chart represents a different software component of the EP-WorkMate system. The arrows in the chart represent the inheritance relationship between the different components, so that components that are represented lower on the chart were derived from components that are connected higher on the chart. For example, a CReviewView component (in the lower left-hand corner) is a specific instance of a CSignalDoc component immediately above it.

544. The CMonitorDoc component received data from the CNetworkDB component and distributed the data to a plurality of measurement processing components based on the appropriate sensor type.

545. Each of those measurement processing components facilitated receiving data of particular sensor types. The measurement processing components then fed the data to display components that rendered the data. Accordingly, the measurement processing components facilitated receiving the data and facilitated rendering the data. The portion of code in the MonitorDoc.cpp file that distributed the sensor data according to sensor type is copied below:

```
// this function processes and moves the data from acquire buffer to  
the display buffer from the acquire_pop to the display_push
```

```
void CMonitorDoc::ProcessInputData(void)
```

```
{
```

```
short iValue=0;
```

```
USHORT usInterval=0;
```

```
SChannelConfig* pConfig = NULL;
```

```
if( m_usCalibrate)
```

```
{
```

```
    for (short channelN=0; channelN<MAX_SCREEN_CHANNELS;  
channelN++)
```

```
    {
```

```
        pConfig=&m_shSignalHeader.map.config[theApp.m_CathMapDoc.m  
_iRealTimePageNumber][channelN];
```

```
        if (pConfig->position)
```

```
        {
```

```
            // this is necessary to calibrate all channels, even if not being  
            displayed so they can be stored
```

```

        iValue=DecodeChannelCalibrate(pConfig-
>neg_offset, pConfig->pos_offset);

        UpdatePlotBuffer(channelN, iValue, 0);

        m_ariDisplayBuffer[m_usDisplayPush][channelN]=iValue;
//theApp.m_dMonitor.m_usAcquirePop Clarify ??

    }

}

--m_usCalibrate;

} else {

    for( short channelN=0;
channelN<MAX_SCREEN_CHANNELS; channelN++) {

        // now process the incoming data

```

```
pConfig =  
&m_shSignalHeader.map.config[theApp.m_CathMapDoc.m_iRealTimePageNumber][channelN];
```

```
if (pConfig->position) {
```

```
    iValue =DecodeChannel(pConfig->neg_offset,  
pConfig->pos_offset);
```

```
    USHORT interval=0;
```

```
    if (pConfig->analysis) {
```

```
        interval=  
PerformAnalysis(realtime_analysis[channelN], iValue);
```

```
        // Replace stim. channel with a clean narrow  
pulse
```

```
        if (pConfig->analysis==TRIG_STIM) {
```

```
        iValue=CLOGDoc::StimConvert(  
realtime_analysis[channelN], interval);
```

```
CheckAutoRecordStim(realtime_analysis[channelN], interval);  
  
    }
```

```
    if (interval) {
```

```
FirstAnalysisCheck(realtime_analysis[channelN], (short*)&interval);
```

```
    if (interval) {
```

```
        if (m_realtime_index.surface_screen==  
theApp.m_CathMapDoc.m_iRealTimePageNumber) {
```

```
            if  
(m_realtime_index.surface_config==channelN) {
```

```

UpdateTigaHR(interval);

    }

}

}

}

```

```

short filter_value=theApp.m_flFilter.FilterSample(
realtime_filter[channelN], iValue);

```

```

if( theApp.m_bDisplayIntervals) {

```

```

if (interval && interval<MAX_INTERVAL) {

```

```

// pressure is the only negative interval
allowed and is already offset by INTERVAL_OFFSET

```

```

if (pConfig->analysis==TRIG_PRE) {

```

```
UpdatePlotBuffer(channelN /* pConfig-  
>position-1 */ , filter_value, interval);
```

```
} else {
```

```
UpdatePlotBuffer(channelN /* pConfig-  
>position-1 */ , filter_value, ((realtime_analysis[channelN].thd->rate)  
? (60200U/interval) : interval)+INTERVAL_OFFSET); // the 60200  
is a round up factor
```

```
}
```

```
m_ariDisplayBuffer[m_usDisplayPush][channelN]=interval+ECG_V  
ALUE_LIMIT;
```

```
} else {
```

```
UpdatePlotBuffer(channelN /* pConfig-  
>position-1 */ , filter_value, 0);
```



```
m_ariDisplayBuffer[m_usDisplayPush][channelN]=filter_value;  
  
    }
```

```
    } else {
```

```
        UpdatePlotBuffer(channelN /* pConfig->  
position-1 */ , filter_value, 0);
```

```
m_ariDisplayBuffer[m_usDisplayPush][channelN]=filter_value;  
  
    }
```

```
    if  
(theApp.triggered_sweep==TRIG_SWEEP_LEVEL) {
```

```
        if (realtime_analysis[channelN].level)
```

```

        //AfxTrace("level %d,%d,%d - %d,%d\n",
triggered_sweep_start, channelN,
realtime_analysis[channelN].last_level,
theApp.m_uiMSecTimer,m_usAcquirePop);

```

```

CheckTriggerLevel(realtime_analysis[channelN], filter_value);

    }

```

```

        if (theApp.m_auto_record_mode &
realtime_analysis[channelN].auto_rec) {

```

```

            if
(realtime_analysis[channelN].CheckAutoRecord)

```

```

realtime_analysis[channelN].CheckAutoRecord(&realtime_analysis[c
hannelN], interval);

```

```

        }

    }

}

```

```
}
```

```
if( m_realtime_index.surface_screen==-1) {
```

```
//ClearTigaHR();
```

```
} else {
```

```
if(
```

```
m_realtime_index.surface_screen!=theApp.m_CathMapDoc.m_iReal  
TimePageNumber) { // dont need to do it if done above
```

```
pConfig=&m_shSignalHeader.map.config[m_realtime_index.surface_  
screen][m_realtime_index.surface_config];
```

```
iValue=DecodeChannel(pConfig->neg_offset, pConfig-  
>pos_offset);
```

```

        usInterval= PerformAnalysis(surface_analysis, iValue);

        if (usInterval) UpdateTigaHR(usInterval);

        if (theApp.m_auto_record_mode &
surface_analysis.auto_rec) {

CWMAApp::CheckAutoRecordRate(&surface_analysis, usInterval);

        }

    }

}

        if (m_realtime_index.stim_screen>-1 &&
m_realtime_index.stim_screen!=theApp.m_CathMapDoc.m_iRealTi
mePageNumber) { // dont need to do it if done above

        if ((theApp.m_auto_record_mode & stim_analysis.auto_rec)
|| (theApp.triggered_sweep==TRIG_SWEEP_STIM)) {

```

```
pConfig=&m_shSignalHeader.map.config[m_realtime_index.stim_screen][m_realtime_index.stim_config];
```

```
        iValue=DecodeChannel(pConfig->neg_offset, pConfig->pos_offset);
```

```
        usInterval=PerformAnalysis(stim_analysis, iValue);
```

```
        CheckAutoRecordStim(stim_analysis, usInterval);
```

```
    } else {
```

```
        if (theApp.m_dLog->m_iSaveData) {
```

```
pConfig=&m_shSignalHeader.map.config[m_realtime_index.stim_screen][m_realtime_index.stim_config];
```

```
        iValue=DecodeChannel(pConfig->neg_offset,  
pConfig->pos_offset);
```

```
        usInterval=PerformAnalysis(stim_analysis, iValue);
```

```
    }
```

```
}
```

```
}
```

```
if (m_realtime_index.pressure_screen==-1) {
```

```
    //ClearTigaBP();
```

```
} else {
```

```
    if
```

```
(m_realtime_index.pressure_screen!=theApp.m_CathMapDoc.m_iRe  
alTimePageNumber) { // it wasnt done above
```

```
pConfig=&m_shSignalHeader.map.config[m_realtime_index.pressure  
_screen][m_realtime_index.pressure_config];
```

```
        iValue=DecodeChannel(pConfig->neg_offset, pConfig-  
>pos_offset);
```

```
        PerformAnalysis(pressure_analysis, iValue);
```

```
    }
```

```
}
```

```
        if(++m_usAcquirePop>=ACQUIRE_BUFFER_SIZE)  
{    m_usAcquirePop=0;    }  
  
        m_usDisplayPush=m_usAcquirePop;  
  
    }
```

Byrd Decl., Ex. 1067, Ex. Q (corresponding to Ex. 1132) at
MonitorDoc.cpp at 710-902.

546. One of the measurement processing components in the EP-WorkMate
system software was HolterDoc. HolterDoc was the processing component for
Holter data. I understand that Holter data is derived from electrodes, which are a

particular type of sensor. The HolterDoc processing component processed the sensor data. For example, the method HolterLoadPoll in HolterDoc conducted a variety of mathematical operations on the electrode-based sensor data. Byrd Decl., Ex. 1067, Ex. Q (corresponding to Ex. 1132) at HolterDoc.cpp at 719 (Ex 1067). The HolterDoc processing component then fed the data to the HolterView display component. For example, the method RefreshHolter in HolterDoc.cpp line 836 invoked the InvalidateView method, which in turn invoked the standard class method InvalidateRect, which triggered the framework to invoke the OnDraw method in the HolterView display component in HolterView.cpp at line 577. This rendered the data for display. Accordingly, the HolterDoc processing component facilitated receiving the sensor data of a particular sensor type and facilitated rendering the data for display.

547. Another of the measurement processing components was MonitorDoc. MonitorDoc was the processing component for pressure data, *see* method ProcessInputData in MonitorDoc.cpp at line 711, which was derived from pressure sensors. The MonitorDoc processing component separated sensor data according to the specific sensor type, performed necessary processing and updated the relevant document object that contained the specific data, and triggered an update event. This update event caused the MonitorView display component's OnPaint method to be invoked, *see* method OnPaint in MonitorView.cpp at line

856, which rendered the data for display. Accordingly, the MonitorDoc processing component facilitated receiving the sensor data of a particular sensor type and facilitates rendering the data for display.

548. I understand that parties agree that the term "rendering" means "processing data into a different format including but not limited to drawing." The EP-WorkMate system also processed diagnostic measurement parameter values into a different formats including but not limited to drawing. The diagnostic parameter values were processed, for example, into line graphs on the screens in the Holter window and in the other modes on the displays of the computer monitor that is part of the EP-WorkMate system.

549. Accordingly, the EP-WorkMate system practiced this limitation of claim 1.

550. The EP-WorkMate system included a multi-mode graphical user interface host comprising diagnostic measurement user interfaces including display components corresponding to data output rendered by specified ones of the plurality of measurement processing components.

551. As explained, the EP-WorkMate system had a human-computer interface with multiple distinct displays such as, for example, various system and diagnostic displays. For example, in the EP-WorkMate system, "[t]he user interface consist[ed] of two monitors performing separate tasks." Byrd Decl., Ex.

1067, Ex. F (corresponding to Ex. 1121) at SJM03071661. One of the monitors was referred to as the Real-Time Monitor, whereas the other was called the Review Monitor. Byrd. Decl. ¶ 9. Additionally, "[t]he EP-WorkMate main application can be broken down into three main windows: the Review window, the Database window and the Setup window. Each monitor could then have multiple windows." Byrd Decl., Ex. 1067, ¶ 9 & Ex. F (corresponding to Ex. 1121) at SJM03071665.

552. The multi-mode graphical user interface host in the EP-WorkMate system included diagnostic measurement user interfaces. For example, the Real-Time Monitor "displays signals continuously for the signals defined in the setup window." Byrd Decl., Ex. 1067, ¶ 10 & Ex. F (corresponding to Ex. 1121) at SJM03071665. The Review Monitor "is used to view stored signals, along with a log of events, the stimulator and a holter window." *Id.* Furthermore,

553. Selecting the **Monitor** button will display the real-time signals on the right most review window. . . . Selecting the **Setup** button will display the Setup screen. . . . Selecting the **Stim** button will display the stimulator window. . . . Selecting the **Holter** button will display the Holter window in a split screen fashion on the left or right side of the review monitor. . . . Pressing the left mouse anywhere in the Holter signal area will highlight the ECG with a small white box and display the corresponding ECG in the Review screen. . . . Selecting the **Split** button will display a second Review screen to the left of the initial Review screen.

. . . Selecting the **Cine** button will display the Cine window on the review screen. .

. . Selecting the RF button will display the RF window on the review screen.

554. Byrd Decl., Ex. 1067, Ex. F (corresponding to Ex. 1121) at SJM03071677.

555. The diagnostic measurement user interfaces in the EP-WorkMate system included display components corresponding to data output rendered by specified ones of the plurality of measurement processing components, as I explained in paragraphs 541-549, above. Therefore, it is apparent that the EP-WorkMate practiced this claim limitation.

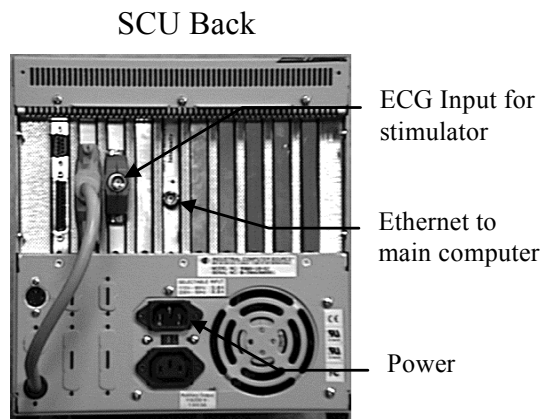
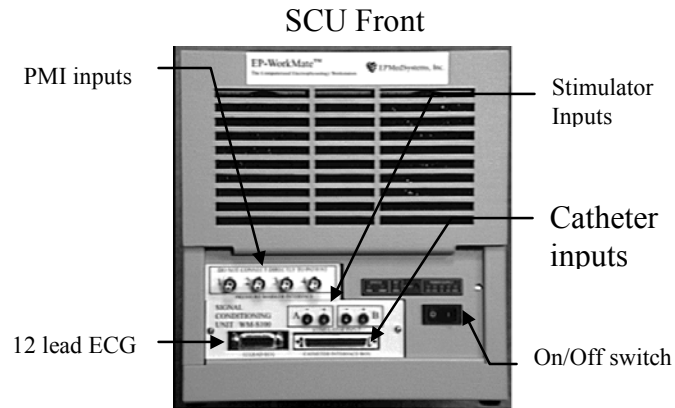
556. The term "multi-mode graphical user interface host" means "software that supports a graphical user interface (including windows or portions of windows) for presenting at least two types of diagnostic information (including within the same window)". The EP-WorkMate system practiced claim 1.

557. For example, in the diagram provided in paragraph 542, above, each of the class hierarchies terminated in a distinct visible document window that was displayed by the application, including CReviewView, CPrevReviewDoc, CHolterView, CMonitorView, CRfView, CCineView, and CLogView. Each of these was a separate display component object that was used to display information to the user.

558. The EP-WorkMate system included one or more kernel mode drivers that extract processed sensor data from a peripheral interface card providing a hardware interface for one or more invasive diagnostic measurement devices.

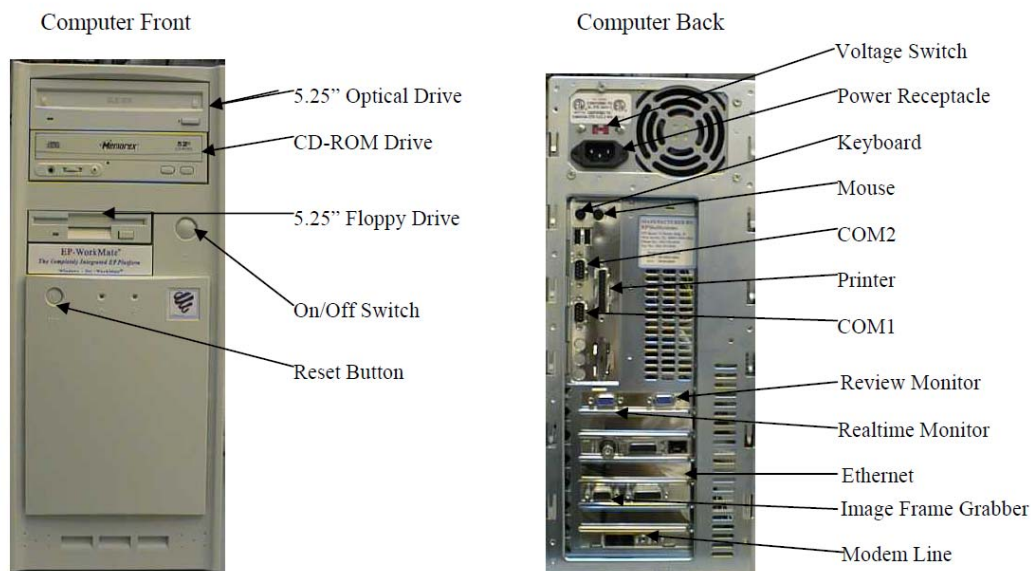
559. The EP-WorkMate system used one or more invasive diagnostic measurement devices as the basis of its sensor data. Byrd Decl., Ex. 1067, ¶¶ 4, 8. For example, there is a section in the EP-WorkMate User Guide entitled "Invasive Pressure Monitoring," which describes how to use the EP-WorkMate system for such measurements. See Byrd Decl., Ex. 1067, Ex. F (corresponding to Ex. 1121) at SJM03071707. The EP-WorkMate system also had as many as 120 catheter inputs. *Id.* at SJM03071731.

560. The EP-WorkMate system then processed the sensor data from the invasive diagnostic measurement devices. The EP-WorkMate system used two computers. The first computer, called the Signal Conditioning Unit ("SCU") (available in various versions depending on the number of inputs needed) had inputs for the various sensors, converted the signals to digital, and performed digital signal processing on the sensor signals. Byrd Decl., Ex. 1067, ¶ 3. Pictures of the front and back of the 72-channel version of the SCU, denoting the pressure inputs, follow:



Byrd Decl., Ex. 1067, Ex. F (corresponding to Ex. 1121) at SJM03071730.

The second computer was the central processing unit ("CPU"). Byrd Decl., Ex. 1067, ¶ 3 (Ex 1067).



Byrd Decl., Ex. 1067, Ex. F (corresponding to Ex. 1121) at SJM03071726.

561. The signals were sent from the SCU to the CPU via Ethernet, using the Ethernet connectors depicted on the preceding figures. Byrd Decl., Ex. 1067, ¶¶ 4-5.

562. The EP-WorkMate system also had a peripheral interface card, which provided a hardware interface for the pressure sensing guidewires. As depicted in the preceding diagram, the Ethernet connector was on a peripheral interface card. *Id.* The Ethernet card was the SMC EtherPower Ethernet card (model SMC 8432). Byrd Decl., Ex. 1067, ¶ 5.

563. The EP-WorkMate system also included one or more kernel mode drivers that extracted the processed sensor data from the Ethernet card. The EP-WorkMate system used two software components for this purpose. First, a device

driver that was supplied by the manufacturer of the Ethernet card extracted the processed sensor data from the Ethernet card to the CPU's memory. Byrd Decl., Ex. 1067, ¶ 6. I understand that the driver was the stock driver provided by SMC for use with the SMC 8432 card. Byrd Decl., Ex. 1067, ¶¶ 5 & 6 (Ex 1067).

564. The driver provided by SMC was a kernel mode driver. I obtained a Windows NT 4.0 version of the device driver that would have been current for the SMC 8432 Ethernet card at the time the EP-WorkMate system was first demonstrated in 1999 (smcpwrn.sys), which is provided as Ex. 1069. By analyzing the binary image in the distribution package, I determined that the portable executable header for the driver imports functions from the HAL and NDIS kernel mode components. Examples of these functions include READ_PORT_ULONG (from the Hardware Abstraction Layer component known as the HAL) and NdisMRegisterMiniport (from the Network Device Interface Specification component known as NDIS.) *See generally* Open Systems Resources, Inc., "READ_PORT_ULONG," http://www.osronline.com/ddkx/kmarch/k103_3pki.htm (Apr. 11, 2003) (*last accessed* April 17, 2013) (Ex. 1070) (describing READ_PORT_ULONG); Microsoft Developer Network, "NdisMRegisterMiniport function," <http://msdn.microsoft.com/en-us/library/windows/hardware/ff553602%28v=vs.85%29.aspx> (*last accessed* April

17, 2013 (Ex 1071) (describing NdisMRegisterMiniport, and noting that this function is used by drivers). Because the SMC 8432 device driver calls these kernel mode functions, it executes in the kernel mode.

565. Second, the EP-WorkMate system used the WINDIS Protocol Driver Library from Printing Communications Associates, Inc. to extract the processed sensor data from the CPU's memory into the EP-WorkMate main software application. Byrd Decl., Ex. 1067, ¶ 6. I have personal knowledge of and experience with the WINDIS Protocol Driver. The WINDIS Protocol Driver is a software component that operates in user mode and allows user mode applications to send and receive network communications messages via standard network devices such as Ethernet cards without requiring additional kernel mode programming.

566. Thus, the EP-WorkMate user mode software received processed sensor data from the SCU by first generating a data request in the CPU via the WINDIS Protocol Driver. The WINDIS Protocol Driver sent the data request message to the Ethernet card in the CPU, which transmitted the message to the SCU. The messages were received by the SCU, which sent a response message containing processed sensor data back to the Ethernet card in the computer. The processed sensor data was then extracted from the Ethernet card to the computer by the Ethernet device driver provided by SMC. The processed sensor data was then

extracted from the Ethernet device driver by the WINDIS Protocol Driver, and was finally extracted by the EP-Workmate user mode software.

567. The term "kernel mode drivers" means "software components that execute on the most privileged level in a computer, which permit a computer system to communicate with a device" The term "processed sensor data" means "sensor data that has been systematically modified, including by means of analog to digital conversion". A "peripheral interface card" is "a printed circuit board or adapter that can be plugged into a computer to provide added functionality or new capability, which provides specialized services that are not built into the computer, and that interfaces with peripherals."

568. The EP-WorkMate system practiced these limitations in claim 1. For example, the EP-WorkMate system applied appropriate mathematical operations to the sensor data. The SCU used mathematical operations in the process of quantizing the analog signals to digital. The SCU also processed the sensor data using mathematical operations after it had been converted to digital. Byrd Decl., Ex. 1067, ¶ 4.

569. The EP-WorkMate system also had code that facilitated communicating commands and data between interface hardware and a set of one or more user mode processes, as described above.

570. Accordingly, the EP-WorkMate system practiced each limitation of claim 1 of the '994 patent.

571. The EP-WorkMate system qualifies as prior art both under 35 U.S.C. §§ 102(a) & (b).

572. I understand from my discussions with counsel that a device is prior art under 35 U.S.C. § 102(b) if it was in public use or on sale in the United States more than one year prior to the date of the application for patent. Accordingly, a device would be prior art under § 102(b) if it practiced each limitation of claim 1 of the '994 patent and was in public use or on sale in the United States prior to May 20, 2001.

573. I also understand from my discussions with counsel that a device is prior art under 35 U.S.C. § 102(a) if it practices each limitation of the patent and was known or used by others in the United States or was patented or described in a printed publication in the United States or a foreign country before the invention was made.

574. The EP-WorkMate system was on sale in the United States prior to May 20, 2001. It was also known and used by others in the United States prior to May 20, 2002. EP-MedSystems launched the Windows NT version of the EP-WorkMate system in May 2000, at the 21st Annual Scientific Sessions held by the

North American Society of Pacing and Electrophysiology in Washington D.C.

Byrd Decl., Ex. 1067, ¶ 11 & Exs. B & C (corresponding to Exs. 1117-1118).

575. EP MedSystems started offering the Windows NT-based version of the EP-WorkMate system for sale starting in 2000. For example, EP MedSystems attended multiple conferences with the purpose of selling the systems, including the American Heart Association conference in November 2000, the American College of Cardiology conference in March 2001, and the North American Society of Pacing and Electrophysiology conferences in May 2000 and 2001. At those conferences, EP MedSystems had a booth at which an EP-WorkMate system running on Windows NT was set up for viewing. EP MedSystems personnel also spoke with U.S.-based doctors who were interested in acquiring the system, which resulted in subsequent quotes for purchase. Byrd Decl., Ex. 1067, ¶ 12 (Ex 1067).

576. When potential buyers asked for price quotes for the EP-WorkMate system, EP MedSystems provided price quotes that included upgrades to version 3.0. For example, a price quote for an EP-WorkMate system given to St. Mary of Nazareth Hospital in Illinois on March 23, 2000 quoted a price including "Windows NT Upgrade (when Available)." See Byrd Decl., Ex. 1067, Ex. D (corresponding to Ex. 1119). A price quote for an EP-WorkMate system given to Metro Health Medical Center in Ohio eight days later included "Windows NT Upgrade (when Available)" and noted "System is 'Windows-Ready.'" *Id.* A price

quote for an EP-WorkMate system given to St. Joseph's Hospital in California on July 6, 2000 included "Windows software provided free of charge when available" *Id.*

577. In 2001, EP MedSystems published a full-color, multipage brochure, which was used for advertising version 3.0 of the EP-WorkMate system to U.S.-based doctors and hospitals. Byrd Decl., Ex. 1067, ¶ 12 & Ex. E (corresponding to Ex. 1120).

578. EP-WorkMate systems with version 3.0 software first started shipping in April 2001. Byrd Decl., Ex. 1067, ¶ 11. By March 31, 2002, EP MedSystems had sold substantial quantities of version 3.0 EP-WorkMate systems. EP-MedSystems' Form 10-QSB for the quarterly period ended March 31, 2002, as filed with the United States Securities and Exchange Commission ("SEC"), stated,

579. Net sales were \$2,989,000 for the three months ended March 31, 2002 as compared to \$2,068,000 for the comparable period in 2001. This increase is the result of a 47% increase in the number of EP-WorkMates(R) sold due to continued market acceptance of our new NT Platform coupled with improved sales efforts in our domestic market.

580. EP Medsystems, Inc., Quarterly Report (Form 10-QSB), at 13 (Mar. 31, 2002) (*available at*

<http://www.sec.gov/Archives/edgar/data/1012394/000100547702002206/d02-37396.txt>) (last visited April 17, 2013) (Ex. 1068).

581. Accordingly, the EP-WorkMate system was in public use or was on sale in the United States before May 20, 2001, and was known or used by others in the United States prior to May 20, 2002. Because the EP-WorkMate system practiced each limitation in claim 1 of the '994 patent, claim 1 of the '994 patent is invalid as anticipated under 35 U.S.C. § 102.

X. Conclusion

582. For the reasons stated herein, it is my opinion that the '994 patent is invalid as both anticipated and obvious.

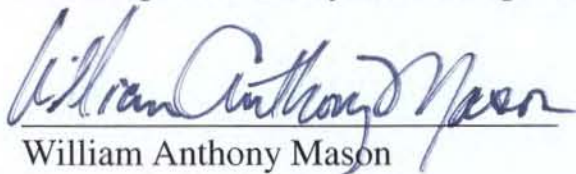
A. Possible Modifications or Supplementations to Analyses or Opinions

583. This declaration represents my opinions to date regarding the matters set forth above. As additional evidence becomes available, I may find it appropriate to revise or supplement my analysis, opinions, and/or conclusions. Thus, I reserve the right to modify or supplement this opinion as additional information becomes available to me.

584. I have been advised by counsel that I may be asked to review additional documents, including analyses done by any person retained by the patent owner to render an opinion in this case.

585. I further reserve my right to use summary or other demonstrative exhibits at a hearing or trial in this matter.

586. I hereby declare under penalty of perjury under the laws of the United States of America that the foregoing is true and correct, and that all statements made of my own knowledge are true and that all statements made on information and belief are believed to be true. I understand that willful false statements and the like are punishable by fine or imprisonment, or both (18 U.S.C. § 1001).


William Anthony Mason

Date: 29 April 2013